

THE UNIVERSITY OF BAMENDA

**NATIONAL HIGHER
POLYTECHNIC
INSTITUTE**



**CENTRE FOR
CYBERSECURITY
AND
MATHEMATICAL
CRYPTOLOGY**

**PAIRING ARITHMETIC AND PERFORMANCE ANALYSIS
OF zk-SNARK PROTOCOLS FOR PRIVACY AND
SCALABLE BLOCKCHAIN SYSTEMS**

A dissertation submitted to the Centre for Cybersecurity and Mathematical Cryptology
in the National Higher Polytechnic Institute of The University of Bamenda in partial
fulfillment of the requirements for the award of a Master of Science Degree in
Cybersecurity and Mathematical Cryptology

BY:

ABDOUL-AZIZ SOUAIBOU
REGISTRATION NUMBER: UBA24EP003
(B.Sc. in Pure Mathematics)

SUPERVISORS:

Supervisor

Fouotsa Emmanuel, PhD
(Professor)
(The University of Bamenda)

Co-Supervisor

Azebaze Guimagang Laurian, PhD

JULY, 2026

© Copyright by **ABDOUL-AZIZ SOUAIBOU**, 2026.
All rights reserved

DECLARATION OF ORIGINALITY OF STUDY

I, ABDOUL-AZIZ SOUAÏBOU, registration N^o : UBa24EP003, in the Department of Centre For Cyberecurity And Mathematical Cryptology, National Higher Polytechnic Institute, The University of Bamenda hereby declare that this work titled "Pairing Arithmetic and Performance Analysis of zk-SNARK Protocols for Privacy and Scalable Blockchain Systems" is my original work. It has not been presented in any application for a degree or any academic pursuit elsewhere. I have acknowledged all borrowed ideas nationally and internationally through citations.

Date: _____

Signature of author: _____

CERTIFICATION OF CORRECTIONS AFTER DEFENCE

This is to certify that this dissertation titled "Pairing Arithmetic and Performance Analysis of zk-SNARK Protocols for Privacy and Scalable Blockchain Systems" is the original work of ABDOUL-AZIZ SOUAÏBOU. This work is submitted in partial fulfilment of the requirements for the award of a master of Science Degree in Cybersecurity and Mathematical Cryptology in the National Higher Polytechnic Institute of The University of Bamenda, Cameroon.

Supervisor: _____

Prof. Fouotsa Emmanuel

Co-Supervisor: _____

Dr. Azebaze Guimagang Laurian

Head of Division: _____

Prof. Fouotsa Emmanuel

Director: _____

Prof. Fidelis Cho-Ngwa

ACCEPTANCE OF DISSERTATION

Having met the stipulated requirements, this dissertation entitled “Pairing Arithmetic and Performance Analysis of zk-SNARK Protocols for Privacy and Scalable Blockchain Systems” has been accepted by the Postgraduate School of The University of Bamenda in partial fulfilment of the requirements for the award of a Master of Science Degree in Cybersecurity and Mathematical Cryptology (Option:Algebra, Coding Theory, Cryptology and Cybersecurity).

Chairperson, Master’s Dissertation Committee : _____

Prof.

Date: _____

ABSTRACT

Scalability is a major challenge for current blockchains, hindering their mass adoption. zk-SNARK proof systems specifically Groth16 address this challenge by enabling the verification of thousands of transactions via a single validity proof submitted to the main chain. However, the performance of these systems depends directly on the choice of elliptic curve and protocol. Existing literature lacks sufficient quantitative, comparative analysis regarding how this choice affects the entire pipeline and how different zk-SNARKs compare across elliptic curves. This thesis fills that gap. We analyze the privacy and integrity guarantees provided by Groth16 and their impact on performance using arithmetic benchmarks (Miller loop, final exponentiation, MSM) performed with the Arkworks library (Rust, release mode) on BN254 and BLS12-381 curves. We then deploy three protocols (Groth16, Marlin, Halo2) within a Layer 2 architecture, using circuit sizes ranging from 512 to over 4 million constraints, to measure and compare the performance of the two curves. We compare the performance of the aforementioned protocols across both curves. Some results of our work show that verification time and proof size remain constant for each protocol regardless of circuit size, validating the succinctness required for scalability. The computational cost of the full pairing operation is nearly identical for both curves ($\approx 1706\mu s$); however, BLS12-381 proves slower for most other arithmetic operations, resulting in a measured overhead for the complete Groth16 pipeline (up to $1.76\times$). No single curve or protocol is universally optimal; performance depends on the environment and specific requirements. These results highlight that the choice of a protocol and a curve should be guided by empirical evaluation rather than solely by theoretical parameters.

Keywords: privacy, scalability, Blockchain, Groth16, Marlin, Halo2, zk-Rollups.

RÉSUMÉ

Parmi les principaux défis des blockchains actuelles figure la scalabilité, qui freine leur adoption à grande échelle. Les systèmes de preuve zk-SNARK, et Groth16 en particulier, répondent à ce défi de scalabilité en permettant la vérification des milliers de transactions grâce à une seule preuve de validité, soumise à la chaîne principale. Cependant, la performance de ces systèmes dépend directement du choix de la courbe elliptique et du protocole. La littérature existante documente insuffisamment, de manière quantifiée et croisée, l'impact de ce choix sur l'ensemble du pipeline et la comparaison des zk-SNARKs, relativement aux courbes elliptiques. Dans ce travail nous analysons les garanties de confidentialité et d'intégrité fournies par Groth16 ainsi que leur impact sur la performance, à l'aide de benchmarks arithmétiques (boucle de Miller, exponentiation finale, MSM) réalisés avec la bibliothèque Arkworks (Rust, mode release) sur les courbes BN254 et BLS12-381. Nous déployons ensuite les trois protocoles (Groth16, Marlin, Halo2) dans une architecture de Layer 2, avec des échelles de circuits allant de 512 à plus de 4 millions de contraintes, puis mesurons et comparons la performance des deux courbes. Nous comparons la performance des protocoles susmentionnés sur les deux courbes. Les résultats de notre expérimentation montrent que le temps de vérification et la taille de la preuve restent constants pour chaque protocole, quelle que soit la taille du circuit, validant la succincité recherchée pour la scalabilité. Le coût computationnel de l'accouplement complet est presque identique entre les deux courbes ($\approx 1706 \mu s$), bien que BLS12-381 se révèle plus lente pour la plupart des autres opérations arithmétiques, avec un surcoût mesuré sur le pipeline Groth16 complet (jusqu'à $1,76\times$). Aucune courbe ni aucun protocole n'est universellement optimal. La performance dépend de l'environnement et de la priorité du besoin. Ces résultats soulignent que le choix d'un protocole et d'une courbe devrait être guidé par une évaluation empirique plutôt que par les seuls paramètres théoriques.

Mots-clés: confidentialité, scalabilité, Blockchain, Groth16, Marlin, Halo2, zk-Rollups.

DEDICATION

To my mother AISSATOU, who left us too soon & to my father SOUAIBOU for the values and the love that made this work possible.

To my grandmother ROUKAYATOU, and my whole family, GODJE and BAHAJARA, I dedicate this work.

ACKNOWLEDGEMENTS

First of all, I would like to thank my main supervisor Prof. FOUOTSA EMMANUEL, for allowing me to be one of his students, one of those who will follow in his footsteps in the field of cryptography. I thank him for his availability, guidance, advice, and patience. He remains a role model for me, and I would like to follow in his footsteps.

I also thank Dr. AZEBAZE GUIMAGANG LAURIAN, for his support and advice, which greatly helped me in completing this work.

I must acknowledge the tireless work, dedication, guidance, rigor, and discipline of Mr. GUSTAVE TCHOFFO SAAH, which served as the guiding light for this project. Thank you so much, Sir.

I thanks my uncles ADAMOU, AMADOU, Elh. HAMADOU, my older brother MOUHAMADOU BACHIROU for their moral, financial and other support.

I also thank ARES for their support of this program, which allowed students to be put in the best possible study conditions.

I would like to thank the entire staff of the Centre for Cybersecurity and Mathematical Cryptology (CCMC) at the University of Bamenda, precisely under the control of NAHPI, for the training I received and for the material support specifically, providing me with a computer that I used throughout the year.

I also want to thank my parents, uncles, and aunts for their emotional, material, and financial support.

To my classmates: I cannot conclude without thanking you. Together, we learned, shared, discussed, stayed up late working, and grew together. Thank you for those wonderful moments we shared.

And how could I finish without thanking the One who made all of this possible! Oh Allah, all praise belong to You.

TABLE OF CONTENTS

COPYRIGHT PAGE	i
DECLARATION OF ORIGINALITY OF STUDY	ii
CERTIFICATION OF CORRECTION AFTER DEFENCE	iii
ACCEPTANCE OF DISSERTATION	iv
ABSTRACT	v
DEDICATION	vii
ACKNOWLEDGEMENTS	viii
LIST OF TABLES	xii
LIST OF FIGURES	xiv
LIST OF ABBREVIATIONS	xv
LIST OF SYMBOLS	xvii
1 INTRODUCTION	1
1.1 Background of the Study	1
1.2 Statement of the Problem:	3
1.3 Rationale	3
1.4 Research Questions	4
1.4.1 Main Research Question	4
1.4.2 Specific Research Questions	4
1.5 Objectives	4
1.5.1 Overall Objective	4
1.5.2 Specific Objectives	5
1.6 Significance of the Study	5
1.7 Scope of the Study	5
1.8 Structure of the Dissertation	6
2 LITERATURE REVIEW	7
2.1 Conceptual Literature Review	7
2.1.1 Definition of Key Concepts	7

2.1.2	Contextual Background	10
2.2	Theoretical Literature Review	11
2.2.1	Elliptic Curve Theory	11
2.2.2	Bilinear Pairing Theory	11
2.2.3	Zero Knowledge Proof Theory	12
2.2.4	QAP (Quadratic Arithmetic Program) Theory	12
2.3	Empirical Literature Review	12
2.3.1	Review of Previous Studies	13
2.3.2	Identification of Gaps	17
3	MATERIALS AND METHODS	19
3.1	Blockchain	19
3.1.1	Block Structure and Chaining Mechanism	19
3.1.2	Consensus Mechanisms: Proof of Work and Proof of Stake	20
3.1.3	The Blockchain Trilemma	21
3.1.4	The Scalability Crisis: Quantification	22
3.1.5	The Confidentiality Crisis	22
3.1.6	Zero-Knowledge Proofs as a Solution	23
3.2	Mathematical Preliminaries	25
3.2.1	Elliptic Curves over Finite Fields	25
3.2.2	Pairing Groups and Security Assumptions	28
3.2.3	Miller’s Algorithm	30
3.2.4	Final Exponentiation: Four Methods Compared	31
3.2.5	Pairing-Friendly Curves: BN254 and BLS12-381	33
3.3	Rigorous Formal Analysis of Groth16	34
3.3.1	From Arithmetic Circuits to Quadratic Arithmetic Programmes	34
3.3.2	The Groth16 Protocol Diagram	44
3.4	BLS signature	44
3.4.1	Mechanism	44
3.4.2	BLS contribution in Blockchain scalability	46
3.5	ZK-Rollups Pipeline	47
3.5.1	Cost Model and Scalability Analysis	47
3.6	Experimental Setup	49
3.6.1	Benchmark Circuit	49
3.7	Software Environment Specifications and Circuit Configuration	50
3.7.1	Detailed Specifications of the Hardware and Software Environment	50
3.7.2	Circuits	51
4	RESULTS AND DISCUSSION	52
4.1	Results	52

4.1.1	Zk-SNARKs (Groth16): Data Confidentiality and Integrity Analysis	52
4.1.2	Layer 2 Architecture Deployment and Scalability	53
4.1.3	Baseline Analysis: Groth16 Arithmetic Analysis, Curve Selection and Analysis	54
4.1.4	Large-Scale Multi-Protocol Global Comparative Analysis	55
4.2	Discussion	59
4.2.1	Confrontation with Literature	61
4.2.2	Limitations of the Study	61
4.2.3	Implications	62
5	CONCLUSION AND RECOMMENDATIONS	63
5.1	Conclusion	63
5.2	Summary of Findings	64
5.3	Difficulties Encountered	64
5.4	Recommendations	65
5.4.1	For the Direct Continuation of this Work	65
5.4.2	For the Community and Practice	65
5.5	Self-analysis	65
	REFERENCES	67
	APPENDIX	72
	Appendix A ADDITIONAL MATERIALS	72
A.1	Algebraic Geometry Concepts	72
A.2	Proofs	85
A.2.1	Arithmetic in Extension Fields $\mathbb{F}_p^4$ and $\mathbb{F}_p^6$	89
A.3	Practices	93

LIST OF TABLES

Table 2.1	Digital Signature Schemes Classified by Their Difficulty Based (Johnson et al., 2001)	10
Table 3.1	Concrete examples of trade-off between scalability, decentralization, and security in blockchain.	21
Table 3.2	Comparison of transaction throughput, finality time, and cost across different blockchain networks. TPS=Transaction Per Second, CPT=Cost per Transaction	22
Table 3.3	Hierarchy of the Security Assumptions Used in This Thesis . . .	30
Table 3.4	Comparison of pairing methods	32
Table 3.5	Comparison of the computational complexities of couplings on different families of curves (extract from TABLE 4.10 of (Ghammam, 2016))	33
Table 3.6	Coefficients of the matrix L	38
Table 3.7	Rust compiler and Arkworks version	49
Table 3.8	Hardware Specifications of the Experimental Computer	50
Table 4.1	Benchmark Groth16 - Setup, Prove, Verify on BN254 and BLS12-381	52
Table 4.2	Groth16 Benchmark - arkworks - circuit: $x^2 - 2x - 15 = 0$ (solutions: $x = 5, x = -3$) - Phase 1: Elementary operations	54
Table 4.3	Groth16 Benchmarks on BN254 and BLS12-381 (arkworks) R1CS=R1CS Constraints, Setup=Global SRS Setup, Wgen=Witness Generation, PT=Prover Time, VT=Verification Time, PS=Proof Size	55
Table 4.4	Marlin Benchmarks on BN254 and BLS12-381 (arkworks) R1CS=R1CS Constraints, Setup=Global SRS Setup, Wgen=Witness Generation, PT=Prover Time, VT=Verification Time, PS=Proof Size	56
Table 4.5	Halo2 (PSE) Benchmarks on BN254 and BLS12-381 PLONK-Cts=PLONKish Constraints, Setup=Global SRS Setup (KZG), Wgen=Witness Generation, VKey=verifier key generation, PKey=Prover Key generation, PT=Prover Time, VT=Verification Time, PS=Proof Size	57
Table 4.6	Prover Time and Proof Size Across Protocols and Curves	57
Table A.1	Local uniformizers on an elliptical curve	76
Table A.2	Orders of some rational functions in P_∞	84
Table A.3	Recapitulatif table	84
Table A.4	Synthetic Table: Operation Costs in Extension Fields	91

Table A.5	Comparison of the pairing-friendly curves BN254 and BLS12-381	92
Table A.6	Mathematical parameters of BN254 and BLS12-381	92
Table A.7	Comparison of different curves favorable to pairings.	93

LIST OF FIGURES

Figure 1.1	Satoshi Nakamoto, Bitcoin founder	1
Figure 1.2	Relationship between blockchain transparency, privacy, zero-knowledge proofs, and scalability solutions.	3
Figure 2.1	Blockchain as transparent and decentralized encrypted data sharing	8
Figure 2.2	Relations between ZKP, NIZK and zk-SNARK, (Liang et al., 2025)	9
Figure 3.1	Block structure and chaining mechanism in a blockchain.	20
Figure 3.2	Blockchain Trilemma	22
Figure 3.3	Without ZKP: Eve freely observes the transaction data between Alice and Bob.	23
Figure 3.4	With ZKP: Eve is blocked. ZKP guarantees data confidentiality	24
Figure 3.5	Geometric construction of $P + Q$ on $E : y^2 = x^3 - x + 1$. The red dashed line meets the curve at R ; reflecting R across the x -axis gives $P + Q$	26
Figure 3.6	Miller's algorithm execution	31
Figure 3.7	Link between $P \subseteq NP \subseteq PSPACE$	35
Figure 3.8	Groth16 Pipeline Process	45
Figure 3.9	ZK-Rollup Transaction Process, illustrating where zk-SNARKs is used in Blockchain	48
Figure 4.1	Groth16's benchmark times comparison BN254 Vs BLS12-381	56
Figure 4.2	Marlin (arkworks) benchmark results comparing BN254 and BLS12-381	56
Figure 4.3	Halo2 (PSE) benchmark results comparing BN254 and BLS12-381	57
Figure 4.4	Trade-off between proof size and prover time (at $k=22$) across Groth16, Marlin, and Halo2	58
Figure A.1	This is a representation of an elliptic curve with a twist point of order 6, because $6P = O$: the point at infinity.	74
Figure A.2	Extension Tower	90

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
BLS	Boneh–Lynn–Shacham (Signature / Curve family)
BN254	Barreto–Naehrig 254-bit curve
CDH	Computational Diffie–Hellman
co-CDH	co-Computational Diffie–Hellman
CPU	Central Processing Unit
CRS	Common Reference String
DLP	Discrete Logarithm Problem
ECC	Elliptic Curve Cryptography
ECDH	Elliptic Curve Diffie–Hellman
ECDLP	Elliptic Curve Discrete Logarithm Problem
FFT	Fast Fourier Transform
GKR	Goldwasser–Kalai–Rothblum (Protocol)
GMR	Goldwasser–Micali–Rackoff (Protocol)
GPU	Graphics Processing Unit
GSM	Global System for Mobile Communications
Halo2	PLONKish proof system without trusted setup
IP	Interactive Proof
IPA	Inner-Product Argument
kGas	kilo-Gas (Ethereum computational cost unit)
KZG	Kate–Zaverucha–Goldberg (Polynomial Commitment)
L1	Layer 1 (Base blockchain layer)
L2	Layer 2 (Execution/Scaling layer)
MSM	Multi-Scalar Multiplication
NFS	Number Field Sieve
NIZK	Non-Interactive Zero-Knowledge Proof
NP	Nondeterministic Polynomial (Complexity class)
NTT	Number Theoretic Transform
NVMe	Non-Volatile Memory Express

OS	Operating System
PCP	Probabilistically Checkable Proof
PLONK	Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge
PoS	Proof of Stake
PoW	Proof of Work
PSE	Privacy & Scaling Explorations
PSPACE	Polynomial Space (Complexity class)
QAP	Quadratic Arithmetic Program
q-PKE	q-Power Knowledge of Exponent (Assumption)
R1CS	Rank-1 Constraint System
RAM	Random Access Memory
RSA	Rivest–Shamir–Adleman
SATA	Serial Advanced Technology Attachment
SHA-256	Secure Hash Algorithm 256
SNARK	Succinct Non-interactive Argument of Knowledge
SSD	Solid State Drive
TPS	Transactions Per Second
ZKP	Zero-Knowledge Proof
zk-SNARK	Zero-Knowledge Succinct Non-interactive Argument of Knowledge

LIST OF SYMBOLS

k	Base field
$\overline{k}$	Algebraic closure of k
$\mathbb{F}_q$	Finite field with $q = p^n$ elements (p prime)
$\mathbb{Z}/r\mathbb{Z}$	Cyclic group of order r
$k[x_1, \dots, x_n]$	Polynomial ring in n variables over k
I	Ideal of a polynomial ring
$V(S)$	Affine variety defined by the set S
$\mathcal{O}(1), \mathcal{O}(n)$	Asymptotic complexity notation (Big O)
E	Elliptic curve
Δ	Discriminant of an elliptic curve
$\mathcal{O}$	Point at infinity (identity element of the group $(E, +)$)
λ	Slope of the line in point addition or doubling formulas
μ_r	Group of r -th roots of unity in $\mathbb{F}_{p^{12}}^*$
$\Phi_{12}(p)$	12-th cyclotomic polynomial evaluated at p
u	Integer parameter defining BN or BLS curve families
Φ	Arithmetic circuit
$\mathcal{X}$	Set of variables in an arithmetic circuit
w	Witness vector
$\circ$	Hadamard product (entry-wise multiplication)
$\ell_i(X)$	Lagrange basis polynomial
τ	Secret evaluation point in the Structured Reference String (SRS)
$\alpha, \beta, \gamma, \delta$	Secret scalar parameters in a trusted setup
σ	Cryptographic signature
$\square$	End of proof marker (QED)
$\stackrel{?}{=}$	Conditional equality check verification operator

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

Since the founding article of Satoshi Nakamoto (See 1.1) in 2008, (Nakamoto, [2008](#)) in response to the subprime economic crisis (financial crisis that occurred in the United States between 2007 and 2010, leading to the bankruptcy of numerous companies and requiring government assistance for their recovery), public blockchains are based on a fundamental principle: distributed trust without central authority is possible, provided that each participant



Figure 1.1: Satoshi Nakamoto, Bitcoin founder

can verify the integrity of the system independently. This principle is mathematically attractive, and provides a robust solution to the transactional limitations that we had with banks. But at the same time, it carries within itself a contradiction: the transparency necessary for public verification and the confidentiality of users. A financial transaction, a health insurance contract, a judicial record, an electronic vote: none of these application cases can tolerate that its data be accessible by any node (user) in a decentralized network. Hiding this data without a verification mechanism would destroy the very main reason for the existence of the blockchain (Zheng et al., [2018](#)). This dilemma between privacy and transparency does not come from a bad design or poorly designed software, because even when utilizing industry-leading cryptographic and software development expertise, it will remain. It is indeed mathematical and deep. How to prove the validity of an assertion without revealing the assertion itself ? Particularly, how to show that a transaction (amount, balance, identity, ...) is valid without revealing its data to the verifier, while he must know for example the identity, verify the sufficiency of the balance, and so on ? This question mobilized the best cryptographers in the world since the founding works of Shafi Goldwasser, Silvio Micali and Charles Rackoff in 1985, up to contemporary constructions: Groth16, PLONK, which today advance blockchains such as zkSync and Zcash.

The cryptography is used to protect data and to prove that a person is indeed authorized to perform an action. In asymmetric cryptography, there exist several methods such as RSA, lattice-based cryptography, ECC, and so on, to provide such security guarantees. Today, elliptic curve cryptography has become dominant: a 128-bit security level requires an ECC key of 256 bits compared to 3072 bits for RSA. These small and efficient keys ECC translate directly into an advantage in bandwidth, computation time, and energy cost for decentralized systems, which generate millions of proofs and signatures daily. Studies have shown that the vast majority of transactions on public blockchains remain traceable back to their real issuer, despite the use of pseudonymity (Feng et al., 2019). Naive attempts at privacy (transaction mixing, CoinJoin) have been compromised by graph analysis (Meiklejohn et al., 2013). The Goldwasser–Micali–Rackoff method offers a true confidentiality guarantee (Goldwasser et al., 1989).

Furthermore, the Bitcoin network processes 7 transactions per second, while Ethereum processes 15 transactions per second. In contrast, the private Visa network processes 24,000 per second (VoD, 2021), (Thibault et al., 2022b). This slowness of the both first (public) blockchains is caused by the consensus protocol, which requires that the majority of participants verify each transaction. Zk-Rollups came with a solution by grouping thousands of transactions off-chain, producing a single zk-SNARK proof of their validity, and submitting this proof on-chain for verification in $O(1)$ (Thibault et al., 2022b). These two solutions for confidentiality and scalability both rely on bilinear pairings (and the proof systems that depend on them). The choice of curve and the optimization of pairings are therefore cryptographic parameters with a direct impact on the performance and security of these systems, which manage billions of dollars in assets.

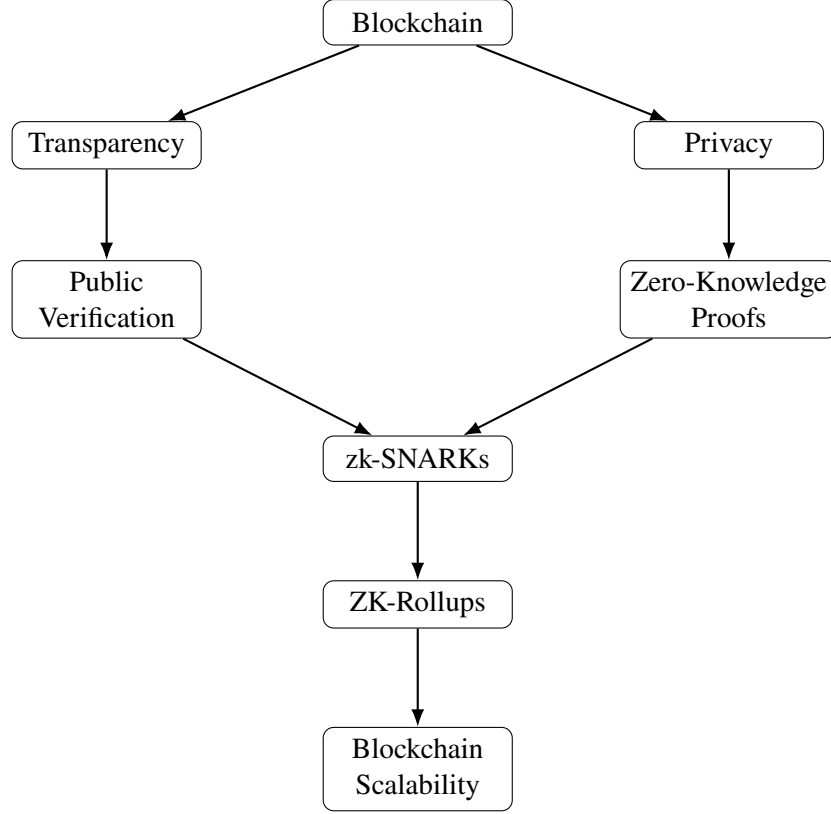


Figure 1.2: Relationship between blockchain transparency, privacy, zero-knowledge proofs, and scalability solutions.

1.2 Statement of the Problem:

The central problem of this dissertation is the following: The arithmetic of bilinear pairings constitutes the computational support of zk-SNARK systems and zk-Rollups. However, the existing studies do not establish in both the empirical comparison of curves (BN254, BLS12-381) and protocols.

1.3 Rationale

The justification of this work is organized around three major converging ideas. On the scientific level, the existing literature on zk-SNARKs and blockchains presents a persistent fracture between the applicative works which use these systems as black boxes (the verifier receives a proof and knows that it is true, but sees neither the sensitive data, nor the detail of the intermediate steps) and the theoretical works which construct them without implementing them in the concrete requirements of blockchains. In the literature, (Liang et al., 2025) formally document this gap between research and practice, and identify pairing arithmetic as the unanalysed foundation of the majority of practical proof systems. Filling this gap through a fine analysis and a complete formal construction would constitute an original scientific contribution.

On the technical level, production blockchains manage billions of dollars in assets

and process millions of transactions every day whose security depends directly on the correctness of the underlying systems (SNARKs). The absence of rigorous formalization of these systems exposes their deployments to vulnerabilities as highlighted by (Liang et al., 2025) through their analysis of existing libraries.

On the academic and contextual level, the development of African expertise in advanced cryptography is a documented necessity in pan-African research discussions on blockchain and AI (African Union, 2020), (GSMA Intelligence, 2023). This work contributes to this effort by producing a rigorous analysis of the mathematical foundations of zk proof systems, applicable to the contexts of decentralized digital finance and sovereign identity which interest African economies, where Mobile phones and decentralized digital identity systems can help millions of unbanked people use modern financial services (Demirgüç-Kunt et al., 2022).

1.4 Research Questions

1.4.1 Main Research Question

Given that, within the scope of this study, the ZKPs studied are those based on elliptic curves, the fundamental question that emerges from this subject is the following: How can we reconcile the absolute confidentiality of ZKPs offered in theory with the implementation and scalability challenges in real blockchain applications? In simpler terms, how can we use ZKPs in real blockchains while maintaining a good level of privacy and performance?

1.4.2 Specific Research Questions

The main question can be split in the following specifics questions:

1. How can zk-SNARK (Groth16) guarantee data confidentiality and integrity in the blockchain while maintaining acceptable system performance (scalability)?
2. How can zk-SNARK solutions be deployed and optimized in layer 2 architectures to significantly improve blockchain scalability?
3. What can be the performance difference between BN254 and BLS12-381 across the entire Groth16 pipeline?
4. Which possible combination of protocol (Groth16, Marlin, Halo2) and elliptic curve (BN254, BLS12-381) offers the best compromise between confidentiality and scalability in the real-world industrial context of blockchain deployment?

1.5 Objectives

1.5.1 Overall Objective

The main objective of this work is to analyse and evaluate cryptographic mechanisms based on elliptic curves, particularly zk-SNARKs such as Groth16, to determine how they achieve a balance between confidentiality, integrity and scalability in real blockchain

applications.

1.5.2 Specific Objectives

The specific objectives of this work are the following:

1. Analyze how zk-SNARK (Groth16) guarantee the confidentiality and integrity of data in the blockchain and evaluate their impact on system performance.
2. Investigate the deployment of zk-SNARKs in Layer 2 architectures to improve blockchain scalability.
3. Using Groth16 protocol, measure and compare performance of BN254 and BLS12-381.
4. To experimentally evaluate the comparative performance of Groth16, Marlin and Halo2, using BN254 and BLS12-381 curves to determine their suitability for real-world industrial requirements in terms of confidentiality and scalability.

1.6 Significance of the Study

This work presents significance at several levels. On the theoretical level, this dissertation establishes a continuous causal chain between elementary arithmetic in $\mathbb{F}_{p^{12}}$ and the measurable performances of blockchain systems in production. This chain constitutes a contribution to applied mathematical cryptography notably DLP, low group. On the practical level, the results of this work have a direct applicability to Blockchain systems in production. The identification of the optimal algorithms for each curve provides proof system developers with a basis for their implementation choices. The formal analysis of the ZK-Rollup pipeline provides a rigorous framework for evaluating existing scalability architectures and in order to design new solutions.

On the academic level, this dissertation contributes to the development of expertise in advanced cryptography in the African context, by providing a reference work on the mathematical foundations of zk-SNARKs accessible to researchers and engineers of the region.

1.7 Scope of the Study

This study concentrates on zk-SNARK proof systems based on ECC, specifically Groth16, evaluated on BN254 and BLS12-381 curves. The analysis of pairings is limited to the reduced Tate pairing and the optimal Ate pairing, which are the pairings most used in the practice.

We deliberately excluded the following aspects from the scope of this work: zk-STARK systems, which rely on hash functions and not on pairings, the post-quantum resistance of pairings (an important open problem linked to the vulnerability of Shor's algorithm) is mentioned as a perspective but not treated in this dissertation, the complete software

implementation of the analysed systems. Some of the aspects are not studied because of the lack of time. However, more details are provided in the appendix for some concepts.

1.8 Structure of the Dissertation

We have organized this dissertation into five chapters. In Chapter 1 we present the general introduction, including the context, problem statement, research questions and objectives, as well as the justification and scope of this work.

The Chapter 2 titled literature review, we structure it into conceptual, theoretical, and empirical reviews, and we conclude it with the formal identification of the four gaps addressed by this dissertation.

After then in Chapter 3, we present the materials and methods used, including the necessary mathematical preliminaries and the methodology to be adopted in order to achieve our four specific objectives.

Chapter 4 concerns the results and their discussion, which we have organized into four sections corresponding to the four specific objectives.

In the last one, titled conclusions and recommendations, we synthesize the contributions of our work and proposing directions for future generations of research.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews the literature relevant to this work along three complementary axes: the conceptual literature review, the theoretical literature review and the empirical literature review.

2.1 Conceptual Literature Review

This section establishes the key definitions and the contextual background that motivate this study, situating the problem of scalability and confidentiality in public Blockchain

2.1.1 Definition of Key Concepts

Definition 2.1. (*Blockchain*) Blockchain is defined as a decentralized, transparent system through which encrypted data circulates. This highlights the paradox mentioned in the introduction: transparency and confidentiality, which will be the focus of our analysis.

In a blockchain system, each record called a block contains a cryptographic fingerprint (hash) of the block that precedes it, a timestamp, and a set of transactions. This chaining of blocks by cryptographic hashing makes the system tamper-proof, because the modification of one block invalidates all the blocks that follow it. This therefore makes the chain immutable (Luo et al., [2024](#)).

Many definitions have been given in the literature (Chehade, [2022](#)), (Di Pierro, [2017](#)), (Wright & De Filippi, [2015](#)).

When a block is created, it is distributed to participants for validation and addition to the chain. This validation is performed using the ZKP protocol, which directly involves users' private data (VoD, [2021](#)). A block's capacity to hold a significant number of transactions is one of the factors that define the scalability of the blockchain (Rebello et al., [2024](#)).

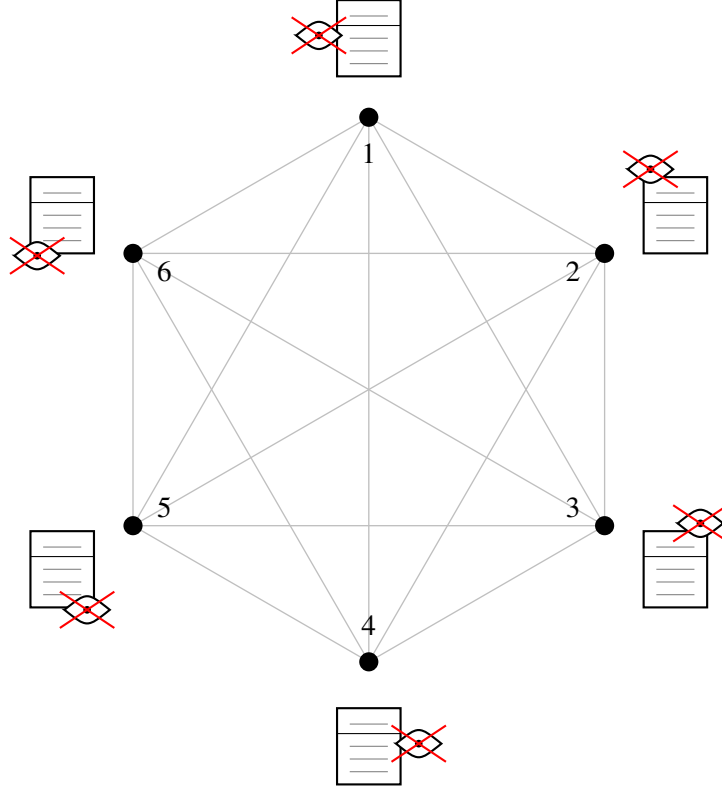


Figure 2.1: Blockchain as transparent and decentralized encrypted data sharing

Definition 2.2. (Scalability) The **scalability** of a blockchain refers to its ability to maintain a certain level of performance and security while handling a growing volume of transactions (Chen et al., 2024). However, other definitions can be found in (Rebello et al., 2024), (Mssassi & Abou El Kalam, 2024). The scalability of a network is determined by two metrics which are throughput, measured in TPS, and latency (the time necessary to reach the finality of a transaction).

Definition 2.3. (Privacy) The **privacy** of Blockchain describes how it systems manage the confidentiality of transaction and record data (including the identity of sender and recipient, the amount, and so on). Many definition were given in the literature (Garcia et al., 2024), (Holvast, 2007), (Pfitzmann & Hansen, 2010). For a definition adapted to our domain, blockchain, we will use the definition of (Zheng et al., 2018). The authors define privacy in blockchain systems as the ability of users to control the disclosure of their identities, transaction details, and associated metadata while interacting with a distributed ledger. In privacy, we distinguish two dimensions: confidentiality and anonymity. Confidentiality protects the content of transactions, whereas anonymity protects the identity of stakeholders (Portuondo et al., 2024).

According to Wikipedia (Wikipedia contributors, 2026), most public blockchains use pseudonymity to identify their users. Analyses of these Blockchains have revealed that certain attacks made it possible to trace transactions back to their author (Portuondo et al., 2024). Some Blockchains use cryptographic addresses instead of the identity of their

users. But the transactions remain visible on the chain and this method is not that effective.

Definition 2.4. (ZKPs) In the sense of Shafi Goldwasser, Silvio Micali and Charles Rackoff (Goldwasser et al., 1989), founding fathers of **ZKP**, a proof system is zero-knowledge disclosure if the verifier learns nothing other than the validity of the statement proven. An other definition is given by (Sheybani et al., 2025). In public blockchains, information about a transaction is visible to everyone. It is therefore possible to link this information to an identity. ZKPs play a very important role by allowing parties to prove the validity or correctness of certain data (transactions, account balances, identity) without revealing it, thus guaranteeing the anonymity of users within the network (Portuondo et al., 2024), (Feng et al., 2019).

Definition 2.5. (zk-SNARKs) **zk-SNARKs** are subtypes of NIZKs with a property that makes them unique: succinctness. This property allows them to produce very small proofs in terms of time and memory, for both the verifier and the prover (Liang et al., 2025). First, ZKPs are subdivided into two types: interactive proofs (IP), which requires several interactions between prover and verifier. A non-interactive proofs (NIZK), is where the prover constructs the proof and sends it to the verifier, therefore a single interaction. The non-interactive proofs are then distinguished by their specific properties, where we find snarks, as illustrated in Figure (2.2).

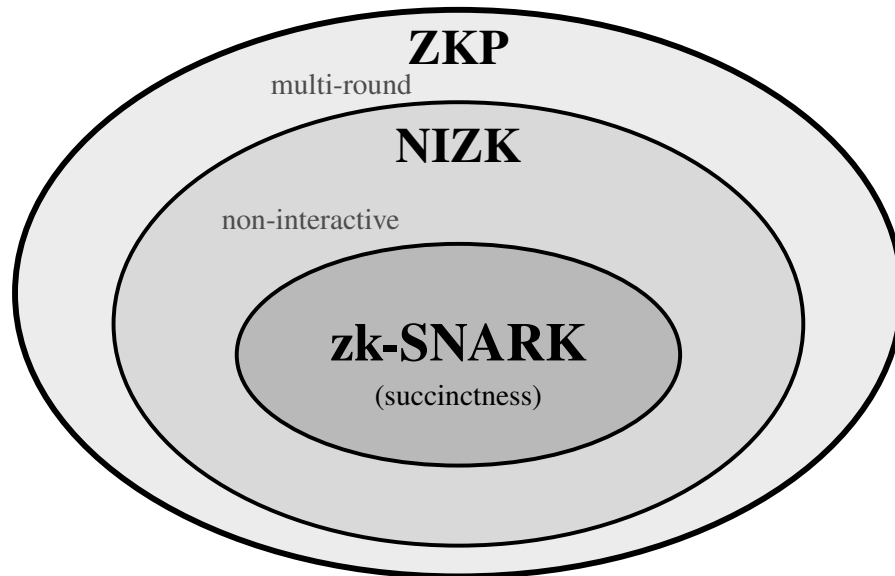


Figure 2.2: Relations between ZKP, NIZK and zk-SNARK, (Liang et al., 2025)

Definition 2.6. (Digital signature) A **digital signature** is a cryptographic method used to sign digital documents in a way analogous to signing physical documents with an anchor pen.

In 1976, Whitfield Diffie and Martin Hellman invented public-key cryptography. The first implementable of a practical system public-key was created by Rivest Shamir and

Adleman, and it is called the RSA cryptographic scheme. The security of RSA schemes is based on the difficulty of factoring large natural numbers $N = pq$, p and q prime numbers. In 1985, Taher Elgamal created a public-key cryptosystem based on the same problem as that of Diffie and Hellman (finding g^{ab} given g^a and g^b without knowing a and b) in the finite field $\mathbb{F}_q$. In 1987, Koblitz and Miller developed an improved version of Elgamal, by replacing the finite field $\mathbb{F}_q^*$ with an elliptic curve E , the difficulty is based on the discrete logarithm problem in the group of an elliptic curve $E(\mathbb{F}_q)$. 2.1 classifies digital signatures by their difficulty based.

Table 2.1: Digital Signature Schemes Classified by Their Difficulty Based (Johnson et al., 2001)

Family	Security is Based on	Examples
Integer Factorization (IF)	Hard to factor a product of large integers	RSA, Rabin
DL	The hardness of finding an exponent x : $g^x \equiv A(mod p)$	ElGamal, Schnorr
EC	The hardness of finding an exponent x : $g^x \equiv A(mod p)$ defined over an elliptic curve	ECDSA, EdDSA
ECDH	The hardness of finding g^{xy} given g^x and g^y	BLS

Digital signatures guarantee the authenticity, integrity, and non-repudiation of exchanged data in blockchain (Johnson et al., 2001). On the blockchain networks, validation of each transaction requires a signature. With the increasing number of users, the issue of needing to increase the block size and bandwidth required raises, for data propagation and signature verification, thus limiting the system’s scalability. However, solutions have been proposed, notably the BLS signature scheme, introduced in 2001 by, Dan Boneh, Ben Lynn, and Hovav Shacham (Boneh et al., 2001). These schemes improve scalability through the aggregation mechanism. We describe it in the section (3.4).

2.1.2 Contextual Background

The challenges related to digital identity management and trust in digital services remain significant and unresolved. Despite the rapid adoption of mobile technologies, a substantial portion of the population is still excluded from formal banking services. Digital connectivity and mobile financial services play a central role in expanding access to financial services for this population (Bank, 2022), (Klapper et al., 2021).

In this context, Blockchains are attracting interest due to their fundamental properties: decentralization, transparency, and resistance to falsification. Beyond digital payment systems, blockchain-based systems offer secure management of personal data in environments where access to reliable digital identities is limited. Furthermore, privacy-preserving cryp-

tographic mechanisms such as ZKPs and, more specifically, SNARKs, enable information verification while ensuring confidentiality. Their adoption in decentralized systems would help address concerns related to data protection, confidentiality, and network trust.

2.2 Theoretical Literature Review

This section examines the mathematical foundations upon which this work is built, covering elliptic curve theory, bilinear pairing theory, zero-knowledge proof theory and QAP. Our work, uses four fundamental theories: elliptic curve theory, bilinear pairing theory, zero knowledge proof theory and QAP theory. In the following lines, we explain each theory, give the founder and why is it used in our work.

2.2.1 Elliptic Curve Theory

The roots of elliptic curve theory originate in 19th-century algebraic geometry, particularly in the work of Karl Weierstrass on elliptic functions. It was Neal Koblitz and Victor Miller who, independently in 1985, proved that elliptic curves could be used in cryptography. This led to the birth of Elliptic Curve Cryptography (ECC) (Koblitz, 1987).

An elliptic curve E , is defined over a finite field $\mathbb{F}_p$ by

$$y^2 = x^3 + ax + b \pmod{p}, \quad \text{with } 4a^3 + 27b^2 \neq 0, \quad a, b \in \mathbb{F}_p \quad (2.1)$$

means that E is the set of points that verify 2.1 and the point at infinity. E equipped with the geometric addition law forms an abelian group.

Elliptic curves will constitute the algebraic building block on which the schemes in this work will be developed like the cyclic groups G_1 and G_2 , necessary for key generation and verification, and on which bilinear pairings are defined, which we will describe in the following section.

2.2.2 Bilinear Pairing Theory

The cryptographic use OF Weil pairing and Tate pairing was initiated by Dan Boneh and Matthew Franklin in 2001 with the first Identity-Based Encryption scheme, then formalized by Paulo Barreto, Steven Galbraith, Colm Ó hÉigeartaigh and Michael Scott in order to obtain efficient pairings on BN- and BLS-type curves.

A bilinear pairing is a map

$$e : G_1 \times G_2 \rightarrow G_T \quad (2.2)$$

where G_1 and G_2 are additive groups and G_T is a multiplicative group over a field extension. It is bilinear, non-degenerate, and computable.

Pairings are essential to three of our objectives: they enable the construction of signature schemes in Objective 1, constitute the central proof verification mechanism in Objective 2, and finally, in Objective 4, the choice of the curve directly determines the protocol's performance in terms of pairing computation time and proof size.

2.2.3 Zero Knowledge Proof Theory

The concept was introduced in 1985 by Shafi Goldwasser, Silvio Micali, and Charles Rackoff in their paper “The Knowledge Complexity of Interactive Proof Systems” (Goldwasser et al., 1989). This work earned the authors the 2012 Turing Award. Today, several variants exist: the NIZK systems of Manuel Blum, Paul Feldman, and Silvio Micali (1988) (Blum et al., 1988), Groth16 of Jens Groth (Groth et al., 2018) (Groth, 2016) and Bryan Parno et al. in 2013 with Pinocchio (Danezis et al., 2013). They possess the properties of completeness, soundness, and zero-knowledge.

A zero-knowledge proof is a protocol defined between a prover P and a verifier V . P convinces V that a statement is true (that it knows a witness w such that $f(x, w) = 1$ without revealing any information about w , $(x, w) \in \mathbb{R}$ for some binary relation $\mathbb{R}$.

The theory of ZKPs is the conceptual core of Specific Objective 3, to guarantee confidentiality and privacy.

2.2.4 QAP (Quadratic Arithmetic Program) Theory

Building on the previous work of Bryan Parno, Matthew Howell, Craig Gentry, and Mariana Raykova on Pinocchio in 2013 (Danezis et al., 2013), Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova formalized the QAP in 2012 in the paper “Quadratic Span Programs and Succinct NIZKs without PCPs” (Gennaro et al., 2013). The theory became a standard for compiling arithmetic circuits into succinct arguments.

Every computable function can be transformed into an arithmetic circuit, then into an R1CS constraint system, and finally into a QAP. Formally, a QAP is defined by sets of polynomials $A(x)$, $B(x)$, $C(x)$ and a target polynomial $H(x)$ such that: $(a_1, \dots, a_n)$ satisfies the computation if and only if:

$$H(x) \mid \left[\sum_k a_k A_k(x) \right] \cdot \left[\sum_k a_k B_k(x) \right] - \left[\sum_k a_k C_k(x) \right] \quad (2.3)$$

It means that the polynomial

$$\left(\sum_k a_k A_k(x) \right) \left(\sum_k a_k B_k(x) \right) - \left(\sum_k a_k C_k(x) \right) \quad (2.4)$$

is divisible by the polynomial $H(x)$.

QAPs transform a real-world problem written in a high-level language into a mathematical form suitable for zk-SNARK proofs. In Objective 2, this will allow us to take problems such as signature verification, delegated computation, and membership verification, and make them cryptographically provable.

2.3 Empirical Literature Review

In this section, we will conduct a literature review of previous studies in the field and identify the gaps in these contributions.

2.3.1 Review of Previous Studies

In this section, we present the most important previous work on ZK protocols using elliptic curves. We focus this review primarily on non-interactive protocols, but this does not preclude mentioning others.

Theoretical foundations 1985–1990. It all began with the seminal article by Goldwasser, Micali, and Rackoff (**GMR**) published in 1985 (Goldwasser et al., 1989), which introduced the interactive proof model. The authors formally defined the concepts of completeness and soundness. GMR has no curve dependency. Its relevance to blockchain scalability is indirect but fundamental: every protocols compared in this thesis inherits its correctness by this model. Our own experimental validation of soundness (Chapter 4, systematic "OK" verification status) is based on the formal definition gave by GMR. The following year (1986), **Fiat and Shamir** transformed the interactive protocol into a non-interactive one. He used cryptographic hashing to replace verifier challenges (Fiat & Shamir, 1986). This technique would later be used in modern protocols such as PLONK and Halo2. independent of the underlying curve; however, its security guarantee is contingent upon the curve once instantiated within Groth16, Marlin, or Halo2, since the hash is now linked to elliptic curve group elements. A ZK-Rollup could not submit a single proof to Layer 1 without a real-time interactive session with the verifier; non-interactivity offers an advantage to the entire deployment model tested in Chapter 4. Two years later (1988), Blum, Feldman, and Micali developed non-interactive zero-knowledge proofs (**NIZK**) (Blum et al., 2019). The protocol uses a common reference chain (CRS), shared between the prover and the verifier. This is also used in Groth16, another zero-knowledge proof protocol described below. It does not depend on any specific curve, yet its instantiation in Groth16 becomes specific to a given curve; the CRS consists of elliptic curve group elements associated with BN254 or BLS12-381. The "per-circuit trusted setup" that we quantify for Groth16 in Chapter 4 (Table 4.2, unamortized setup time) derives directly from this model hence its contribution to blockchain system scalability. In 1987, to bring all these protocols closer to practice, the work of Goldreich, Micali and Wigderson, via the 3-graph coloring problem, proved that any NP problem can be demonstrated in ZK (Goldreich et al., 1986). Of a purely fundamental nature, this result does not depend on the curve and has no direct bearing on our experimental comparison.

Delegated calculation verification 2000–2010. Other work followed, notably that of Micali in 2000 (CS Proofs) with the first "computationally sound proofs" (Micali, 2000). In 2008, Goldwasser, Kalai, and Rothblum (GKR) introduced an interactive verification protocol for layered circuits (Goldwasser et al., 2008).

Modern era of succinct zk-SNARKs 2010.

Pinocchio (2013): Created by Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova in 2013, Pinocchio is the very first practical zk-SNARKs protocol for general calculations. It has reduced verification time to the order of milliseconds and size of

the proof to less than 300 bytes (Parno et al., 2016). The author used QAPs (Quadratic Arithmetic Programs), mathematical tools that allow the translation of computer code into systems of polynomial equations, the validity of which can be verified through homomorphic functions on elliptic curves. The cost of generating proofs using the protocol is too high. It also requires a trusted setup that is not universal, meaning that each program modification requires a different setup. This protocol lays the foundation for constraint arithmetic. Subsequent work simply optimized its QAP model, eliminating the trusted setup. This is the case, respectively, with Groth16, Marlin, and PLONK.

Groth16 (2016): the performance Baseline. Created by Jens Groth in 2016, this protocol has made it possible to design even more compact proofs: 128 bytes with a constant verification time (Groth, 2016). It is based on the QAP model and uses pairings on elliptic curves. Trusted setup is specific to each circuit, meaning that any modification or update to the application invalidates the reference key, requiring a new trusted setup ceremony. This aspect is central to this thesis the entire construction relies on bilinear pairings $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, making every phase of the protocol (setup, proof, verification) a direct function of the pairing cost on the chosen curve. This is precisely the dependency we quantify in Chapter 4: a BLS12-381/BN254 performance ratio ranging from $1.45\times$ to $1.76\times$ depending on the pipeline phase, with a notable exception on the full pairing operation itself (near-identical cost, $\approx 1706 \mu s$). The constant proof size independent of the circuit makes this protocol an ideal candidate when storage or transmission costs are the limiting factor. Groth16 is our formally analysed reference protocol; it also serves as the basis for establishing our comparative evaluation methodology against other protocols such as Marlin and Halo2.

Argument d’Inner-Product, Bootle et al (2016): scalable rupture created by Bootle, Jonathan and Cerulli, Andrea and Chaidos, Pyrros and Groth, Jens and Petit, Christophe, it is the first protocol eliminating the trusted setup and reducing the size of the proofs from a linear factor to a logarithmic factor ($O(\log n)$). The authors introduced a new technique, Inner-Product Argument (IPA), which allows for recursive compression of the proof. The prover seeks to demonstrate knowledge of two vectors whose dot product yields a given value without revealing the vectors themselves. DPA reduces the size of the proof by half with each iteration of these vectors, resulting in a logarithmic proof that the prover submits instead of the original vectors, (Bootle et al., 2016). The raw size of the evidence remains large for practical, large-scale implementation. The method relies on the complexity of the discrete logarithm problem within an elliptic curve group; however, it does not depend on pairing arithmetic, which is the focus of this thesis. Although the protocol eliminates the need for a trusted setup, this technique alone does not solve the issue of verification costs. We intentionally excluded this method from our experiments because we wanted to compare only those protocols that allow us to evaluate the impact of the elliptic curve choice while maintaining the same cryptographic framework. This ensures that the results

are not skewed by differences in underlying cryptographic assumptions.

Gradual elimination of trusted setup 2018-2020 Bulletproofs (2018): rank proof optimization in their paper entitled "*Bulletproofs: Short Proofs for Confidential Transactions and More*" (Bünz et al., 2018), Benedikt B., Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell introduced a new non-interactive proof-of-knowledge protocol. This protocol enabled the implementation of short, dense, and pleasant-to-use proofs of rank without a trusted setup, making them ideal for blockchain confidentiality. Bulletproofs made the IPA technique of Jonathan et al. more efficient. Indeed, the protocol uses the bulletproof technique to prove the computation on hidden data on Pedersen commitments while eliminating the trust setup. The verification time is linear $O(n)$. That is to say, it increases as the size of the circuit increases. As with the IPA lineage mentioned above, Bulletproofs relies on the discrete logarithm problem on a standard curve, without requiring pairings. While it eliminates the need for trust, it is incompatible with the succinctness required by a zk-Rollup; its $O(n)$ verification cost creates a bottleneck for block validators. For the same reasons as before, we exclude this protocol from our analysis.

Libra (2019): the Effectiveness of the Prover created by Xie, Tiacheng and Zhang, Jiaheng and Zhang, Yupeng and Papamanthou, Charalampos and Song, Dawn in 2019, Libra made a contribution by making the proof generation time linear $O(n)$. The protocol uses the Goldwasser GKR technique for interactive verification, applies the fiat-shamir transformation to make the mechanism non-interactive, and finally applies it to multilinear polynomials to obtain fast verification. GKR is inflexible, which makes translating certain real-world calculations difficult and causes a loss of protocol efficiency(Xie et al., 2019). Libra's approach relies on a sum-check verification protocol over generic finite fields, thereby inheriting a curve-independent nature unlike GKR. Although not directly tested, its exclusion follows the same logic applied to GKR. The circuit models examined in this thesis (R1CS for Groth16/Marlin, Plonkish for Halo2) do not share Libra's structural constraints; consequently, any attempt at comparison is methodologically inconsistent.

Plonk (2019): industrial standardization in 2019, Gabizon, Ariel, Williamson, Zachary J, and Ciobotaru, Oana, made a major contribution to ZKP protocols. Through their article "*Plonk: Permutations over Lagrange-bases for ecumenical non-interactive arguments of knowledge*," they established a universal trusted setup technology. This means that a single trust ceremony is sufficient to execute an infinite number of different circuits. The flexibility of this protocol relies on three techniques: "Plonkish" arithmetic: another way of translating a computation compared to R1CS, studied in this dissertation. The computation is organized in a table where each row represents a constraint. This explains its flexibility, as it does not force the constraints to have a standard form. The second technique is custom gates: instead of being limited to a single operation (multiplication in R1CS), the protocol allows combining several operations. The third allows proof com-

pression thanks to KGR and the large product permutation technique. This makes it a flexible and compact protocol (Gabizon et al., 2019). The size of the proof and the Gas cost of the verification are slightly higher than Groth16 due to the evaluation of multiple polynomials. Used for comparison. Similar to Groth16 and Marlin, the protocol relies on KZG (kate-zaverucha-goldberg) polynomial commitments and thus on bilinear pairings making the choice of curve a significant factor for performance. Halo2 employs the same arithmetic as PLONK. In our experience, it is the highest-performing protocol in terms of proof generation time (Table 4.5: 93.037s at $k=22$ on BN254). PLONK itself was not experimentally tested; instead, we chose to test its direct descendant, Halo2 (the PSE fork, which uses KZG rather than the original Zcash IPA). However, its arithmetic construction explains why Halo2 shares the same pairing-friendly curve dependency as Groth16 and Marlin, unlike the original Zcash/Halo2 variant based on IPA.

Marlin (2020): the universal alternative R1CS to make SNARKs universal and efficient while retaining the flexibility of existing protocols such as PLONK. It is the result of work by Chiesa, Alessandro and Hu, Yuncong and Maller, Mary and Mishra, Pratyush and Vesely, Noah and Ward, Nicholas led in 2020 (Chiesa et al., 2020). The protocol uses R1CS, AHP (algebraic holographic proof), which transforms the rigid nature of R1CS into a holographic polynomial form, KZG, allowing the resulting AHP polynomials to be committed to a small group element. The verifier checks the evaluations of these polynomials in constant time. Like Groth16, this protocol relies heavily on elliptic curves. It is based on the KZG commitment scheme and, consequently, on bilinear pairings. This shared reliance on elliptic curves means that the comparison between BN254 and BLS12-381 is directly applicable to both protocols in our study. While this technique saves time in blockchain rollups, it becomes computationally heavy and slow for large-scale calculations. In our experiments, Marlin performed the worst of the three protocols tested, both in terms of proof generation time (792.010s on BN254 at $k=22$) and proof size (743-951 bytes). When combined with the memory failure observed with BLS12-381 at $k=20$ (see Table 4.3), this highlights the practical challenges associated with this protocol.

Halo (2019): (Recursive Accumulation) Created in 2019 by Sean Bowe, Jack Grigg, and Daira Hopwood, the protocol definitively eliminates the trusted setup and allows for the recursive accumulation of proofs, so that one proof verifies other proofs (Bowe et al., 2019). The protocol uses a polynomial engagement scheme of the IPA type applied to specific elliptic curve cycles: Pasta (Pallas/Vesta). The absence of bilinear pairings makes the cost of the final verification high and complex to execute on a classic EVM (Ethereum Virtual Machine). Another version of this protocol, a technical evolution of Zcash, exists and is called Halo2, based on KZG, therefore requiring a universal trusted setup and quick verification. It is the one that we used in our benchmarks. The original Zcash variant uses IPA which is pairing-free and features a transparent setup but entails slower verification. The PSE version used in our analysis employs KZG, which relies on pairings and

a universal SRS, offering fast verification; it is the fastest of the three protocols tested in this thesis in terms of large-scale proof generation time (Table 4.5). It is therefore the ideal candidate when proof generation latency is the limiting factor for high throughput ZK-Rollup deployment. Halo2 (PSE) completes our experimental trio, representing the most recent generation of Plonkish style arithmetisation with pairings. Our experiments show that it delivers superior performance regarding proof generation time (with proof sizes of 352-464 bytes, placing it between Groth16 and Marlin).

Recent directions post-2020. Subsequently, other protocols emerged such as Spartan and HyperPlonk (based on GKR) and the STARK family of constructions (STARKs [Ben-Sasson et al., 2018], Aurora, Fractal) all of which rely on hash functions and error-correcting codes. While these protocols are quantum-resistant, they generate very large proofs. They do not rely on discrete logarithms or pairings; we therefore excluded them from our comparison for the same reason as Bulletproofs and the IPA lineage: they do not share the reliance on pairing arithmetic examined in this thesis.

There are numerous zero-knowledge protocols. We have presented and illustrated several of them based on their cryptographic foundations, year of creation, authors, and crucially for this thesis their reliance on elliptic curve pairings.

2.3.2 Identification of Gaps

From GMR to Halo 2, the literature has progressively solved 3 distinct problems:

1. the actual existence of zero-knowledge non-interactive evidence (Blum-Feldman-Micali, 1988)
2. the succinctness of these proofs via QAPs and pairings (Pinocchio, Groth16)
3. Elimination of the specific trusted setup for each circuit and its replacement by a universal SRS (PLONK, Marlin) or total transparency (Bulletproofs, STARKs, Halo).

By comparing these works, 3 unresolved tensions emerge:

1. **Contradiction between theoretical announcements and empirical validation.** The foundational articles (Plonk, Marlin, etc.) announce asymptotic gains ($O(\log n)$, $O(1)$, etc.) but do not compare these protocols on the same elliptic curves, with the same hardware implementation, and on the same basic circuit. Groth16 Benmarke alone (Arkworks), PLONK/Halo2, distinct Rust implementations: Zcash/PSE vs Aztec. Thus, there is a lack of **cross-comparison**.
2. **Choice of elliptical curve is treated as secondary.** The articles present the curves without systematically quantifying the impact of the BN254 vs. BLS12-381 choice on proof time, proof size, and verification time. However, this choice has direct consequences on on-chain deployment.

3. **Speed-size-setup trade-off is rarely quantified comparatively.** Bootle et al. (2016), Bulletproofs (2018), and Halo (2019) eliminated the trusted setup and gained an $O(n)$ verification. However, few studies place this compromise on the same empirical scale as Groth16/Marlin/Halo2 for increasing circuit sizes (500 to 4,000,000 constraints).

We further support our analysis with these recent articles that have worked in the same direction or somewhat similarly. First, none of studies analyzes the internal arithmetic of bilinear pairings, which are the fundamental computational concepts for all zk-SNARK systems. (Portuondo et al., 2024) and (Xue et al., 2024) recognize computational complexity as a major challenge, but without a path toward resolution. (Liang et al., 2025) identify the misuse of curves as a practical problem, but without arithmetic analysis: Miller loop, final exponentiation, and comparison of algorithms on the BN254 and BLS12-381 curves. Second, in the studies, Groth16 is mentioned but without formal analysis (Xue et al., 2024). Third, the zero-knowledge properties are defined informally by (Portuondo et al., 2024) through an illustrative example. Some authors, namely (Xue et al., 2024) and (Unnikrishnan & Victor Paul, 2025), did not provide a formal statement of these properties in their works. (Unnikrishnan & Victor Paul, 2025) implements a zk-Rollup but replace pairings with hash functions. This deprives the system of the cryptographic rigor that only a pairing-based system can provide.

This dissertation directly addresses these three shortcomings by proposing:

1. an implementation and controlled benchmark of Groth16, Marlin and Halo2 on BN254 and BLS12-381, under identical hardware conditions.
2. a systematic quantification of the trade-off between prover time / proof size / verification time on a scale of 2^9 to 2^{22}
3. an analysis combining these empirical results with theoretical properties (setup, cryptographic assumptions) to formulate recommendations for protocol selection according to the Blockchain deployment context.

We establish a formal and quantitative chain linking pair arithmetic optimization, rigorous analysis of proof systems (Groth16), demonstration of their confidentiality properties, and their deployment in zk-rollups. To this end, we define in the following chapter the materials and methods necessary to perform these tasks.

CHAPTER 3

MATERIALS AND METHODS

This chapter presents the materials, methods, and procedures used to achieve our objectives. We present the architecture of Blockchain in Section 3.1, then the mathematical preliminaries including the arithmetic of bilinear pairings, Miller’s algorithm, and final exponentiation in Section 3.2. Section 3.3 then provides a rigorous formal analysis of the Groth16 protocol, from arithmetic circuits to QAP. Section 3.4 presents the BLS signature scheme and its contribution to Blockchain scalability. Section 3.5 formalises the zk-Rollup pipeline. Finally, Sections 3.6 and 3.7 detail the software and hardware environment specifications and the benchmark circuits used to conduct our experiments.

3.1 Blockchain

3.1.1 Block Structure and Chaining Mechanism

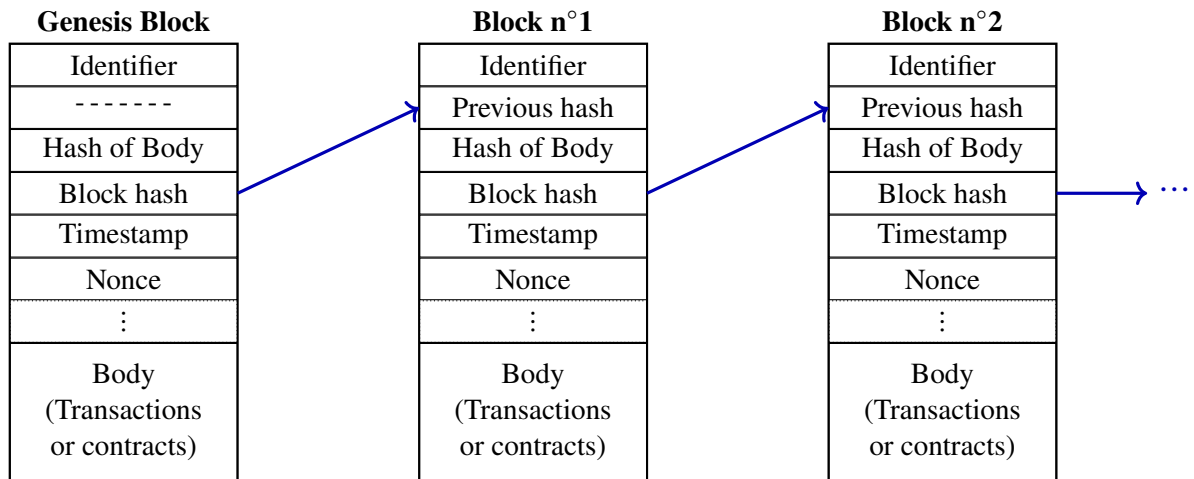
Block Structure Blocks are composed of two parts: the **header** and the **transactions**.

The header groups together all the information of a block except the transactions. It makes it possible to perform the proof of work and has a fixed size of 80 bytes. using the following data:

- **Version(4 octets)**: Bitcoin protocol version used for the block.
- **Hash of the previous header (32 bytes)**: result of the digest of the previous block header hashed by a hash function. This is what makes the blocks immutable in the time.
- **Merkle root (32 bytes)**: constructed from the transactions contained in a block, each transaction hash corresponds to a Merkle leaf. The hashes are concatenated two by two and rehashed until a single hash is obtained at the end, which represents the root of the tree. This root is what makes it possible to fix the block transactions, because if one leaf is modified, the SHA-256 of the root is also modified. This changes the block header and causes the invalidation of the following blocks.
- **Timestamp (4 bytes)**: this is the creation date of the block in Unix time (seconds elapsed since 1970). However, there are two constraints:
 - **upper-bound constraint**: the time must not be too far in the future (≤ 2 hours);
 - **lower-bound constraint**: it must also not be too far in the past.
- **Bits (4 bytes)**: this is the 256-bit binary number of the **target**. Indeed, during the creation of a block, its hash must not exceed the **target** value. The latter is automatically adjusted by the system every 2016 blocks (2 weeks). The smaller the **target**, the more difficult mining becomes.

- **nonce (4 octets):** 32-byte binary number. During the creation of a block, the miner varies the nonce until obtaining a hash lower than the **target**. This is the proof of work used to add a new block. Given that blocks must not exceed 1 MB, and that 80 bytes are allocated to the header, the transactions occupy: 999,920 bytes. Each block contains the hash of the previous block header, which makes it possible to create an immutable chain of blocks.

Figure 3.1: Block structure and chaining mechanism in a blockchain.



3.1.2 Consensus Mechanisms: Proof of Work and Proof of Stake

In this section, we present the proof-of-work mechanisms most commonly used in Blockchain. First, proof of work (PoW) and second proof of stake (PoS).

Consensus Mechanism: the consensus mechanism is a process that allows blockchain miners to validate and create blocks. This mechanism uses the Byzantine Generals Problem to ensure trust in the network. Consensus ensures that the decision has been made by the majority of participants in the network.

Proof of Work it was introduced by Cynthia Dwork and Moni Naor in 1992, (Dwork & Naor, 1992) to block email spam. It is the most widely used system in blockchain consensus. Satoshi Nakamoto used this mechanism to solve the double-spending problem that threatened cypherpunks before Bitcoin.

It is a complex computation, costly in time and energy. It is often based on asymmetric problems that can be encountered in everyday life, such as the Rubik's Cube and the Eternity II puzzle. These problems are difficult to solve, but once the solution is found, verification becomes easy.

Proof of Stake created in 2012 by King et Nadal as an alternative to Proof of Work due to its high energy consumption, (Masseport, 2021) it allows the next block creator to be selected randomly. The selection criteria are based on factors such as age or wealth,

etc. For example, if the selection criterion is based on the amount of currency held, then someone holding 15% of the network's currency will have a 15% chance of being selected. This encourages users to save more, and may reduce trust in the network, which in turn may decrease the value of the currency.

After the creation of a block, it is distributed to the rest of the network for verification in order to add it to the main chain. To perform this verification, the ZKP protocol is used. The cost in time and memory for generating this proof has a direct impact on the scalability of the blockchain.

The confidentiality of the data used for generating the proof is also ensured by the protocol.

3.1.3 The Blockchain Trilemma

Presented by Vitalik Buterin in 2017 (Nakai et al., 2023), the concept states that no blockchain system can simultaneously satisfy the three properties of scalability, decentralization, and security.

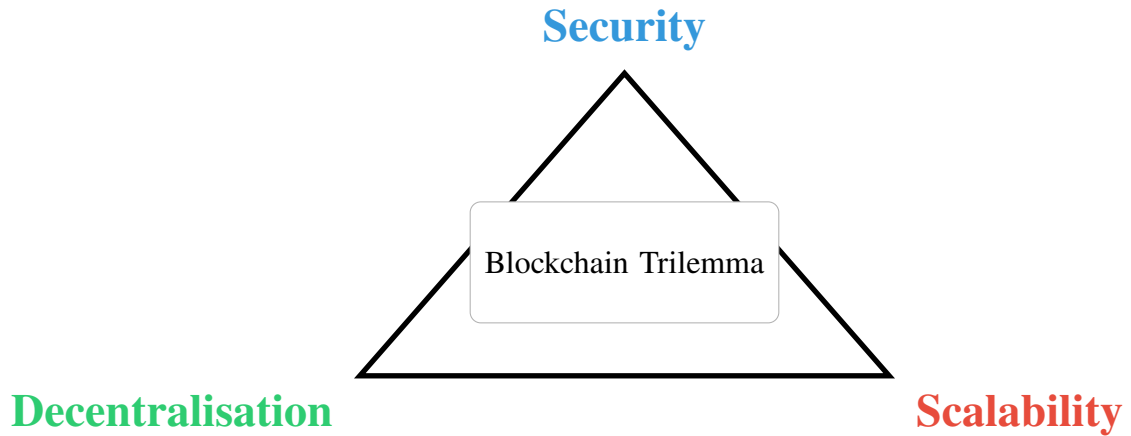
The security of a blockchain is its ability to validate transactions correctly and resist attacks. Decentralization refers to the absence of a central entity controlling the network. Finally, scalability is its ability to process a large number of transactions quickly and at minimal cost. We illustrate this concept through the following blockchain examples 3.1. Bitcoin, as a public blockchain, has good security and decentralization, but low scalability (Bulgakov et al., 2024). Which allows us to fill in the first row. By knowing that private blockchains are centralized, we fill the second row. For the third row, refer to: (Wijaya et al., 2025).

Table 3.1: Concrete examples of trade-off between scalability, decentralization, and security in blockchain.

System	Security	Decentralization	Scalability
Bitcoin	Yes	Yes	No
Hyperledger Fabric	No	Yes	Yes
Fantom	Yes	No	Yes

This trade-off is rooted in the structure of the blockchain. Indeed, increasing the block size improves scalability but reduces decentralization, because only powerful nodes with large storage capacity will be able to store the chain. On the other hand, reducing the time between blocks improves throughput, of course; however, it facilitates forks. This weakens the network against attacks such as the 51% attack (Gervais et al., 2016), section 2.1.1. The architecture based on ZK-Rollups is an alternative that makes it possible to overcome this problem by moving computation off-chain while maintaining the security of the main chain (Unnikrishnan & Victor Paul, 2025).

Figure 3.2: Blockchain Trilemma



3.1.4 The Scalability Crisis: Quantification

Table 3.2: Comparison of transaction throughput, finality time, and cost across different blockchain networks. TPS=Transaction Per Second, CPT=Cost per Transaction

Network	TPS	Finality Time	CPT
Bitcoin	~ 7	~ 60 min	Variable(\$0.5-\$10, can exceed \$50)
Ethereum L1	~ 15	~ 12 sec	Variable(\$0.20-\$20, can exceed \$100)
Visa	~ 24,000	~ 2 sec	Negligible(\$0.01-\$0.05)

Two factors explain Ethereum’s limitation to 15 TPS: the block size; blocks are limited by gas (unit measuring the amount of computation required to perform an operation on the blockchain). Increasing this limit forces nodes to store more data, which threatens its decentralization.

The other factor is the block interval: 12 seconds. Reducing this interval facilitates forks and therefore increases the number of orphan blocks (Unnikrishnan & Victor Paul, 2025). According to 3.2, Ethereum requires 1,600 seconds to process 24,000 transactions, i.e., more than 26 minutes, while Visa processes the same volume in only about 2 seconds. This is a real limitation to its adoption as a global financial system.

The solution introduced in the last paragraph of the section 3.1.3, constitutes a direct response to this limitation, with a throughput reaching up to 65,000 TPS.

3.1.5 The Confidentiality Crisis

The fundamental limitation of public blockchains is that they are designed to be transparent, because this guarantees trust in the network. However, this transparency makes all user data readable. For a Blockchain such as Bitcoin or Ethereum, each transaction contains: the sender’s address, the receiver’s address, the transferred amount, and the

timestamp. This information is readable by anyone at any time.

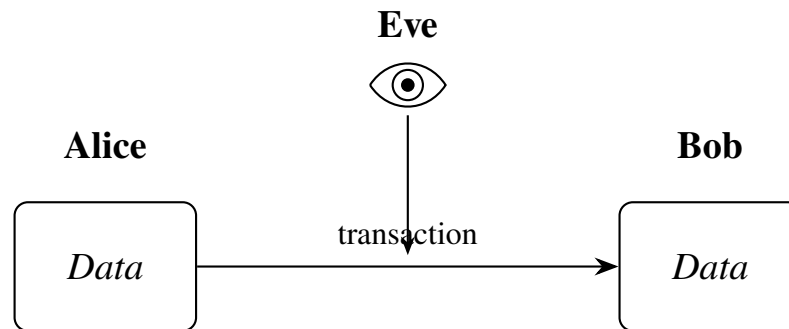


Figure 3.3: Without ZKP: Eve freely observes the transaction data between Alice and Bob.

Wallet addresses in public blockchains look like: 0x742d35Cc6634C0532925a3b8D4... This address appears anonymous. However, in reality, it is only pseudonymous. If an address is linked even once to a real identity (through a public transaction or a payment to a merchant), all its past and future transactions then become traceable. Companies such as Chainalysis use transaction graph analysis to connect pseudonymous addresses to real individuals. This, thanks to heuristic clustering techniques, transactions having certain patterns on the blockchain can be linked together (Chainalysis, [n.d.](#)).

That said, naive techniques such as transaction mixing and ring signatures used to hide the identity of signers have all been compromised. Only zero-knowledge proofs, in the rigorous sense defined by Shafi Goldwasser, Silvio Micali, and Charles Rackoff, can formally guarantee data confidentiality. They make it possible, for any proven statement, to construct a simulator capable of producing an indistinguishable transcript without access to sensitive data. This is the case, for example, with zk-SNARKs.

3.1.6 Zero-Knowledge Proofs as a Solution

This section constitutes a bridge between the blockchain problems presented in sections 3.1.1 to 3.1.5 and the mathematics that we will present in the following sections. We subdivide the section into two subsections: why ZKPs in the first place and explaining ZKPs in detail in the second part.

Need of ZKP: ZKPs make it possible to simultaneously address the two crises we have identified, namely the privacy crisis and the scalability crisis. Regarding the privacy crisis, ZKPs allow one to prove that a transaction is valid without revealing the data of that transaction. In Zcash, for example, the prover demonstrates that he has sufficient funds without revealing data such as its identity, the amount, and the recipient. This corresponds exactly to the main property of zero-knowledge in the sense of Goldwasser, Micali, and Rackoff.

Regarding scalability, ZKPs make it possible to aggregate thousands of transactions off-

chain and submit a single succinct proof on-chain. The on-chain verifier only needs to verify this proof in constant time, regardless of the number of transactions. (Unnikrishnan & Victor Paul, 2025) showed that this approach enabled the Ethereum blockchain to reach up to 65,000 transactions per second, compared to 15 in the standard setting.

Simple data encryption through classical methods protects the data, but at the same time prevents their verification. On-chain optimization, that is, increasing the block size in order to improve scalability, compromises decentralization. Only ZKPs provide verification and confidentiality at the same time without compromising decentralization.

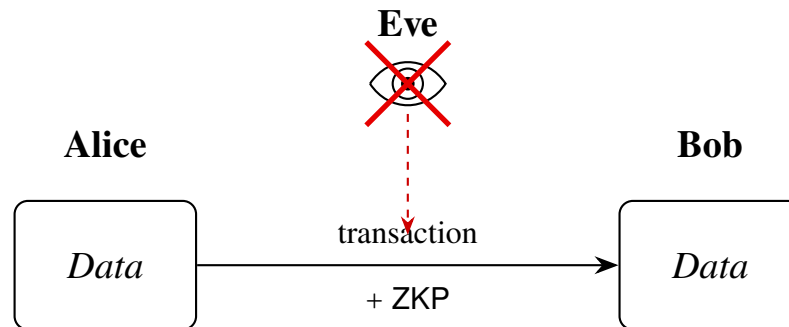


Figure 3.4: With ZKP: Eve is blocked. ZKP guarantees data confidentiality

Working Principle ZKPs: one of the examples mentioned in the literature (Augot et al., 2021) is **Where's Waldo?** The game is well known: it involves those books with large, very dense illustrations and very rich in details. Waldo is hidden there, easily recognizable once you have found him.

A child (the prover) is going to prove to his father (the verifier) that he has found Waldo without showing where Waldo is. He proceeds as follows:

1. He prepares a cardboard as large as nine times the page
2. He cuts out of this cardboard the silhouette of Waldo
3. He presents this cardboard facing his father
4. He positions the book behind the cardboard so as to make Waldo appear in the cut-out
5. The father sees Waldo through the cut-out but does not see Waldo's position on the page

The father is convinced that his child knows where Waldo is, without learning where Waldo is.

The idea behind ZKP protocols is as follows: “we have two people, Alice and Bob, and Bob wants to prove to Alice that a statement is true, without revealing any information about the proof. In other words, he wants to prove to Alice that he has certain knowledge. Alice has to verify that the proof of Bob is valid. Due to their respective roles in this process, Alice is often called the verifier and Bob is often called the prover.” (Gerards, 2022)

We summarize this process into three algorithms and one algorithm for simulation.

- **Setup**, is constructed by V . It takes as input a security parameter λ in order to generate a common reference string CRS , a verification state Ver , and a secret trapdoor used later in the simulator. V sends the CRS to P , who will use it to generate his proof.
- **Prove**, takes as input the CRS , a statement s , and a witness w , and outputs the proof π . P encrypts π and transmits it to V .
- **Verify**, takes as input the verification state Ver generated by *Setup* and the encrypted proof π received from P , and outputs 1 if the proof is accepted, or 0 otherwise.
- **Sim**, takes as input the secret trapdoor $trap$, the CRS , and s , and outputs the proof π , or at least a proof similar to π . If the proof π of Alice and Bob is valid, the simulator also produces a valid proof. Otherwise, *Sim* also generates an invalid proof. *Sim* makes it possible to produce a proof without knowing the witness.

zk-SNARKs such as Groth16 rely on bilinear pairings for proof verification in constant time and with a small proof size. Hence the need to analyse this protocol in depth.

3.2 Mathematical Preliminaries

In this section, we introduce the mathematical concepts used in chapter4 necessary to interpret results. We first define the pairing groups, the notion of embedding degree, and the security assumptions required for the verification equation (4.1) and the formal analysis of the Groth16 protocol (Section 3.3). Next, we define the Miller loop, the final exponentiation, and the parameters for the BN254 and BLS12-381 curves, which are useful for analysing the arithmetic cost of pairings on both curves (Tables 4.1 and 4.2, Section 4.1.3).

In what follows, we consider $K = \mathbb{F}_q$ be a finite field and $\bar{K}$ its algebraic closure, where $q = p^n$ with p a prime integer. Let $K[x_1, \dots, x_n]$ be a polynomial ring in n variables with coefficients belonging to K . We consider $I \subset K[x_1, \dots, x_n]$ an ideal of $K[x_1, \dots, x_n]$.

3.2.1 Elliptic Curves over Finite Fields

Weiestrass Normal Form: the general form of the Weiestrass equation for an elliptic over is given by:

Definition 3.1.

$$E : y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6, \quad \text{where } a_1, a_2, a_3, a_4, a_6 \in \mathbb{K}, \quad (3.1)$$

and $\mathcal{O} = [0 : 1 : 0]$ is the point at infinity.

The reduced form of this form is given by:

$$E : y^2 = x^3 + Ax + B, \quad A, B \in \mathbb{K}, \quad \Delta = -16(4A^3 + 27B^2) \neq 0 \quad (3.2)$$

Definition 3.2. The quantity Δ given by,

$$\Delta = -16(4A^3 + 27B^2). \quad (3.3)$$

is the discriminant of the Weiestrass equation.

For an elliptic curve, Δ is always different to zero.

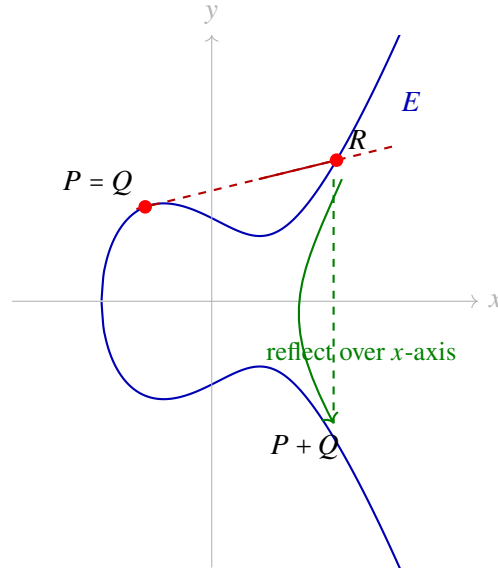
An elliptic curve is simply a set of points which verify (3.2), and the point at infinity $\mathcal{O}$:

$$E = \{(x, y) \in K^2 \mid y^2 = x^3 + Ax + B\} \cup \{\mathcal{O}\}. \quad (3.4)$$

Group Law: the group law is defined geometrically as follows: Having $P, Q \in E$, we define $P + Q$ by:

- Draw the line ℓ between P and Q (tangent at P if $P = Q$).
- This line cuts E at a third point R in $\mathbb{P}^2$.
- Define $P + Q$ as the symmetric of R via the x -axis.
- The neutral element is the point $\mathcal{O}$.

Figure 3.5: Geometric construction of $P + Q$ on $E : y^2 = x^3 - x + 1$. The red dashed line meets the curve at R ; reflecting R across the x -axis gives $P + Q$.



Let $P = (x_1, y_1)$ and $Q = (x_2, y_2)$ be two affine points of $E : y^2 = x^3 + Ax + B$, distinct from $\mathcal{O}$.

Case 1: Addition ($P \neq Q$ and $P \neq -Q$, i.e., $x_1 \neq -x_2$):

$$\begin{aligned} \lambda &= \frac{y_2 - y_1}{x_2 - x_1}, \\ x_3 &= \lambda^2 - x_1 - x_2, \\ y_3 &= \lambda(x_1 - x_3) - y_1. \end{aligned} \quad (3.5)$$

Then $P + Q = (x_3, y_3)$.

Case 2: Doubling ($P = Q, y_1 \neq 0$):

$$\begin{aligned}\lambda &= \frac{3x_1^2 + a}{2y_1}, \\ x_3 &= \lambda^2 - 2x_1, \\ y_3 &= \lambda(x_1 - x_3) - y_1.\end{aligned}\tag{3.6}$$

Then $2P = (x_3, y_3)$.

Case 3: $P = -Q$: i.e., $x_1 = x_2, y_1 = -y_2$, so $P + Q = \mathcal{O}$.

Case 4: $P = \mathcal{O}$ or $Q = \mathcal{O}$: $P + \mathcal{O} = P$ and $\mathcal{O} + Q = Q$.

The couple $(E, +)$ forms an abelian group.

Definition 3.3.

For $n \in \mathbb{Z}$ and $P \in E$, the scalar multiplication is defined by:

$$\begin{aligned}[n]P &= \underbrace{P + P + \dots + P}_{n \text{ times}}, \quad (n > 0), \\ [0]P &= \mathcal{O}, \\ [-n]P &= [n](-P).\end{aligned}\tag{3.7}$$

Cardinality of the curve

Theorem 3.1. (*Hasse*) Let E be an elliptic curve over $\mathbb{F}_p$. Then,

$$|E(\mathbb{F}_p)| = p + 1 - t_p \quad \text{with } t_p \text{ satisfying } |t_p| \leq 2\sqrt{p}.\tag{3.8}$$

Proof. See A.1 □

Then, number of points $|E(\mathbb{F}_p)|$ is linked to the Frobenius trace t_p .

Property 1. Let E be an elliptic curve over a field K , and let $T \in E[m]$ be a point of order m , i.e. $[m]T = \mathcal{O}$.

Then there exists a function $f_T \in \bar{K}(E)^*$ such that:

$$\text{div}(f_T) = m(T) - m(\mathcal{O}).\tag{3.9}$$

Moreover, this function is unique up to a multiplicative constant.

Function g and composition with $[m]$: The construction of Weil pairing requires a second function linked to f_T by multiplication by m . Let $T \in E[m]$ and f_T defined as above. Let's take $T' \in E(\bar{K})$ with $[m]T' = T$. There exists $g \in \bar{K}(E)^*$ with:

$$\text{div}(g) = [m]^*(T) - [m]^*(\mathcal{O}) = \sum_{R \in E[m]} (T' + R) - (R)\tag{3.10}$$

And we can normalize to obtain the fundamental compatibility relation:

$$f_T \circ [m] = g^m\tag{3.11}$$

Proof. A.2

□

For more details and go deeply about arithmetic behind, see A.2.1.

The group law and scalar multiplication defined here are the operations measured in Table 4.2 (columns "scalar multiplication in G_1 " and "in G_2 "), where the most marked difference between BN254 and BLS12-381 is observed (459.3 ms versus 787.6 ms, a factor of $\approx 1.7\times$). Hasse's theorem, for its part, forms the foundation for the security analysis in Section 4.1.3: it is the Hasse bound that allows the curve's prime order r to be determined and the security level of BN254 to be compared with that of BLS12-381.

The two results in Property 1 and relation (3.10) are not benchmarked in this thesis. They are retained because they justify the existence and uniqueness of the Miller function used in Section 4.1.3; without them, the Miller loop presented in Section 3.2.3 would lack justification.

3.2.2 Pairing Groups and Security Assumptions

The calculation of bilinear pairings relies on three distinct algebraic groups. These groups have specific properties. Their definition is based on the notion of torsion subgroups of elliptic curves.

$$E[r] = \{P \in E(\overline{\mathbb{F}}_p) \mid rP = \mathcal{O}\}, \quad (3.12)$$

where $\overline{\mathbb{F}}_p$ denotes the algebraic closure of $\mathbb{F}_p$ and $\mathcal{O}$ the point at infinity. When $\gcd(r, p) = 1$, the torsion subgroup has the structure

$$E[r] \cong \mathbb{Z}/r\mathbb{Z} \times \mathbb{Z}/r\mathbb{Z}. \quad (3.13)$$

(Ghammam, 2016)

Definition 3.4 (Embedding Degree). Let $E/\mathbb{F}_p$ be an elliptic curve whose order $\#E(\mathbb{F}_p)$ is divisible by a prime number r . The embedding degree of E with respect to r is the smallest positive integer k such that

$$r \mid p^k - 1. \quad (3.14)$$

That is, the multiplicative group $\mathbb{F}_{p^k}^*$ contains an element of order r . The embedding degree determines the extension field in which the pairing takes its values. For a pairing-based system to be efficient in practice, k must also remain small enough to allow efficient computations.

Definition 3.5 (Bilinear pairing). A bilinear pairing is an efficiently computable map

$$\hat{e} : \mathbb{G}_1 \times \mathbb{G}_2 \longrightarrow \mathbb{G}_T \quad (3.15)$$

satisfying the following properties:

1. **Bilinearity**

$$\hat{e}(aP, bQ) = \hat{e}(P, Q)^{ab} \quad (3.16)$$

for all $P \in \mathbb{G}_1$, $Q \in \mathbb{G}_2$, and $a, b \in \mathbb{Z}$.

2. **Non-degeneracy**

$$\hat{e}(P, Q) \neq 1_{\mathbb{G}_T} \quad (3.17)$$

whenever $P \neq \mathcal{O}$ and $Q \neq \mathcal{O}$.

3. **Computability** The pairing $\hat{e}(P, Q)$ can be computed efficiently for every pair

$$(P, Q) \in \mathbb{G}_1 \times \mathbb{G}_2. \quad (3.18)$$

For curves such as BN254 and BLS12-381 (Ghammam, 2016), we have

$$k = 12.$$

Example 1. (Bilinear pairing) The optimal pairing Ate is defined on the BN curves by the following non-degenerate bilinear map, (Ghammam, 2016):

$$e : E(\mathbb{F}_p)[r] \times \Psi_6 E(\mathbb{F}_{p^2})[r] \longrightarrow \mathbb{F}_{p^{12}}^\times, \quad (3.19)$$

$$(P, Q) \longmapsto \left(f_{6u+2, Q}(P) l_{[6u+2]Q, \pi(Q)}(P) l_{[6u+2]Q + \pi(Q), \pi^2(Q)}(P) \right)^{\frac{p^{12}-1}{r}}.$$

$\Psi_6 E(\mathbb{F}_{p^2})[r]$ represents the proper r -torsion eigenspace associated with the sixth cyclotomic component of the elliptic curve over $\mathbb{F}_{p^2}$.

Example 2 (Optimal Ate Pairing on BLS12 Curves). The Optimal Ate pairing defined on BLS12 curves is the following non-degenerate bilinear map:

$$e_{\text{opt}} : G_1 \times G_2 \longrightarrow G_3, \quad (P, Q) \longmapsto f_{u, Q}(P)^{\frac{p^{12}-1}{r}}, \quad (3.20)$$

where $f_{u, Q}$ denotes the Miller function associated with the parameter u , r is the prime order of the subgroups G_1 and G_2 , and the exponent $\frac{p^{12}-1}{r}$ corresponds to the final exponentiation.

Security Assumptions

The security of all constructions studied in this dissertation including the Groth16, BLS signature, ZK-Rollup relies on several computational hardness assumptions, which are considered as computationally infeasible. That is illustrated in (3.3)

Table 3.3: Hierarchy of the Security Assumptions Used in This Thesis

Assumption	Simplified Definition	Role in This Thesis
ECDLP	Elliptic Curve Discrete Logarithm Problem: given P and $Q = xP$, recovering the scalar x is computationally infeasible.	Provides the fundamental security assumption for the elliptic-curve groups G_1 and G_2 .
CDH / co-CDH	Computational (co-)Diffie-Hellman Problem: computing the value $g^{ab} \pmod{p}$ knowing values g^a and g^b	Ensures the security of BLS signatures and signature aggregation schemes.
q -PKE	q -Power Knowledge of Exponent Assumption: an algorithm producing certain algebraic outputs must "know" the corresponding exponent.	Used to establish the knowledge soundness of the Groth16 zk-SNARK proof system.

Examples 3.19 and 2 have their costs measured in Table 4.2 under the "complete pairing" column. Furthermore, Definition 3.5 represents the formal property empirically tested in Section 4.1.1: the "OK" status obtained for the four combinations in Table 4.1 serves as experimental verification that Equation (4.1) holds. Finally, the security assumptions table directly underpins the analysis of soundness in Section 4.1.1, which relies on the hardness of the DLP and the q -PKE.

3.2.3 Miller's Algorithm

Pairings require the computation of the rational functions $f_{r,P}$ and $f_{r,Q}$ using Miller's algorithm. The computation of these functions relies on Miller's recursion formula: (Ghammam, 2016)

$$f_{i+j,P} = f_{i,P} f_{j,P} \frac{l_{[i]P,[j]P}}{v_{[i+j]P}}, \quad (3.21)$$

where:

- $l_{[i]P,[j]P}$ is the equation of the line passing through the points $[i]P$ and $[j]P$, and
- $v_{[i+j]P}$ denotes the equation of the vertical line at the point $[i+j]P$.

In each iteration of Miller's algorithm, we have two steps: the first is doubling, which is automatic whether the iteration bit is 0 or 1. The second is addition. It only takes place if the iteration bit is 1. Thus, having sparse intermediate polynomials and a sparse r , that is, polynomials with zero coefficients, would considerably reduce computation time.

In the literature, we find several versions of Miller's algorithm: **Classical Miller Algorithm** or **Standard Miller Algorithm**, **Euclidean Miller Algorithm** and **Fibonacci Miller Algorithm**. In modern implementations of BN254 and BLS, the classic Miller

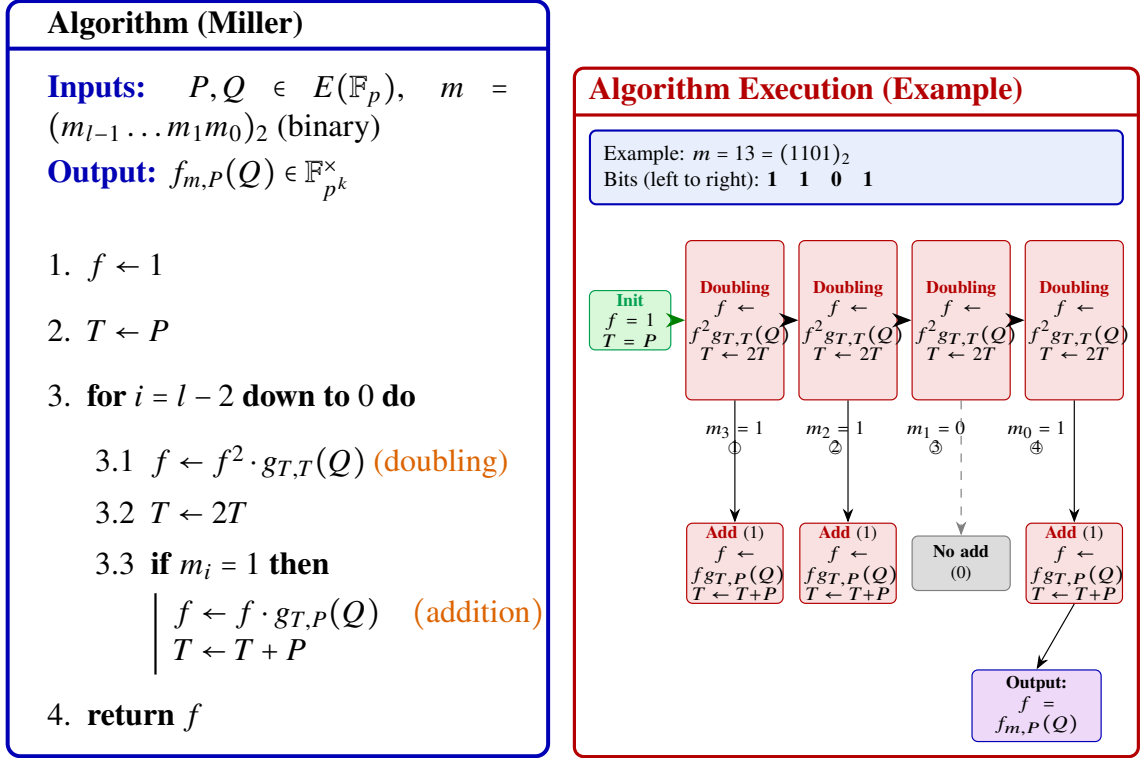


Figure 3.6: Miller's algorithm execution

method remains the most widely used solution. Pairing computation is traditionally divided into two stages: the Miller loop and the final exponentiation. This algorithm corresponds to the "Miller loop" timed in Table (4.2). The distinction between the Miller loop ?? and the final exponentiation, introduced here, is necessary to correctly interpret this table: the two stages are measured separately before being recombined in the "complete pairing" column, allowing for the interpretation given in Section 4.1.3 namely, that the gap between the two curves stems from operations preceding the pairing (setup, scalar multiplications) rather than from the pairing itself.

3.2.4 Final Exponentiation: Four Methods Compared

It should be noted first of all that the BN and BLS curves have a degree of embedding k equal to 12.

Pairing Need: the Miller loop computes an intermediate value

$$f_{u,Q}(P) \in \mathbb{F}_{p^{12}}^*. \quad (3.22)$$

However, this value does not necessarily belong to the target group

$$\mathbb{G}_T = \mu_r, \quad (3.23)$$

where

$$\mu_r = \left\{ x \in \mathbb{F}_{p^{12}}^* \mid x^r = 1 \right\}.$$

In other words, several distinct elements of $\mathbb{F}_{p^{12}}^*$ may correspond to the same pairing value.

To eliminate this ambiguity and obtain a unique representative in the target group, the result of the Miller loop is raised to the exponent

$$\frac{p^{12} - 1}{r}. \quad (3.24)$$

The final pairing value is therefore

$$e(P, Q) = f_{u,Q}(P)^{\frac{p^{12}-1}{r}} \in \mu_r. \quad (3.25)$$

This operation, known as the *final exponentiation*, projects the intermediate result onto the cyclic subgroup of order r , thereby ensuring the non-degeneracy and uniqueness of the pairing.

Exponent decomposition:

The exponent is traditionally decomposed as follows:

$$\frac{p^{12} - 1}{r} = \underbrace{\frac{(p^6 - 1)(p^2 + 1)}{\Phi_{12}(p)}}_{\text{easy part}} \cdot \underbrace{\frac{\Phi_{12}(p)}{r}}_{\text{hard part}}, \quad (3.26)$$

where

$$\Phi_{12}(p) = p^4 - p^2 + 1.$$

Easy part: the easy part

$$\frac{(p^6 - 1)(p^2 + 1)}{\Phi_{12}(p)} \quad (3.27)$$

is computed mainly using: Frobenius maps, inversion, a small number of multiplications. Two Methods for Optimizing the Calculations of Squares in the Cyclotomic Group: **Karabina Method, Granger et Scott.**

Hard part: the hard part

$$\frac{\Phi_{12}(p)}{r}$$

does not admit any obvious algebraic simplification and constitutes the main computational bottleneck of the final exponentiation.

We now present the different methods for calculating the difficult part of the final exponentiation cited in the literature, notably **the naive method, Devegili and Scott, Scott et al or Chain Addition, Fuentes et al.** (Ghammam, 2016).

Table 3.4: Comparison of pairing methods

Methods	Cost
Naïve method	$759S + 335M$
Devegili and Scott	$196S + 26M + 4F$
Scott et al.	$190S + 19M + 7F$
Fuentes et al.	$189S + 16M + 3F$

In this table we illustrate the Miller and final exponentiation costs for the BN and BLS curves. From this table, (3.5) we can clearly see that the BN curves are faster than the BLS curves.

Table 3.5: Comparison of the computational complexities of couplings on different families of curves (extract from TABLE 4.10 of (Ghammam, 2016))

Cuves	Method	Miller Complexity	Loop	Final Exponentiation Complexity
BLS12	Aranha et al. AFK12	10865		$8524M + 6I$
	Notre travail	10785		$8116M + 6I$
BLS24	Aranha et al. AFK12	14927		$25412M + 10I$
	Notre travail	14574		$23864M + 10I$
BN	Aranha et al. AFK12	16553M		7218M + 4I
KSS18	Aranha et al. AFK12	13168M		$23821M + 8I$

This methodological comparison allows for a direct interpretation of the "final exponentiation" column in Table 4.2. Indeed, the cost measured using arkworks corresponds to the most optimized final exponentiation method available in the library; in this subsection, we therefore provide the framework necessary to compare this cost with the optimization methods presented earlier. Decomposing the final exponentiation into an easy part and a hard part also helps explain the hypothesis formulated in Section 4.1.3

3.2.5 Pairing-Friendly Curves: BN254 and BLS12-381

- The Barreto-Naehrig (BN) family of elliptic curves is defined over the finite field $\mathbb{F}_p$. With an embedding degree $k = 12$, 3.4 these curves are defined by the equation:

$$E : y^2 = x^3 + b \quad \text{with } b \neq 0, \quad (3.28)$$

parametrized by an integer u as follows:

$$\begin{cases} r = 36u^4 + 36u^3 + 18u^2 + 6u + 1, \\ p = 36u^4 + 36u^3 + 24u^2 + 6u + 1, \\ t = 6u^2 + 1. \end{cases} \quad (3.29)$$

The integer u is chosen such that p and r are primes. It is recommended to choose a sparse u in order to simplify the pairing calculation on these curves.

- The BLS12-381 curve is a pairing-friendly elliptic curve from the Barreto-Lynn-Scott family. Having an embedding degree $k = 12$, it is used in modern cryptographic systems based on bilinear pairings, notably SNARKs. It is defined over a large finite field $\mathbb{F}_p$, where p is a 381-bit prime number, hence its name BLS12-381. The parameters used and results of arithmetic calculations obtained by Loubna and Fouotsa in their work, (Ghammam & Fouotsa, 2016) are as follows. BLS curves are defined over $\mathbb{F}_p$ by the equation: $E : y^2 = x^3 + b$ and are parametrized by an integer

$u \in \mathbb{Z}$ such that:

$$\begin{cases} p = \frac{(u-1)^2(u^4-u^2+1)}{3} + u, \\ r = u^4 - u^2 + 1, \\ t = u + 1. \end{cases} \quad (3.30)$$

Where t is the trace of the Frobenius map on the curve, and u is chosen such that p and r are primes and have the sizes corresponding to the desired security level.

In this subsection, we have defined the two objects compared throughout Chapter 4 (Tables 4.1 to 4.6): without the parameterization of p , r , and t , we could neither justify the size of our base fields (254 bits versus 381 bits) nor explain the difference in security levels discussed in Section 4.1.3 (BN254: 100 to 110 bits, degraded by NFS attacks; BLS12-381: 128 bits guaranteed). It is this contrast nearly similar pairing costs but different security levels that drives the conclusion in Section 4.2.3 that BLS12-381 offers, in this context, a better security-to-cost ratio for the isolated pairing operation.

3.3 Rigorous Formal Analysis of Groth16

In this section, we will analyse different stages of the Groth16 protocol.

3.3.1 From Arithmetic Circuits to Quadratic Arithmetic Programmes

The Step 0 In this section, we have a problem, but not just any problem. ZKPs are designed to solve several fundamental problems related to confidentiality, verification, and trust in systems. The main problem these technologies solve is that of verification without revealing information. In our work, we consider the problem in Example 3.

Example 3. A professor says to his students: Prove to me that you know a solution to this equation,

$$E : x^2 - 2x - 15 = 0 \quad (3.31)$$

but I do not need to see it. This equation has two solutions: -3 and 5 . Now each student that we will call the Prouver and denote by P , will try to prove to the teacher that we call Verifier and denote by V , that he knows either -3 or 5 such that $E(-3) = 0$ or $E(5) = 0$ without revealing it to P . The different variables to be considered in this situation are $x : w = (1, x)$ w begins with 1 by convention. To do that, there are steps that we should follow.

We focus on the NP class, characterized by solutions that are difficult to find but quick to verify, which corresponds exactly to the zero-knowledge proof (ZKP) model. Deletes Figure (3.7).

Step1: Arithmetic circuits

Definition 3.6. (Shpilka & Yehudayoff, 2010) **An arithmetic circuit** Φ over the field $\mathbb{F}$ and the set of variables X (usually, $X = \{x_1, \dots, x_n\}$) is a directed acyclic graph as follows.

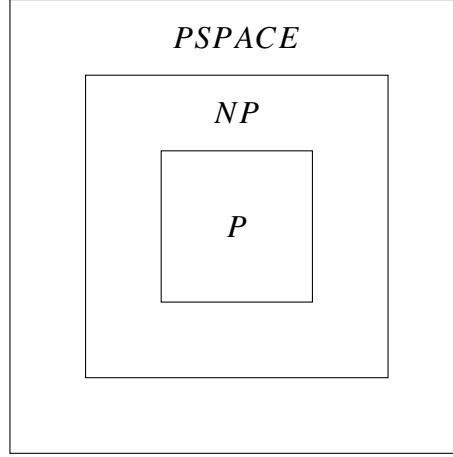


Figure 3.7: Link between $P \subseteq NP \subseteq PSPACE$.

The vertices of Φ are called gates. Every gate in Φ of in-degree 0 is labelled by either a variable from X or a field element from $\mathbb{F}$. Every other gate in Φ is labelled by either $\times$ or $+$ and has in-degree 2. An arithmetic circuit is called a formula if it is a directed tree whose edges are directed from the leaves to the root.

An arithmetic circuit is a system of equations using only addition, multiplication, and equality. It allows you to verify the validity of a given set of inputs. An arithmetic circuit is satisfied if there exists an assignment of variables called signals, such that all the equations are true. For our example, we can break down our equation E as follows:

$$y = x - 5, \quad z = x + 3, \quad yz = 0. \quad (3.32)$$

Each of these equations is then called a constraint.

Definition 3.7. (*Witness vector*) A **witness vector** is a vector that contains the values of all input variables, the output, and intermediate values.

By convention, this vector always starts with 1, (Zhu, 2025). Our witness vector:

$$w = \left[\underbrace{1, y}_{\text{public inputs}}, \underbrace{x, z}_{\text{private inputs}} \right] \quad (3.33)$$

Step2: Rank-1 Constraint System (R1CS)

Definition 3.8 (Hadamard product, (Horn & Johnson, 1991)). The **Hadamard product** of $A = [a_{ij}] \in M_{m,n}$ and $B = [b_{ij}] \in M_{m,n}$ is defined by $A \circ B [a_{ij}b_{ij}] \in M_{m,n}$

Definition 3.9. (*R1CS*) Given coefficient matrices $A, B, C \in F^{m \times n}$ and public input $v \in F^k$ ($k \leq n$), the **R1CS** instance defined by A, B, C, v is satisfied if there exists a witness $w \in F^n$ such that $w = [1, x_1, \dots, x_k]$, and

$$Aw \circ Bw = Cw, \quad (3.34)$$

where $\circ$ is element wise multiplication.

An R1CS can thus be seen like m constraints of the form

$$\langle q_a, w \rangle \cdot \langle q_b, w \rangle = \langle q_c, w \rangle, \quad (3.35)$$

where the selector vectors q_a , q_b , and q_c are rows of the coefficient matrices.

In our case, we are looking for matrices A , B , C with 4 columns because w has 4 variables, and with 3 rows because we have 3 constraints, such that we have this form:

$$C w = A w \circ B w$$

- A encodes the *left* variable of the multiplication.
- B encodes the *right* variable.
- C encodes *result variables*.
- A , B , C are matrices having the same number of columns like w : **4 columns** in our case.
- each column represents the same variable used at the same index.
- the number of lines correspond to the number of constraints : **3 constraints in our case**.

Let's determine these matrices.

Remarque 1. To determine the first line associated to y , we write it like $y = x - 5 = (x - 5) \cdot 1$. Meaning that $x - 5$ is the left part used to determine A , 1 is the right part used to determine B , and y for C .

Then,

$$y = -5 \cdot 1 + 1 \cdot x + 0 \cdot y + 0 \cdot z, \quad (3.36)$$

$$z = 3 \cdot 1 + 1 \cdot x + 0 \cdot y + 0 \cdot z, \quad (3.37)$$

$$0 = 0 \cdot 1 + 0 \cdot x + 1 \cdot y + 0 \cdot z. \quad (3.38)$$

$$A = \begin{pmatrix} -5 & 1 & 0 & 0 \\ 3 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \quad (3.39)$$

$$y = 1 \cdot 1 + 0 \cdot x + 0 \cdot y + 0 \cdot z, \quad (3.40)$$

$$z = 1 \cdot 1 + 0 \cdot x + 0 \cdot y + 0 \cdot z, \quad (3.41)$$

$$0 = 0 \cdot 1 + 0 \cdot x + 0 \cdot y + 1 \cdot z. \quad (3.42)$$

$$B = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \text{ by the same way, we obtain } C = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}. \quad (3.43)$$

And then, we obtain the rang1 form:

$$\underbrace{\begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}}_C \begin{pmatrix} 1 \\ x \\ y \\ z \end{pmatrix} = \underbrace{\begin{pmatrix} -5 & 1 & 0 & 0 \\ 3 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}}_A \begin{pmatrix} 1 \\ x \\ y \\ z \end{pmatrix} \circ \underbrace{\begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}}_B \begin{pmatrix} 1 \\ x \\ y \\ z \end{pmatrix}. \quad (3.44)$$

From there, we can do the Hadamard's product for each line in order to do the proof. But it will not be a ZKP, since w is not hidden.

Step3: QAP

Definition 3.10 (QAP). A **QAP** Q over field $\mathbb{F}$ contains three sets of $m + 1$ polynomials $V = \{v_k(x)\}$, $W = \{w_k(x)\}$, $Y = \{y_k(x)\}$, for $k \in \{0, \dots, m\}$, and a target polynomial $t(x)$.

Suppose F is a function that takes as input n elements of $\mathbb{F}$ and outputs n' elements, for a total of $N = n + n'$ I/O elements.

Then we say that Q computes F if $(c_1, \dots, c_N) \in \mathbb{F}^N$ is a valid assignment of F 's inputs and outputs, if and only if there exist coefficients $(c_{N+1}, \dots, c_m)$ such that $t(x)$ divides $p(x)$, where

$$p(x) = \left(\sum_{k=0}^m c_k v_k(x) \right) \left(\sum_{k=0}^m c_k w_k(x) \right) - \left(\sum_{k=0}^m c_k y_k(x) \right). \quad (3.45)$$

In other words, there must exist some polynomial $h(x)$ such that $h(x) \times t(x) = p(x)$. The size of Q is m , and the degree is the degree of $t(x)$, (Parno et al., 2016).

Before continuing, let's give an overview to clarify our ideas and know where we are going and what we are doing. As we mentioned earlier, we could stop at step 2, but the original purpose of the protocol would be violated, as it would no longer be a zero-knowledge disclosure protocol. Therefore, we must integrate other elements to conceal the secret as much as possible, while still allowing verification of the accuracy of calculations. Since the ZK protocol is, in effect, a verification of the correctness of a calculation.

The algorithm of a ZK protocol consists mainly of 3 algorithms which are: **KeyGen**, **Compute** and **Verify**.

V uses the first algorithm and generates a CRS which he sends to P and P uses it to generate his proof. This CRS (Common Reference String) document contains the following

elements:

$$[\tau^2 G_1, \tau G_1, G_1], \quad (3.46)$$

$$[\tau^2 G_2, \tau G_2, G_2], \quad (3.47)$$

$$[\psi_i]_1 = \frac{\alpha R_i(\tau) + \beta L_i(\tau) + O_i(\tau)}{\gamma}, \quad (3.48)$$

$$[\psi_i]_1 = \frac{\alpha R_i(\tau) + \beta L_i(\tau) + O_i(\tau)}{\delta}, \quad (3.49)$$

$$[\gamma_1, \gamma_0] = \left[\frac{\tau}{\delta} t(\tau) G_1, t(\tau) G_1 \right]. \quad (3.50)$$

where $\alpha, \beta, \tau, \gamma, \delta$ are random scalars.

We will detail what each participant will do while illustrating with our example. At the end, we will see if the verification is formally successful. We begin by detailing the contents of CRS, showing how it is constructed and the added value of each component that constitutes it.

In this step, we first seek to determine the polynomials A, B, and C. But to do so, we will need a new ingredient that is essential in the construction of these polynomials. Since we want polynomials, we are looking for a mathematical tool that will allow us to perfectly describe the matrix equation that we obtained in step R1CS.

Lagrange interpolation allows us to transform the R1CS circuit constraints into polynomials in Groth16, resulting in the QAP form.

In fact, at this construction's level, the verification would be too expensive because P would have to verify all the constraints. So how do we reduce a huge number of verifications to a single one? (the succinct property of SNARKs)

For each variable w_j of w , we know the coefficients it takes in each constraint: in fact, each matrix is associated with a polynomial. The columns of this matrix represent the variables of w , and for each column, its coefficients are each taken from a constraint.

Now, we have m constraints R1CS (3 in our case) and we choose m distinct points p_i (1, 2, 3 in our case). We then want to construct a polynomial that, when evaluated at p_i , exactly reproduces these coefficients. The Lagrange property guarantees their uniqueness.

For each variable of w , we have this; summing them, we obtain A.

Table 3.6: Coefficients of the matrix L

Variable	Constraint 1	Constraint 2	Constraint 3
1	-5	3	0
x	1	1	0
y	0	0	1
z	0	0	0

Assume that $j = 2$, corresponding to the variable x in the witness vector w .

The coefficients of x in the three constraints are: (1, 1, 0), corresponding respectively to constraints 1, 2, and 3.

We choose three distinct interpolation points p_1, p_2, p_3 . For each constraint i , we define the Lagrange basis polynomial $\ell_i(X) = \prod_{\substack{k=1 \\ k \neq i}}^3 \frac{X-p_k}{p_i-p_k}$, which satisfies $\ell_i(p_j) = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$

The polynomial associated with the variable x is then $L_x(X) = 1 \cdot \ell_1(X) + 1 \cdot \ell_2(X) + 0 \cdot \ell_3(X)$. Hence, $L_x(p_1) = 1, L_x(p_2) = 1, L_x(p_3) = 0$. More generally, for the variable indexed by j , we define $L_j(X) = \sum_{i=1}^m L_{i,j} \ell_i(X)$, where $L_{i,j}$ is the coefficient of variable j in constraint i .

$\mathbf{x}$ is one of the coefficients of $\mathbf{w}$. $\mathbf{X}$ is a variable used for Lagrange interpolation.

From now on, since $\mathbf{x}$ already has the value 5, we will consider $\mathbf{x}$ as $\mathbf{X}$. Finally, the polynomial

$A(X) = \sum_{j=0}^n w_j L_j(X)$. Now let us calculate the polynomials A,B,C of our case. We follow as we detail up. In the following, we will consider that, $x = 5, y = 5 - 5 = 0, z = 5 + 3 = 8, yz = 0, w = (1, 5, 0, 8)$.

Let's now determine $A(x), B(x), C(x)$

$$\begin{aligned} A \cdot w(X) &= \begin{pmatrix} -5 \\ 3 \\ 0 \end{pmatrix} \cdot 1 + \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} \cdot 5 + \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \cdot 0 + \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \cdot 8 \\ &= -5 \underbrace{\ell_1(X)}_{L_1(X)} + 3 \underbrace{\ell_2(X)}_{L_2(X)} + 5(\ell_2(X) + \ell_2(X)) + \dots \\ &= -\frac{5}{2}(X-2)(X-3) - 3(X-1)(X-3) + \frac{5}{2}(X-1)(X-3) - 5(X-1)(X-3) \\ &= -8(X-1)(X-3). \end{aligned} \tag{3.51}$$

$$B \cdot w(x) = \ell_1(x) + \ell_2(x) + 8\ell_3(x) \tag{3.52}$$

$$= \frac{(x-2)(x-3)}{2} - \frac{2(x-1)(x-3)}{2} + \frac{8}{2}(x-1)(x-2) \tag{3.53}$$

$$B(x) = \frac{7x^2 - 21x + 16}{2}. \tag{3.54}$$

$$C \cdot w(x) = 8\ell_2(x) = -8(x-1)(x-3). \tag{3.55}$$

Step4: Trusted Setup determination of $h(x)t(x) = A(x)B(x) - C(x)$

we know that:

$$t(x) = (x-1)(x-2)(x-3) = x^3 - 6x^2 + 11x - 6. \tag{3.56}$$

$$A(x) \cdot B(x) = -28x^4 + 196x^3 - 484x^2 + 508x - 192. \tag{3.57}$$

$$A(x) \cdot B(x) - C(x) = -28x^4 + 196x^3 - 476x^2 + 476x - 168. \tag{3.58}$$

Then,

$$h(x) = \frac{-28x^4 + 196x^3 - 476x^2 + 476x - 168}{x^3 - 6x^2 + 11x - 6} = -28(x - 1) = -28x + 28. \quad (3.59)$$

In the next, we will consider τ , a point on elliptic curve, at which the polynomial will be evaluated. P will take $\tau = 0$, then $t(\tau) = -6$

$$h(\tau)t(\tau) = (-28(x - 1)) \cdot (-6) = 168(x - 1), \quad (3.60)$$

$$h(\tau)t(\tau) = \langle [168, -168], [\tau, 1] \rangle = -168 \quad (3.61)$$

For the trusted setup, we will not give all the parameters simultaneously. We will introduce parameters progressively, such that we explain the use of each of them. That is it follow

$$[\sigma_2, \sigma_1, G_1] = [\tau^2 G_1, \tau G_1, G_1], \quad (3.62)$$

$$[\rho_2, \rho_1, G_2] = [\tau^2 G_2, \tau G_2, G_2], \quad (3.63)$$

$$[\gamma_1, \gamma_0] = [\tau t(\tau) G_1, t(\tau) G_1]. \quad (3.64)$$

As in the evaluation of $h(\tau)t(\tau)$, the three polynomials $A(x)$, $B(x)$ and $C(x)$, will be evaluated respectively at those points. $G_i, i = 1, 2$ are generator of cyclic groups over an elliptic curve.

After computing $A(\tau)$, $B(\tau)$ and $C(\tau)$, P will send to V for verification:

$$[A]_1 \cdot [B]_2 = [C]_1? \quad (3.65)$$

But from there we see that P can take any random a, b and c such that $a \cdot b = c$, and poses

$$[A]_1 = a, [B]_2 = b \quad \text{and} \quad [C]_1 = c \quad (3.66)$$

It appear there clearly P cannot be honest, and other coefficients without the coefficients of his w .

We take α, β such that: $[\alpha]_1 = \alpha \cdot G_1, [\beta]_2 = \beta \cdot G_2$

The SRS becomes then:

$$[\sigma_2, \sigma_1, G_1] = [\tau^2 G_1, \tau G_1, G_1], \quad (3.67)$$

$$[\rho_2, \rho_1, G_2] = [\tau^2 G_2, \tau G_2, G_2], \quad (3.68)$$

$$[\gamma_1, \gamma_0] = [\tau t(\tau) G_1, t(\tau) G_1], \quad (3.69)$$

$$[\psi_i]_1 = (\alpha R_i(\tau) + \beta L_i(\tau) + O_i(\tau)), \quad i = 1, 2, 3, 4. \quad (3.70)$$

P computes:

$$[A']_1 = [\alpha]_1 + \sum_{i=0}^4 w_i L_i(\tau) \cdot G_1, \quad (3.71)$$

$$[B']_2 = [\beta]_2 + \sum_i w_i R_i(\tau) \cdot G_2, \text{ and } [B_1] = [\beta]_1 + \sum_i w_i R_i(\tau) \cdot G_1, \quad (3.72)$$

$$[C']_1 = \sum_i w_i [\psi_i]_1 = \sum_i [\alpha R_i(\tau) + \beta L_i(\tau) + O_i(\tau)]. \quad (3.73)$$

V verifies:

$$[A]_1 \cdot [B]_2 \stackrel{?}{=} [\alpha]_1 \cdot [\beta]_2 + [C]_1 \cdot G_2. \quad (3.74)$$

Decomposition of $[C]_1$:

$$[C]_1 = \underbrace{\psi_1 + 5\psi_2}_{\text{public inputs}} + \underbrace{0 \cdot \psi_3 + 8\psi_4}_{\text{private inputs}}, \quad (3.75)$$

with :

$$\psi_1 = \alpha(L_1(\tau) + L_2(\tau)) + \beta(-5L_1(\tau) + 3L_2(\tau)), \quad (3.76)$$

$$\psi_2 = \beta(L_1(\tau) + L_2(\tau)), \quad (3.77)$$

$$\psi_3 = \beta L_3(\tau) + L_1(\tau), \quad (3.78)$$

$$\psi_4 = \alpha L_3(\tau) + L_2(\tau). \quad (3.79)$$

$$[A]_1 \cdot [B]_2 \stackrel{?}{=} [x]_1 \cdot [B']_2 + \underbrace{(\psi_1 + 5\psi_2) \cdot G_2}_{\text{public}} + \underbrace{(8\psi_4 + h(\tau)t(\tau)) \cdot G_2}_{\text{private}}. \quad (3.80)$$

Improvement of SRS: now, we see also that public and private inputs can be mix. Then, V can have access to the private part and gates the witness. This violates the Zero Knowledge property of the protocol.

To avoid it, we introduce two other parameters γ and δ such that:

public part:

$$[\psi_1]_1 = \frac{\psi_1}{\gamma} \quad (3.81)$$

$$[\psi_2]_1 = \frac{\psi_2}{\gamma} \quad (3.82)$$

private part:

$$[\psi_4]_1 = \frac{\psi_4}{\delta} \quad (3.83)$$

But we add also r and s to guarantee the Zero Knowledge, by making random the proof.

Improvement of SRS: then the new SRS becomes:

random scalars: $\alpha, \beta, \tau, \gamma, \delta$

$$[\tau^2 G_1, \tau G_1, G_1], \quad (3.84)$$

$$[\tau^2 G_2, \tau G_2, G_2], \quad (3.85)$$

$$[\psi_i]_1 = \frac{\alpha R_i(\tau) + \beta L_i(\tau) + O_i(\tau)}{\gamma}, \quad (3.86)$$

$$[\psi_i]_1 = \frac{\alpha R_i(\tau) + \beta L_i(\tau) + O_i(\tau)}{\delta}, \quad (3.87)$$

$$[\gamma_1, \gamma_0] = \left[\frac{\tau}{\delta} t(\tau) G_1, t(\tau) G_1 \right]. \quad (3.88)$$

Step5: Proof generation, now all is good and ready for a Zero Knowledge Proof.

Then to generate the proof, P will compute and send to V .

$$[A'']_1 = [\alpha]_1 + [A]_1 + r[\delta]_1, \quad [\delta]_i = sG_i, \quad (3.89)$$

$$[B'']_2 = [\beta]_2 + [B]_2 + s[\delta]_2, \quad (3.90)$$

$$[C'']_1 = \sum_{i=\ell+1}^i (w_i \psi_4) + h(\tau)t(\tau) + A \cdot s + B \cdot r - rs[\delta]_1. \quad (3.91)$$

V verifies by computing:

$$e([A'']_1, [B'']_2) = e(\alpha G_1, \beta G_2) \cdot e(V_{kx}, \gamma G_2) \cdot e(C'', \delta G_2), \quad (3.92)$$

where

$$V_{kx} = \frac{\sum_{i=1}^{\ell} w_i [\psi_i]_1}{\gamma}.$$

Step6: Formal verification:

$$e([A'']_1, [B'']_2) = e(G_1, G_2)^{A''B''} \quad (3.93)$$

$$= e(G_1, G_2)^{\left(\alpha + \sum_i w_i L_i(\tau) + r\delta\right) \left(\beta + \sum_i w_i R_i(\tau) + s\delta\right)} \quad (3.94)$$

$$\left(\alpha + \sum_i w_i L_i(\tau) + r\delta\right) \left(\beta + \sum_i w_i R_i(\tau) + s\delta\right) \quad (3.95)$$

$$= (\alpha\beta + \alpha \sum_i w_i R_i(\tau) + \alpha s\delta + \beta \sum_i w_i L_i(\tau) + \sum_i w_i L_i(\tau) \cdot \sum_i w_i R_i(\tau) \quad (3.96)$$

$$+ s\delta \sum_i w_i L_i(\tau) + r\delta\beta + r\delta \sum_i w_i R_i(\tau) + rs\delta^2 \quad (3.97)$$

$$= \alpha\beta + \alpha\beta A'' + \beta A'' + \sum_i w_i L_i(\tau) \cdot \sum_i w_i R_i(\tau) + s\delta \left(\alpha + \sum_i w_i L_i(\tau) + r\delta\right) \quad (3.98)$$

$$+ r\delta \left(\beta + \sum_i w_i R_i(\tau) + s\delta\right) - rs\delta^2 \quad (3.99)$$

$$= \alpha\beta + \alpha B'' + \beta A'' + s\delta A'' + r\delta B'' - rs\delta^2 + \sum_i w_i L_i(\tau) \cdot \sum_i w_i R_i(\tau) \quad (3.100)$$

$$= \alpha\beta + \alpha B'' + \beta A'' + s\delta A'' + r\delta B'' + h(\tau)t(\tau) - O(\tau) - rs\delta^2 \quad (3.101)$$

$$= \alpha\beta + \sum_i w_i (\alpha L_i + \beta R_i - O_i)(\tau) + h(\tau)t(\tau) + s\delta A'' + r\delta B'' - rs\delta^2 \quad (3.102)$$

$$= \alpha\beta + \sum_i w_i \psi_i + h(\tau)t(\tau) + s\delta A'' + r\delta B'' - rs\delta^2 \quad (3.103)$$

So at the end we obtain:

$$e([A'']_1, [B'']_2) = e(G_1, G_2)^{\alpha\beta + \sum_i w_i \psi_i + h(\tau)t(\tau) + s\delta A'' + r\delta B'' - rs\delta^2} \quad (3.104)$$

in the left hand.

In the other hand, we have:

$$\begin{aligned} e(\alpha G_1, \beta G_2) \cdot e(\gamma V k_x, G_2) \cdot e(\delta C'', G_2) \\ = e(G_1, G_2)^{\alpha\beta} \cdot e(G_1, G_2)^{\gamma V k_x} \cdot e(G_1, G_2)^{\delta C''} \end{aligned} \quad (3.105)$$

$$= e(G_1, G_2)^{\alpha+\gamma V k_x+\delta C''} \quad (3.106)$$

$$= e(G_1, G_2)^{\alpha+\gamma V k_x+\delta C''}, \text{ by replacing } V k_x \text{ and } C'' \quad (3.107)$$

$$= e(G_1, G_2)^{\alpha\beta+\sum_i w_i \psi_i+h(\tau)t(\tau)+s\delta A''+rB''\delta-rs\delta^2}, \text{ and we hold} \quad (3.108)$$

Application With Our Example: for application, we take $\tau = 0$, $\alpha = 3$, $\beta = 5$, $\delta = 7$, $r = 2$, $s = 4$, $\gamma = 11$.

Computation:

Lagrange polynomials: $\ell_1(x) = \frac{(x-2)\cdot(x-3)}{2}$

$$\ell_2(x) = \frac{(x-1)\cdot(x-3)}{2}$$

$$\ell_3(x) = \frac{(x-1)\cdot(x-2)}{2}, \text{ now at } \tau, \text{ we obtain:}$$

$$\ell_1(0) = 3$$

$$\ell_2(0) = -3$$

$$\ell_3(0) = 1$$

$$[A''(\tau)]_1 = \alpha + \sum_i w_i L_i(\tau) + r\delta = 3 + (-24 + 0 + 0 + 0) + 2 \cdot 7 = -7G_1$$

$$[B''(\tau)]_1 = \beta + \sum_i w_i R_i(\tau) + s\delta = 3 + (1 \cdot R_1 + 5 \cdot R_2 + 0 \cdot R_3 + 8 \cdot R_4) + 4 \cdot 7 = 41G_1 \text{ and}$$

$$[B''(\tau)]_2 = 41G_2$$

$$t(\tau) = -6, \quad s[A''(\tau)]_1 = 4 \cdot (-7) = -28, \quad r[B''(\tau)]_1 = 2 \cdot 41G_2, \quad rs\delta = 2 \cdot 4 \cdot 7 = 56,$$

$$\psi_1 = \alpha L_1 + \beta R_1 - O_1 = 3 \cdot 0 + 5 \cdot 0 - 0 = 3, \quad \psi_2 = -120, \quad \psi_3 = 8, \quad \psi_4 = 0, \quad h(0) =$$

$$28(0 - 1), \quad h(0)t(0) = 28 \cdot (-6) = -168, \quad \frac{h(0)t(0)}{\delta} = \frac{-168}{7} = -24, \quad [\gamma_1, \gamma_0] =$$

$$[\frac{0}{7}(-6)G_1, \frac{-6}{7}G_1] = [0 \cdot G_1, \frac{-6}{7}G_1], \quad [C'']_1 = 0 - 24 - 28 + 82 - 56 = -26.$$

Then the P send to V: $[-7G_1, 41G_2, 41G_1, 104G_1]$

V computes:

$$V k_0 = \frac{\sum_{i=1}^2 w_i [\psi_i]_1}{\gamma} = \frac{-120}{11}. \quad (3.109)$$

$$e([A''(\tau)]_1, [B''(\tau)]_2) = e(-7G_1, 41G_2) \quad (3.110)$$

$$= e(G_1, G_2)^{-7 \cdot 41} \quad (3.111)$$

$$= e(G_1, G_2)^{-287} \quad (3.112)$$

$$e(\alpha G_1, \beta G_2) \cdot e(\gamma V k_x, G_2) \cdot e(\delta C'', G_2) \quad (3.113)$$

$$= e(3G_1, 5G_2) \cdot e\left(\frac{-120}{11}G_1, 11G_2\right) \cdot e(-26G_1, 7G_2) \quad (3.114)$$

$$= e(G_1, G_2)^{15} \cdot e(G_1, G_2)^{-120} \cdot e(G_1, G_2)^{-182} \quad (3.115)$$

$$= e(G_1, G_2)^{-287} \quad (3.116)$$

So there is equality between (3.112) and (3.116), we hold. The student have show to their prof the knowledge of some x without revealing it.

3.3.2 The Groth16 Protocol Diagram

This diagram is a generalized view of the Groth16 protocol, starting from a natural problem and leading to the verification of the proof, see 3.8.

3.4 BLS signature

In this section, we used (Boneh et al., 2001). We give the mechanism of the signature firstly, follows by the contribution of BLS to increase public Blockchains.

3.4.1 Mechanism

The BLS signature scheme has a length of 170 bits. It is based on ECDH problems given in 2.1. The scheme is based on three algorithms: **Key generation**, **Signing**, **Verification**. In the following, we describe each of them.

Setup: G_1 and G_2 elliptic curve subgroups, g_1, g_2 their generators, $|G_1| = |G_2| = p$ where p is a prime number.

Key generation: we choose a random $x \in \mathbb{Z}_p$ and compute

$$v = g_2^x \quad (3.117)$$

Then,

Private key $K_{\text{priv}} = x$,

Public key $K_{\text{pub}} = v$.

Signing: we define a hash function

$$H : \{0, 1\}^* \longrightarrow G_1, \quad (3.118)$$

Given a private key $x \in G_2$, a message $M \in \{0, 1\}^*$, and a signature $\sigma \in G_1$, we compute

$$h = H(M) \quad \text{and} \quad \sigma = h^x. \quad (3.119)$$

The signature is σ in G_1 .

Verification: given a public key $v \in G_2$, a message $M \in \{0, 1\}^*$ and a signature

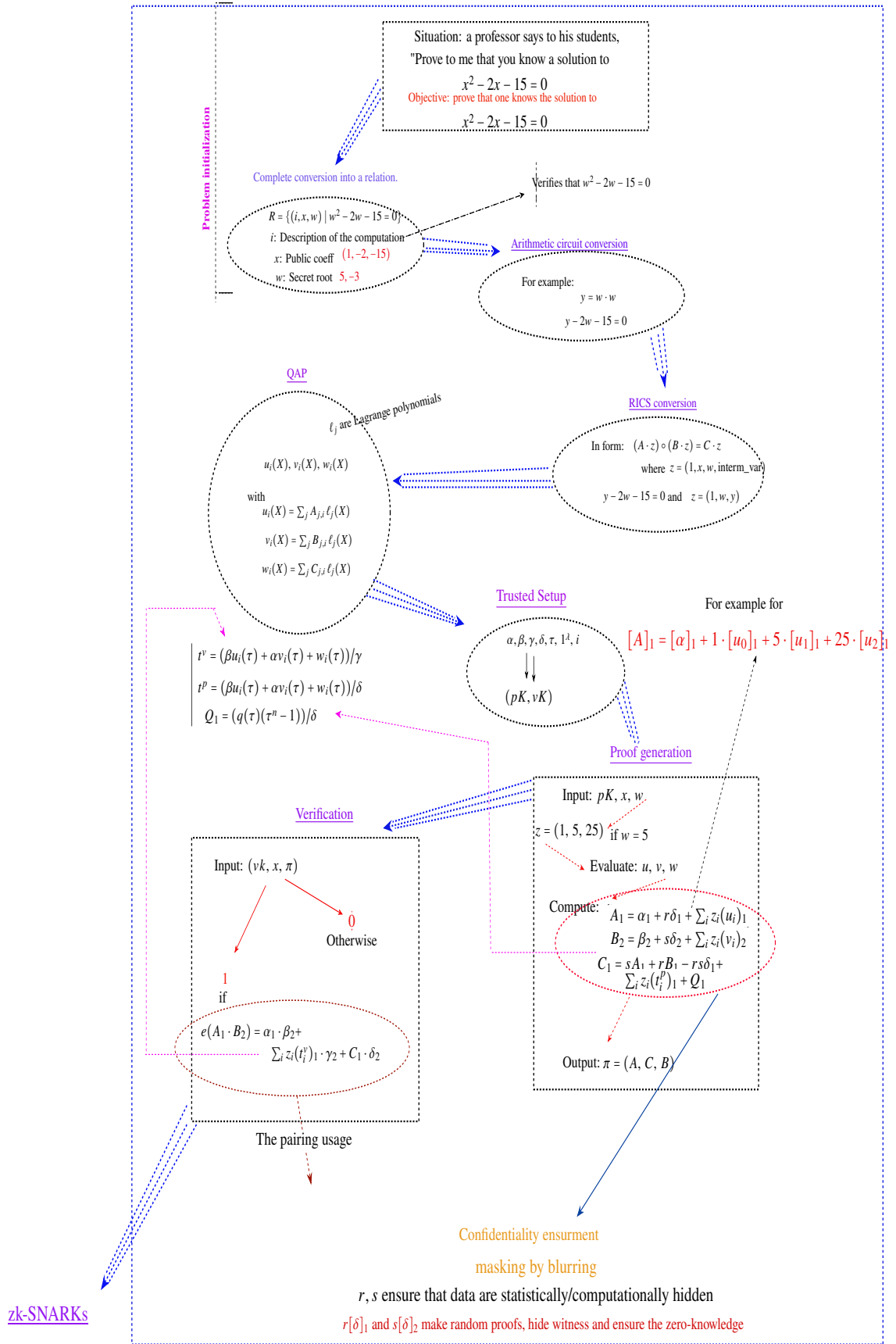


Figure 3.8: Groth16 Pipeline Process

$\sigma \in G_2$, let e be a Weil pairing. We verify that

$$e(v, h) = e(g, \sigma) \quad (3.120)$$

For the verification proof, see A.2

3.4.2 BLS contribution in Blockchain scalability

They developed schemes called aggregate signatures, which allow combining several independent signatures into a single verifiable signature. By using bilinear pairings, BLS signatures compress multiple signatures into a single signature of constant size, while maintaining security guarantees.

Aggregate signatures: let now consider n parties having a public key pair (x_i, v_i) , $x_i \in \mathbb{Z}_p$, $v_i = g_2^{x_i} \in G_2$ and $i = 0, \dots, n$.

We assume that each i signs a message $M_i \in \{0, 1\}$: $\sigma_i = h^{x_i} \in G_1$. The aggregate of those signature is:

$$\sigma = \sigma_1 \sigma_2 \dots \sigma_n \in G_1 \quad (3.121)$$

To verify this signature aggregation, the following properties should hold:

- The messages M_i should be distinct,
-

$$e(\sigma, g_2) = \prod_{i=1}^n e(H(M_i), v_i) \quad (3.122)$$

Here, each signature is verified, and there is not real gain.

Batch verification: from the above, each user signs their message. Now, all users will sign a single message $M \in \{0, 1\}$. The verification will be too much for the n signatures, but it will be faster than the verification done for each signature individually. The verification mechanism is described as follows:

- We choose randomly $c_1, \dots, c_n$, then compute

$$V = \prod_{i=1}^n v_i^{c_i} \quad \text{and} \quad U = \prod_{i=1}^n \sigma_i^{c_i}. \quad (3.123)$$

- Then we do this single verification:

$$e(U, g_2) = e(H(M), V). \quad (3.124)$$

If (3.124) hold, the n signatures are accepted. Otherwise, they are rejected.

For the proof, see A.2.

This technique improves the scalability of blockchains through the aggregation of multiple signatures, thereby reducing the amount of data to be stored and transferred over the network. The presence of a single set of signatures for the verifier also reduces the cost of verification.

3.5 ZK-Rollups Pipeline

From (Thibault et al., 2022a), several solutions exist to improve scalability by building level 2 layers, among which we have: **State channels**, **Sidechains**, **Plasma**, **Rollups** and **Validiums**. There exists two types of Rollups: **Optimistic Rollups** and **zk-Rollups**.

As mentioned in section 3.1.6, the Ethereum Blockchain has very low scalability, which is 15 tx/s. Zk-Rollups have solved this problem by grouping hundreds and thousands of off-chain transactions, producing a single zk-SNARK proof, and then submitting it alone to the main or L1 blockchain for verification. The pipeline then guarantees: completeness; for a valid transaction, its proof is accepted by L1. Soundness: an invalid transaction will only produce proof unacceptable to L1. A system incorporating a zk-RollupS is composed as follows:

- **Client layer**, where users create, transfer and view.
- **Layer2 or off-chain**, where incoming transactions are grouped into batches and a zkp proof is generated to verify their validity.
- **Layer1 or on-chain**, the non-primary network verifies the ZKP proof and stores the batch commitment as a Merkle root, if the proof is valid. (Siddiqui et al., 2026)

From the description below, the roles and architecture of L1 and L2 are given below.

Layer1 Roles Stores the Merkle root of the current state, verifies the validity of transactions, and ensures the availability of compressed transactions.

Layer2 Roles Similarly for L2, being the off-chain environment, it orders the transactions, generates a proof, compresses the data and submits it to L1.

Architecture of L1 and L2: the architecture of L1 and L2 is given by 3.9. But how can we determine the cost of such a system?

3.5.1 Cost Model and Scalability Analysis

The costs are broken down into two parts:

- **L1 cost:** verifying a Groth16 proof is dominated by a few bilinear pairings on elliptic curves and a small MSM on public inputs. On Ethereum, for example, the verification cost is generally between 200 kGas and 250 kGas.
- **L2 cost:** occurring during the proof generation phase, MSMs on additive groups are fundamental and expensive for zk-SNARKs, including Groth16. FFTs/NTTs are also used. Their respective complexities are $O(n)$ and $O(n \log n)$. Execution times given in the literature and industrial implementations vary from a few seconds on GPUs to several tens of seconds on CPUs, depending on the hardware and optimizations used.

Mathematical Model of Scalability

If a batch contains N transactions, and if

The proof sent to L1 (on-chain) is verified in $O(1)$.

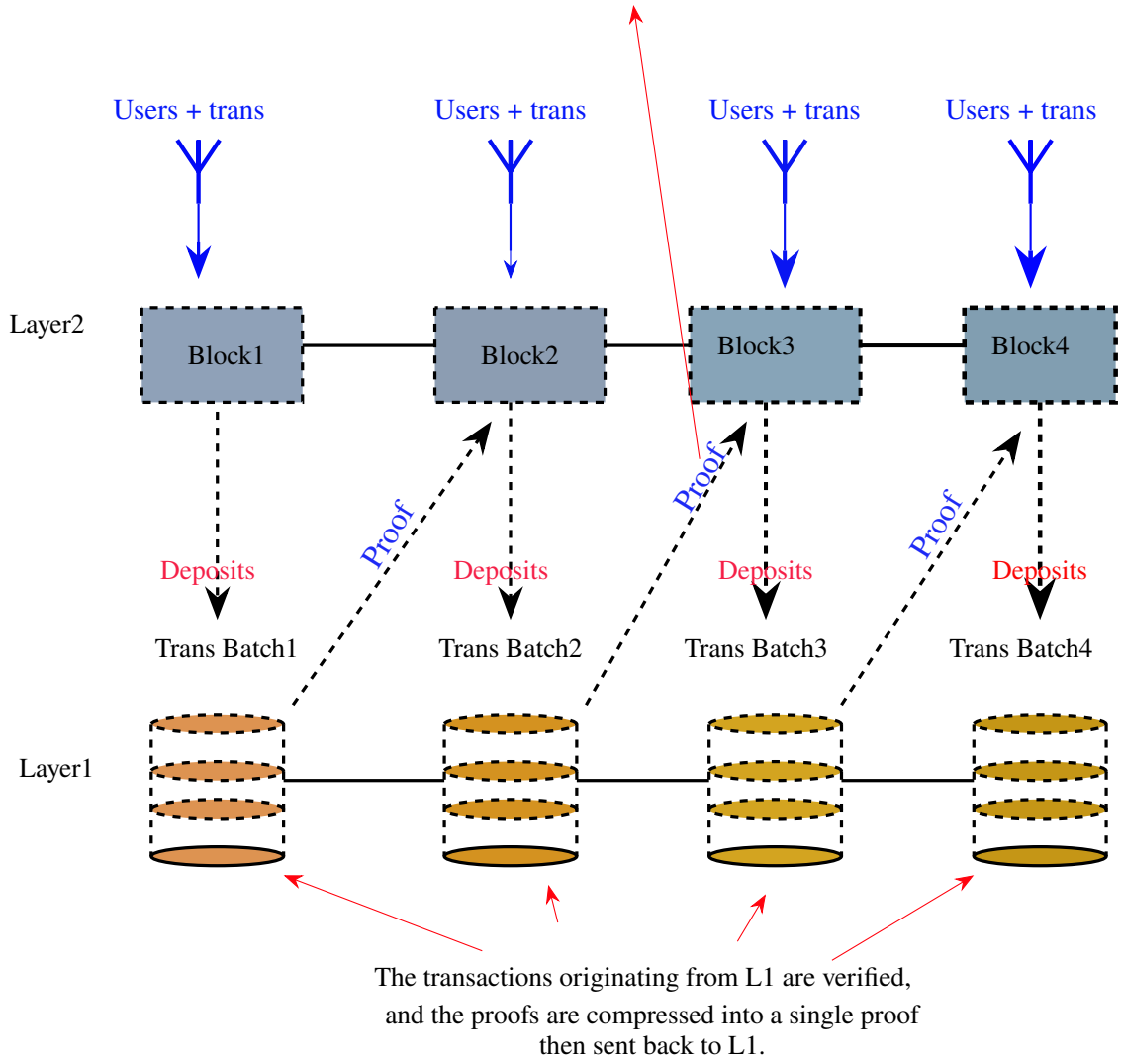


Figure 3.9: ZK-Rollup Transaction Process, illustrating where zk-SNARKs is used in Blockchain

- C_{verify} is the fixed cost of verifying the proof on L1;
- C_{data} is the average cost of publishing data per transaction;

then the average cost per transaction is

$$\text{Gas/tx} = \frac{C_{\text{verify}} + NC_{\text{data}}}{N} = \frac{C_{\text{verify}}}{N} + C_{\text{data}}. \quad (3.125)$$

As

$$N \rightarrow \infty,$$

we obtain

$$\lim_{N \rightarrow \infty} \text{Gas/tx} = C_{\text{data}}.$$

Thus, in a sufficiently large ZK-Rollup, the dominant cost is no longer cryptographic verification but data availability.

3.6 Experimental Setup

All experiments in this thesis were conducted on a personal computer running Windows 10 Pro (version 22H2, OS build 19045.7417) with the following hardware specifications:

1. **Operating System:** Windows 10 Pro (Version 22H2, OS Build 19045.7417)
2. **Processor (CPU):** Intel Core i5-6500 @ 3.20 GHz
3. **Memory (RAM):** 32 GB installed (31.7 GB usable)
4. **Storage:**
 - Intel SSDSC2KF256H6 SATA SSD, 256 GB
 - Samsung MZVLW256 NVMe SSD, 256 GB
5. **System Architecture:** 64-bit operating system, x64-based processor
6. **GPU Accelerator:** we do not use GPU acceleration in the benchmarks. All our experience were run exclusively on the CPU.

For cryptographic performance tests, the Rust language was used (for its qualities: memory safety: preventing measurement interference; cryptographic maturity. Arkworks is more widely used for the research and development of ZKP systems and contains ready crates for Marlin, Groth16, PLONK; see arkworks.rs), with the arkworks library which specializes in zero knowledge proofs, specifically the following libraries:

- **ark-groth16** to implement groth16
- **BN254** for cryptographic operations on BN254 curve
- **BLS12-381** for cryptographic operations on BLS12-381 curve

Details regarding the Rust compiler and the Arkworks benchmark version are in the Table, 3.7

Table 3.7: Rust compiler and Arkworks version

Component	Version
Rust Compiler	rustc 1.96.0 (ac68faa20 2026-05-25)
ark-groth16	0.4.0
ark-bn254	0.4.0
ark-bls12-381	0.4.0

3.6.1 Benchmark Circuit

The circuit used in this implementation encodes the polynomial $x^2 - 2x - 15 = 0$, with $x = 5$ serving as both the solution and the secret. Although simple, the circuit replicates the R1CS structure of the Groth16 protocol, enabling the measurement and reproduction of the underlying cryptographic operations. This circuit allows for a direct correspondence

with the formal analysis conducted in Chapter 3, simultaneously validating both the theory and the implementation something that could not be verified by hand with a production circuit. Furthermore, the objective here is to assess the impact of the curve choice rather than the circuit size. However, this circuit has limitations; notably, Table 4.1 measures a fixed pipeline cost rather than a marginal cost per constraint. The circuit’s homogeneous structure does not reflect the heterogeneity of a real-world circuit (involving hashing, Merkle trees, and signatures). Yet, it should be noted that even large-scale synthetic circuits (4.3- 4.6) remain generic and do not represent an actual transaction.

We defined our circuit using the BN254 and BLS12-381 elliptic curves and employed identical experimental conditions to enable a valid performance comparison between the two curves.

For the comparison, we recorded the following data for each curve.

- **Setup:** generation of the proving and verification keys;
- **Prove:** generation of the zero-knowledge proof;
- **Verify:** verification of the proof against the public inputs;
- **Miller loop:** the core pairing sub-operation;
- **Final exponentiation:** the concluding step of the pairing computation;
- **Full pairing:** the complete pairing operation combining Miller loop and final exponentiation;
- **MSM** $\mathbb{G}_1 \times 3$ MSM over three points in $\mathbb{G}_1$.

3.7 Software Environment Specifications and Circuit Configuration

3.7.1 Detailed Specifications of the Hardware and Software Environment

To eliminate any evaluation bias and ensure strict reproducibility of the experiment, the set of benchmarks was executed on a physical machine whose characteristics are summarized in the Table (3.8).

Table 3.8: Hardware Specifications of the Experimental Computer

Component	Specification
Processor	Intel® Core™ i5-6500 CPU @ 3.20 GHz, 64-bit (x64) architecture, base frequency of 3.19 GHz.
Memory (RAM)	32.0 GB installed (31.7 GB usable), providing sufficient capacity to prevent excessive memory paging during the execution of large-scale circuits.
Storage	Hybrid dual-storage configuration consisting of an Intel SATA SSD (SSDSC2KF256H6, 256 GB) and a Samsung NVMe SSD (MZVLW256).
Graphics	Intel® HD Graphics 530 with 128 MB dedicated video memory.

Compilation chain and software optimizations: the code was compiled under the Rust environment because of its raw performance (eliminates execution pauses, enhancing accuracy in time measurement), the security of memory management, and the maturity of its libraries. The code was executed in release mode (`cargo run -release`) with the following directives in the Cargo.toml file:

- **opt-level = 3:** prioritizes speed of execution,
- **lto = "thin":** optimizes the entire program with a good compromise between performance and compilation time.
- **codegen-units = 1:** favors global optimizations by sacrificing compilation speed,
- **panic = "abort":** reduces the size of the binary and simplifies the handling of fatal errors.

The library `rand::rngs::OsRng` is used for generating secure random values for the secret, while `blake2::Blake2b` serves as a hash function applied to transform the interactive protocol into a non-interactive one via the Fiat-Shamir heuristic.

3.7.2 Circuits

The experimental methodology of this work is divided into two phases:

- **small circuit: the Baseline** Here, we will measure and analyze the performance of the protocol studied using the small circuit used as an example in this paper: $x^2 - 2x - 15 = 0$ reduced to $v_1 - 2x - 15 = 0$ with $v_1 = x \times x$.
- **macro circuit:**
 - since the Groth16 and Marlin protocols both use the R1CS, the same small circuit was made macro for the second phase of the experiment. Indeed, we built a circuit loop controlled by a parameter k . A counter i from 0 to $N = 2^k - 1$ allows the system to generate 2^k independent constraints and instantiates the new witness with x_i .
- for Halo2, using PLONKish algorithms, we imported official PSE libraries via Cargo their GitHub repositories (*Privacy & Scaling Explorations*), a separate circuit. The circuit is arithmetically equivalent and parameterized so that its size can be varied. The circuit is governed by this equation

$$\begin{cases} x_i \times x_i = v_{1,i} \\ v_{1,i} - 2x_i - 15 = 0 \end{cases} \quad (3.126)$$

$$s_mul \times (x_{sq} - x \times x) = 0 \quad (3.127)$$

CHAPTER 4

RESULTS AND DISCUSSION

In this chapter, we present and analyze the results obtained during analysis, implementation and experimental evaluation, and let's discuss its significance in relation to specific objectives. To address the recommendation made in our reviewed reference articles to conduct a thorough and robust analysis of ZKP protocols, we chose the same protocols, Groth16 and Halo2, but replaced PLONK with Marlin, a newer technology. This is because Halo2 is based on the same PLONK architecture, where improvements have allowed for a reduction in proof size and verification time.

4.1 Results

In the first section, we present experimental results using the Groth16 technology on the BN254 and BLS12-381 curves. The objective of this experiment is to perform a comparative analysis of the algorithms for calculating bilinear pairings.

4.1.1 Zk-SNARKs (Groth16): Data Confidentiality and Integrity Analysis

The Groth16 protocol ensures data security in the blockchain through two formal cryptographic mechanisms:

- **confidentiality (perfect Zero-Knowledge):** thanks to the randomly chosen elements (δ, γ) , the prover's algorithm masks and avoids any leakage of information concerning the witness's private variables, during the construction of the final elements of the proof A, B, C . A Blockchain user will only see a series of points on a curve.
- **integrity (soundness):** it relies on the hardness of DLP and q -PKE. A user who does not possess the secret has a negligible probability of providing a valid proof, since the elements must satisfy:

$$e(A, B) = e(\alpha, \beta) + e\left(\sum_{i=0}^l \frac{\beta x_i + \alpha x_i}{\gamma}, \gamma\right) + e(C, \delta) \quad (4.1)$$

Table 4.1: Benchmark Groth16 - Setup, Prove, Verify on BN254 and BLS12-381

Curve	x	Setup	Prove	Verify	Status
BN254	+5	7,51 ms	2,33 ms	1,627 ms	OK
BN254	-3	6,60 ms	2,62 ms	1,885 ms	OK
BLS12-381	+5	13,22 ms	3,38 ms	2,513 ms	OK
BLS12-381	-3	16,98 ms	5,95 ms	2,312 ms	OK
Ratio BLS12-381 / BN254 ($x = 5$)		1,76×	1,45×	1,54×	

Observations:

- The soundness property has been experimentally validated across all executions of the table 4.1. All the evidence produced by the prover has an "OK" status, meaning that it has been accepted by the verifier. The BLS12-381/BN254 ratio for $x=5$ is 1.76x for setup, 1.45x for proof and 1.54x for verification.
- The Groth16 protocol formally relies on two properties: confidentiality (zero-knowledge), ensured by the random elements δ and γ , and integrity (soundness), based on the difficulty of DLP and the q-PKE assumption. No verification failures were observed across all large-scale executions: Table 4.4 to 4.6. Therefore, no proofs were rejected. But the confidentiality verification in this thesis was just a formal verification.

Interpretation:

- Achieving "OK" status on all four tested combinations constitutes direct experimental validation of the formal construction presented in Chapter 3: the verification equation (4.1) is satisfied for each generated proof, which is consistent with the completeness property of the Groth16 analysis protocol.

The fact that all three phases of the pipeline are slower on BLS12-381 (ratio > 1) is consistent with the arithmetic observations in Section 4.2.1, where the majority of elementary operations are also slower on this curve. This internal consistency between the two levels of measurement (isolated elementary operations vs. complete pipeline) strengthens the reliability of both datasets, but does not constitute independent proof of their absolute correctness.

- The absence of verification failures across all tested executions is consistent with the theoretical robustness property of the protocol: an honest prover, possessing a valid witness, always produces an accepted proof. This experimental protocol in this thesis was not designed for direct testing: a simulation or indistinguishability experiment. The experiments confirm the robustness observed in practice but do not provide an empirical test of the zero-knowledge property. Confirming the theory of the article (Liang et al., 2025)

4.1.2 Layer 2 Architecture Deployment and Scalability

Observations: the Layer 2 mechanism tested relies on three steps: off-chain transaction processing, compression into a single proof, and then verification of this single proof on Layer 1.

Tables (4.3), (4.4), (4.5) report, for Groth16, Marlin, and Halo2 respectively, the verification time and proof size across five circuit scales: $k=9$ to $k=22$. The verification time ranges from 2.6 ms to 17.9 ms across all these combinations, while the number of circuit

constraints varies from 512 to 4,194,304. The measured proof size is constant for each protocol, regardless of the scale: 128 B (Groth16), 352-464 B (Halo2), and 743-951 B (Marlin) (Figures 4.4).

Table 4.5 and Figure 4.4 jointly report, at $k=22$, the proof time and proof size of the three protocols: 307.090s/128B (Groth16, BN254), 792.010s/743B (Marlin, BN254), 93.037s/352B (Halo2, BN254), and the corresponding values on BLS12-381.

Interpretation: The stability of verification time and proof size, even as the circuit size increases, is consistent with the expected succinctness of the three tested protocols: the cost borne by Layer 1 (on-chain verification) does not depend on the complexity of the off-chain computation. This enables the scalability targeted by the Layer 2 architecture: an increasing number of transactions can be processed off-chain without increasing the on-chain verification cost. Ranking the three protocols by proof time at $k=22$ (Halo2 < Groth16 < Marlin) and by proof size (Groth16 < Halo2 < Marlin) indicates that, no protocol dominates simultaneously on both criteria: Halo2 minimizes generation time, while Groth16 minimizes the proof size transmitted to Layer 1. For a Layer 2 deployment where on-chain bandwidth is the limiting factor, Groth16 would be preferable. For a deployment where evidence generation latency is critical, Halo2 would be preferable 4.4.

4.1.3 Baseline Analysis: Groth16 Arithmetic Analysis, Curve Selection and Analysis

Arithmetic Cost

Table 4.2: Groth16 Benchmark - arkworks - circuit: $x^2 - 2x - 15 = 0$ (solutions: $x = 5$, $x = -3$) - Phase 1: Elementary operations

Courbe	Mul G1	Mul G2	Miller loop	Final exp	Pairing	MSM G1x3
BN254	157.4us	459.3us	573.0us	630.0us	1707.0us	372.8us
BLS12-381	290.4us	787.6us	705.0us	903.0us	1705.0us	541.9us

Observations:

1. Table (4.2) presents the times measured for the 6 elementary operations scalar multiplication in both groups, Miller loop, final exponentiation, full MSM pairing on the BN254 and BLS12-381 curves with the arkworks implementation.
2. 5 of the 6 operations are slower on BLS12-381. Complete pairing is an exception, with close values between the two curves: 1707 ms on BN254, 1705 ms on BLS12-381.
3. The most significant difference is observed in the scalar multiplication on G_2 : 459.3ms versus 787.6ms, representing a factor of approximately 1.7x.

Curves: Security Analysis

- **BN254 curve:** attacks on the NFS algorithm have lowered the security level of the curve. It now fluctuates between 100 and 110 bits of security, which is below the required standard level of 128 bits.
- **BLS12-381 curve:** with a base field of 381 bits, this curve offers security guaranteeing the minimum level of 128 bits.

Interpretation: the fact that the full pairing has a similar cost for both curves could be explained by: either a more advanced optimization of the Miller loop for BLS12-381 in Arkworks, or a trade-off between the high cost of scalar multiplication and a more efficient execution of the final exponentiation on this curve. Current data does not allow us to choose between these hypotheses; an analysis at the source code level of the library is necessary.

Our observation is specific to our Arkworks implementation, the hardware, and the configuration described in Chapter 3 section (3.6); another library (RELIC, BLST, GNARK) could produce a different, or even reversed, difference.

The contrast between a similar pairing cost and a significantly higher level of security for BLS12-381 (128 bits versus 100-110 bits) suggests that, in this specific context, BLS12-381 offers a better security/cost ratio for the isolated pairing operation, although this advantage is not necessarily found over the entire pipeline.

4.1.4 Large-Scale Multi-Protocol Global Comparative Analysis

Comparative Evaluation of the Two Curves: the following tables represent the overall cross-performance behaviour of Groth16, Marlin and Halo2 from our experience.

Table 4.3: **Groth16** Benchmarks on BN254 and BLS12-381 (arkworks)

R1CS=R1CS Constraints, Setup=Global SRS Setup, Wgen=Witness Generation, PT=Prover Time, VT=Verification Time, PS=Proof Size

Elliptic Curve	Scale (k)	R1CS	Setup	Wgen	PT	VT	PS
BN254 (Arkworks)	$k = 9$	512	175.4 ms	0.3 ms	108.0 ms	4.2 ms	128 B
	$k = 13$	8 192	497.0 ms	4.4 ms	561.6 ms	3.5 ms	128 B
	$k = 17$	131 072	6.953 s	71.8 ms	7.060 s	3.5 ms	128 B
	$k = 20$	1 048 576	57.276 s	582.0 ms	53.460 s	5.4 ms	128 B
	$k = 22$	4 194 304	210.803 s	3.149 s	307.090 s	10.8 ms	128 B
BLS12-381 (Arkworks)	$k = 9$	512	109.1 ms	0.3 ms	60.8 ms	3.7 ms	128 B
	$k = 13$	8 192	597.5 ms	4.2 ms	450.5 ms	3.4 ms	128 B
	$k = 17$	131 072	6.869 s	68.0 ms	6.316 s	3.8 ms	128 B
	$k = 20$	1 048 576	51.112 s	587.6 ms	46.939 s	3.6 ms	128 B
	$k = 22$	4 194 304	191.306 s	2.280 s	189.060 s	3.8 ms	128 B

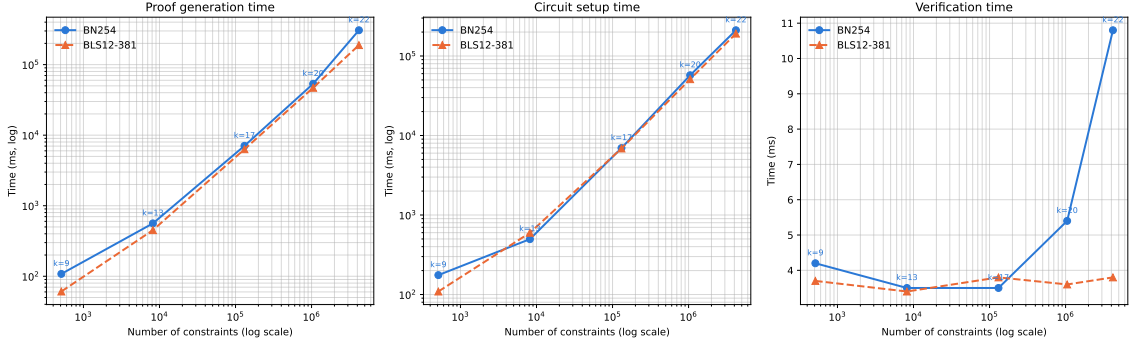


Figure 4.1: Groth16's benchmark times comparison BN254 Vs BLS12-381

Table 4.4: **Marlin** Benchmarks on BN254 and BLS12-381 (arkworks)

R1CS=R1CS Constraints, Setup=Global SRS Setup, Wgen=Witness Generation, PT=Prover Time, VT=Verification Time, PS=Proof Size

Elliptic Curve	Scale (k)	R1CS	Setup	Wgen	PT	VT	PS
BN254 (Arkworks)	$k = 9$	≈ 511	300.1 ms	0.5 ms	829.7 ms	16.6 ms	743 B
	$k = 13$	$\approx 8\,191$	534.9 ms	1.8 ms	1.939 s	5.7 ms	743 B
	$k = 17$	$\approx 131\,071$	7.537 s	19.9 ms	26.678 s	5.6 ms	743 B
	$k = 20$	$\approx 1\,048\,575$	81.117 s	211.5 ms	253.937 s	11.2 ms	743 B
	$k = 22$	$\approx 4\,194\,303$	206.649 s	617.9 ms	792.010 s	9.6 ms	743 B
BLS12-381 (Arkworks)	$k = 9$	≈ 511	85.3 ms	0.5 ms	528.3 ms	10.5 ms	951 B
	$k = 13$	$\approx 8\,191$	925.8 ms	1.8 ms	3.199 s	17.9 ms	951 B
	$k = 17$	$\approx 131\,071$	12.185 s	19.3 ms	42.855 s	9.3 ms	951 B
	$k = 21$	$\approx 2\,097\,151$	167.094 s	308.5 ms	602.495 s	10.3 ms	951 B
	$k = 22$	$\approx 4\,194\,303$	320.271 s	626.6 ms	1216.963 s	12.2 ms	951 B

**Note: Memory failure on Marlin at $k = 20$, resulting in a frozen black screen on BLS12-381.*

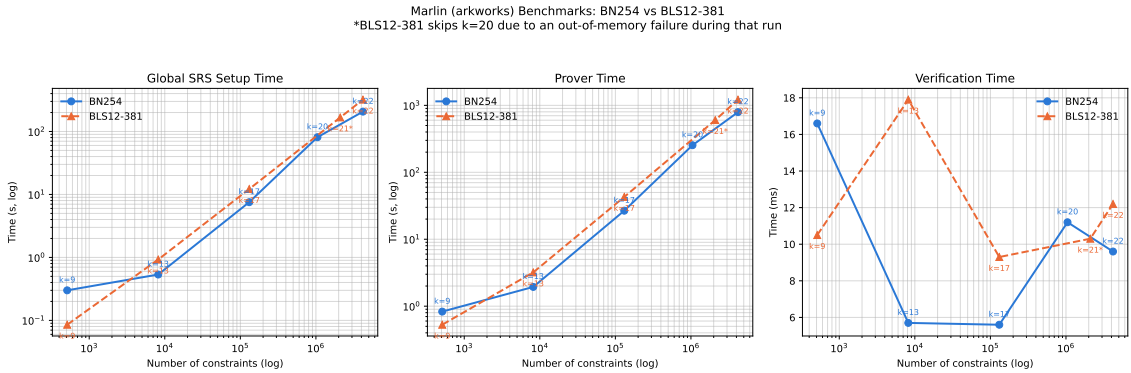


Figure 4.2: Marlin (arkworks) benchmark results comparing BN254 and BLS12-381

Table 4.5: **Halo2** (PSE) Benchmarks on BN254 and BLS12-381

PLONK-Cts=PLONKish Constraints, Setup=Global SRS Setup (KZG), Wgen=Witness Generation, VKey=verifier key generation, PKey=Prover Key generation, PT=Prover Time, VT=Verification Time, PS=Proof Size

Elliptic Curve	Scale (k)	PLONK-Cts	Setup	Wgen	PT	VT	PS
BN254 (Halo2 PSE)	$k = 9$	506	324.5 ms	0.2 ms	213.7 ms	8.4 ms	352 B
	$k = 13$	8 186	1.267 s	2.1 ms	250.0 ms	2.7 ms	352 B
	$k = 17$	131 066	20.886 s	36.7 ms	3.205 s	2.6 ms	352 B
	$k = 20$	1 048 570	205.915 s	373.4 ms	35.078 s	11.8 ms	352 B
	$k = 22$	4 194 298	670.071 s	1.266 s	93.037 s	6.1 ms	352 B
BLS12-381 (Halo2 PSE)	$k = 9$	506	241.8 ms	0.2 ms	124.3 ms	5.0 ms	464 B
	$k = 13$	8 186	3.482 s	2.3 ms	428.2 ms	4.8 ms	464 B
	$k = 17$	131 066	56.411 s	52.8 ms	5.993 s	5.3 ms	464 B
	$k = 20$	1 048 570	456.303 s	325.0 ms	37.572 s	5.7 ms	464 B
	$k = 22$	4 194 298	1797.420 s	1.224 s	138.268 s	5.4 ms	464 B

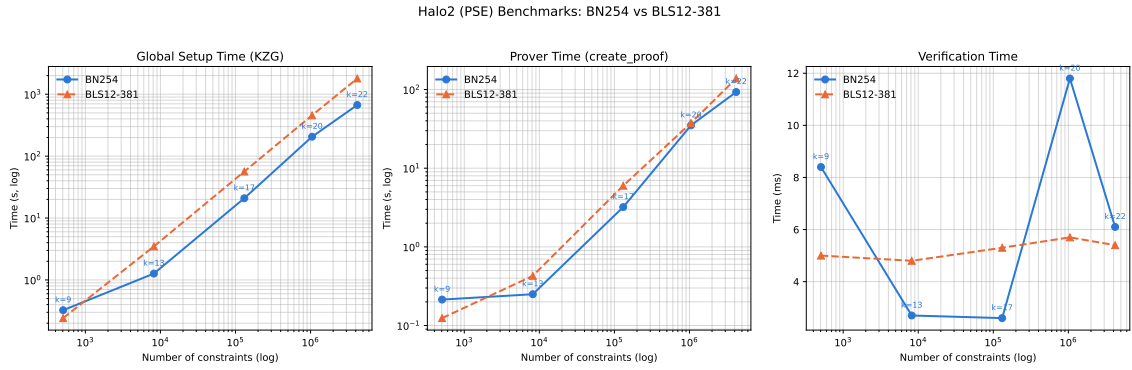


Figure 4.3: Halo2 (PSE) benchmark results comparing BN254 and BLS12-381

Table 4.6: Prover Time and Proof Size Across Protocols and Curves

Curve	Protocol	k=9 (512 cst)	k=13 (8k cst)	k=17 (131k cst)	k=20 (1.04M cst)	k=22 (4.19M cst)
BN254	Groth16	0.108 s	0.561 s	7.060 s	53.460 s	307.090 s
		128 B	128 B	128 B	128 B	128 B
	Marlin	0.829 s	1.939 s	26.678 s	253.937 s	792.010 s
		743 B	743 B	743 B	743 B	743 B
	Halo2	0.213 s	0.250 s	3.205 s	35.078 s	93.037 s
		352 B	352 B	352 B	352 B	352 B
BLS12-381 (Best Curve)	Groth16	0.060 s	0.450 s	6.316 s	46.939 s	189.060 s
		128 B	128 B	128 B	128 B	128 B
	Marlin	0.528 s	3.199 s	42.855 s	602.495 s*	1216.963 s
		951 B	951 B	951 B	951 B	951 B
	Halo2	0.124 s	0.428 s	5.993 s	37.572 s	138.268 s
		464 B	464 B	464 B	464 B	464 B

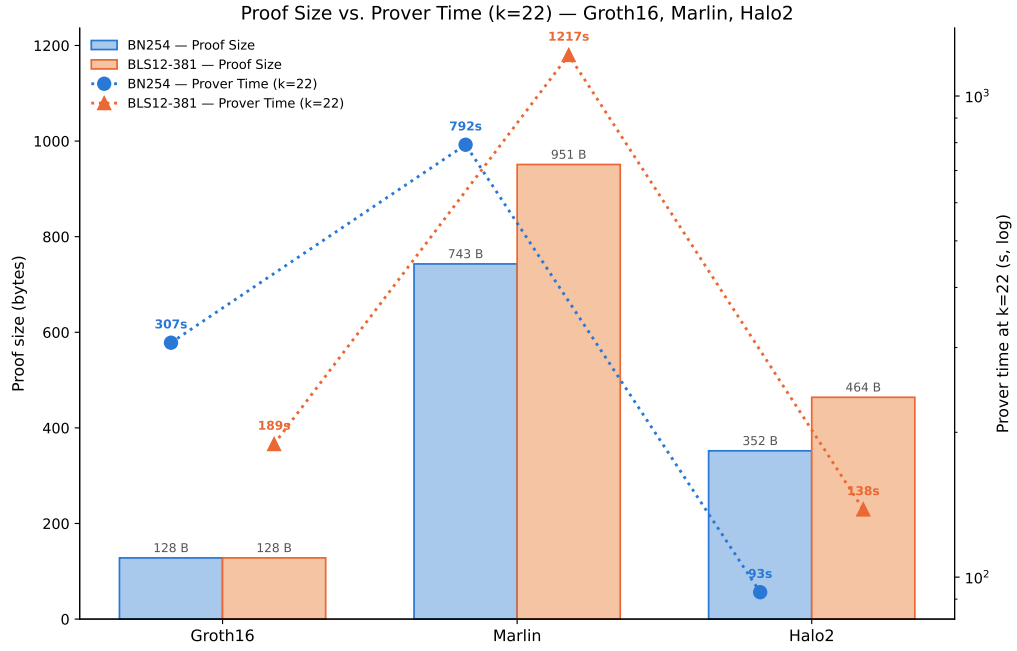


Figure 4.4: Trade-off between proof size and prover time (at $k=22$) across Groth16, Marlin, and Halo2

Observations: Tables 4.3, 4.4, 4.5 report the proof time, verification time, and proof size of Groth16, Marlin, and Halo2 on BN254 and BLS12-381. For the five circuit scales ($k=9, \dots, 22$), the proof time is 307,090s (Groth16/BN254), 792,010s (Marlin/BN254), 93,037s (Halo2/BN254), 189,060s (Groth16/BLS12-381), 1,216,963s (Marlin/BLS12-381), and 138,268s (Halo2/BLS12-381) (Table 4.6, Figure 4.4). In our setting, the verification time remains between 2.6ms and 17.9ms across all protocol/curve/scale combinations. The verified proof size is constant for each protocol, modifying the circuit scale: 128 B (Groth16), 352-464 B (Halo2), 743-951 B (Marlin). A memory defect was recorded on Marlin/BLS12-381 at $k=20$, (replaced by the measurement at $k=21$ in Table 4.4).

Interpretation: the stability of verification time and proof size, regardless of circuit scale, reflects the expected conciseness of the three tested protocols. The ranking of the three protocols by proof time at $k=22$ (Halo2 < Groth16 < Marlin, on both curves) suggests that, in this context, Halo2 would be best suited for deployments requiring rapid proof generation at large scale, while Groth16 would remain preferable when the size of the proof chain is the dominant criterion (128B, compared to 352-951B for the other two protocols). Marlin’s faulty memory on BLS12-381 at $k=20$ is an isolated observation that does not, on its own, allow us to conclude that Marlin has a systematic limit on this curve; however, it does indicate a practical deployment constraint to be noted for large circuits.

4.2 Discussion

This dissertation stems from the premise that cryptographic mechanisms based on elliptic curves (signatures, zk-SNARKs, Groth16) are intended to reconcile confidentiality, transparency and scalability in Blockchain applications. However, existing literature does not clearly demonstrate how elliptic curves, the design of zk-SNARK protocols, their security guarantees, and their implementation are interconnected. This dissertation addresses this gap by focusing on four specific objectives: to analyze the confidentiality and integrity of zk-SNARKs and their impact on performance; to study their deployment in a Layer 2 architecture to improve scalability; to measure and compare the performance of Groth16 on BN254 and BLS12-381; compare the Groth16, Marlin, and Halo2 protocols on BN254 and BLS12-381 according to industry requirements.

1. **Confidentiality, integrity and impact on performance:** according to the results, no effect of the curve choice on the confidentiality and integrity of Groth16 was observed. All generated proofs were successfully verified Table 4.6, regardless of the curve. However, this choice does influence performance. BLS12-381/BN254 performance is $1.45\times$ to $1.76\times$ depending on the pipeline phase (setup, proof, verification), BLS12-381 is slower in this setting. This is explained by the fact that most of the measured arithmetic operations are slower on BLS12-381 (Table 4.1). However, an important exception was observed in our benchmarks: full pairing, more commonly used for on-chain verification, exhibits almost the same execution time on both curves (1707.0 μ s BN254; 1705.0 μ s BLS12-381). This suggests the difference stems from operations upstream of the pairing (setup, scalar multiplications) rather than from the pairing itself, though this remains an interpretation rather than an isolated causal test (Section 4.1.3)

Thus, our results are consistent with the formal confidentiality and integrity guarantees of Groth16 established in Chapter 3, regardless of the curve. But the performance impact is measurable, not uniform across the operation, and must be explicitly budgeted rather than treated as a minor implementation detail.

2. **Layer 2 deployment and scalability:** the main observed result is total stability: the verification time and proof size remain constant for a given protocol and curve, regardless of the circuit size. This stability was verified on five scales ranging from 512 to over 4 million constraints (Tables 4.2 to 4.4). This corresponds to the succinctness property of zk-SNARKs, which makes the Layer 2 model possible: it allows proof verification without the cost depending on the complexity of the off-chain calculations. No exception was observed across these scales.

The proof time increases non-linearly with the circuit size, regardless of the protocol and curve used. This contrast between proof generation time and verification time is what enables scalability in Layer 2 architectures: more expensive calculations

are performed off-chain, and the blockchain only verifies a proof of fixed size. The results confirm that all three protocols can be deployed at Layer 2, but no protocol is superior across all criteria. Groth16 excels in proof size, potentially reducing storage and transmission costs on the Blockchain. Halo2 offers the fastest generation time for large circuits (93.037s for $k=22$ on BN254). However, its proofs are large (352 to 464 bytes). The choice of protocol then depends on the application's needs: prioritizing smaller proofs or faster generation.

3. **Groth16 arithmetic and curve selection:** the result concerning the pairing operation is one of the most interesting of this study. It is widely used in proof verification on the Blockchain. Contrary to the idea that BLS12-381 is always slower than BN254, our measurements showed that the time to execute a pairing is practically the same on both curves. Even though we used only one library (Arkworks), this is confirmed by the work of Ernstberger et al. (Ernstberger et al., 2024), who compared 13 elliptic curves using 9 different libraries.

Their study showed that performance does not depend solely on the elliptic curve. For example, some operations performed on BN254 are not directly faster than on BLS12-381, even though it has a small, finite field. The authors also show that a general-purpose library (gnark-crypto) can prove to be faster than a library specifically designed for BLS12-381 (blstrs) for certain critical operations.

Combining these results, our data and this confrontation with the literature suggest that the observed performance depends heavily on the implementation used and not just on the mathematical properties of elliptic curves. On this basis, the choice of a curve should not be based solely on theoretical criteria, but also on tests performed with the software library and hardware that will actually be used during deployment.

4. **Large-scale multi-protocol global comparative analysis:** the results obtained for large circuits ($k=22$) (Tables 4.2 to 4.5) indicate that no single protocol is the best on all criteria. Halo2 is the fastest for generating proofs, Groth16 comes in second, and Marlin is the slowest. Furthermore, Groth16 produces the smallest proofs, followed by Halo2 and then Marlin. The choice of a protocol therefore depends on the application's priorities, since none of these protocols combines both the best speed and the smallest proof size.

The practical challenge highlighted in this dissertation is memory consumption. Marlin could not be run on BLS12-381 for the largest circuits due to insufficient memory. This is consistent with the work of (Ernstberger et al., 2024), who encountered a similar problem with the PLONK implementation in the gnark library. In their study, some experiments could not be performed on a computer with 128 GB of RAM. This suggests that performance may be linked to whether the machine is optimized for computing power or memory: is therefore essential for the large-scale

deployment of zk-SNARK, and the difficulties do not appear to be specific to Marlin. Furthermore, Kuznetsov et al. (Kuznetsov et al., 2026) show that, for very complex circuits, most of the execution time is spent on witness generation rather than proof generation itself. That said, to improve the performance of a protocol, it is necessary not only to improve the proof and MSM algorithms but also to optimize witness generation. This represents an interesting avenue of research for future work.

4.2.1 Confrontation with Literature

Our results are consistent with those of (Liang et al., 2025), who observed that the choice and use of curves are neglected in most libraries and studies on zk-SNARK. This observation is also supported by the work of (Jang et al., n.d.), who provide a complete formal construction and comparison of four protocols (Groth16, PLONK, Halo2, Bulletproofs) without mentioning the elliptic curve used in their experiments, nor comparing BN254 to BLS12-381.

Our results also converge with Jang et al. (Jang et al., n.d.) regarding the performance ranking of the protocols. Their study concludes that Halo2 offers the best performance when no trusted third party is used, while Groth16 remains effective if the initial configuration is accepted. Our observations in this dissertation support this conclusion. Halo2 generates proofs faster than Groth16 on BN254 (93.037s versus 307.090s). However, the values are not directly comparable because the two studies use different benchmark circuits.

A partial disagreement arises regarding the verification time of Halo2: (Jang et al., n.d.) report linear growth with circuit size. In this study, our data (Table 4.5, Figure 4.3) show, on the contrary, that this time remains almost constant, between 2.6 ms and 11.8 ms, regardless of circuit size, without any growth trend. This difference may be explained by implementation choices, the circuits used, or the experimental conditions. Further work will be necessary to precisely identify the cause.

Prior to this study, the literature separately established that curve choice could pose a practical problem (Liang et al., (Jang et al., n.d.), without arithmetic quantization), and that Groth16, PLONK, and Halo2 could be compared in performance within a given application context (Jang et al., without cross-analysis of the curves). This dissertation connects these two aspects by establishing a quantified link between the performance of basic arithmetic operations and that of protocols on the BN254 and BLS12-381 curves. The various results remain specific to their implementation.

4.2.2 Limitations of the Study

- The results are based on a single implementation per protocol (arkworks, a PSE fork of halo2-proofs) and a single hardware configuration: see Chapter 3.
- We do not experimentally demonstrate that confidential information effectively remains hidden, just formally; only soundness was observed through the proof veri-

fication status.

- Marlin’s memory failure limits direct comparison to $k=20$ on BLS12-381; no solutions are proposed within the scope of this study.
- The comparisons remain qualitative; they are not directly linked to the circuits, libraries, and hardware platforms used.

4.2.3 Implications

The results indicate that no protocol or curve dominates across all criteria tested in this dissertation. The optimal choice depends on the priority: of the intended deployment: minimum proof size (Groth16, BN254, or BLS12-381, regardless of size), large-scale proof generation speed (Halo2), or compliance with a 128-bit security level (BLS12-381, regardless of protocol). Now, minimum proof size + compliance with a 128-bit security level curve BLS12-381 + Halo2 is ideal, see Figure 4.4

CHAPTER 5

CONCLUSION AND RECOMMENDATIONS

We conclude our dissertation by this chapter, by given in the first time a general conclusion follows by conclusion specifics to our objectives then, we close by some recommendations.

5.1 Conclusion

The main message of this dissertation is: there is no single solution for choosing between a zk-SNARK protocols and an elliptic curves. The following parameters must be considered when making a choice: **proof size**, which has a direct on the scalability of the of Layer 2, by reducing the memory used in blocs to store the proof and, it is relative to curve and protocol; **verification time**, reducing this time give an important gain in term of speed in Layer 2 system. It depends on the curve and protocol chosen. Halo2-BN254 remind the high-performance crossover. The **ability to handle large circuits**, is also a parameter. Some protocols do not handle circuit which is growing with the size. Another parameters are **memory consumption** the protocol like Halo2 uses a load of memory when running; and **curve security level**, since the security level required is not reached by the both curves. The BLS12-381 is the highest secured curve among the both with 128 bits level of security. This work addresses a limitation identified in the literature. (Liang et al., 2025) showed that the choice of elliptic curves was sometimes neglected in zk-SNARK libraries, without measuring its impact. (Jang et al., n.d.), for their part, compared several zk-SNARK protocols without specifying or comparing the curves used. In this work, we complement these studies by measuring the influence of curve choice, from basic arithmetic operations to the performance of the complete protocol. We obtained results consistent with recent studies. (Ernstberger et al., 2024), based on numerous curves and libraries, found that performance is linked to the implementation used, and that no single curve is directly the fastest. (Kuznetsov et al., 2026), for their part, working in a different application domain and with different tools, confirm that Groth16 has a constant proof size and verification time regardless of circuit size and complexity, a result consistent with our own observations in Chapter 4. (Liang et al., 2025) highlight the problem of curve selection, (Jang et al., n.d.) compare protocols, (Ernstberger et al., 2024) analyze the influence of curves on performance, and Kuznetsov et al. confirm Groth's succinct properties: all these works complement each other. Our work connects these different aspects by studying arithmetic operations, zk-SNARK protocols, BN254 and BLS12-381 curves, and their use in a zk-Rollup or Layer 2 architecture. Finally, our results are not limited to Layer 2; they can be useful for applications using zk-SNARK, such as decentralized identity systems (DID), sensor networks, electronic voting, and financial

applications. For each domain, the choice of curve and protocol must depend on the application's needs and, above all, be based on an experimental performance evaluation. Because we must not base ourselves solely on theoretical considerations.

5.2 Summary of Findings

Specifically to different objectives of this dissertation, the main result of dissertation thesis is that the confidentiality and integrity of Groth16 remain the same for any elliptic curve used. However from what is observed, performance varies with the chosen curve. Furthermore, this choice impacts operations differently. Our observations suggest that: Groth16's security does not conflict with performance. Choosing a high security level can increase execution time. This cost must be taken into account when choosing the curve when deploying the system.

This dissertation provides experimental evidence, within the configurations tested, that Groth16's succinctness property holds across the different circuit scales examined. This result is also confirmed by an independent study by (Kuznetsov et al., 2026), conducted in a different application domain using different tools. The fact that this property is observed in two distinct contexts strengthens the confidence in the reliability of our result concerning it.

The main lesson learned from this work is that there is no general rule that allows us to assert that BN254 is always more performant than BLS12-381, or vice versa. Performance depends heavily on the operation performed, the protocol used, and the chosen implementation. This was also the finding confirmed by (Ernstberger et al., 2024), who obtained similar results when comparing several cryptographic libraries.

However, one difference remains constant: the security level. BLS12-381 offers a security level of approximately 128 bits, while BN254 provides a level of approximately 100 to 110 bits. Therefore, one should not choose an elliptic curve solely based on its reputation for performance. It would be more appropriate to conduct tests on the library and hardware that will actually be used in order to select the most suitable solution.

5.3 Difficulties Encountered

The first challenge was mathematical foundations. Follows by the direction of the dissertation. The main practical challenge identified during this study is related to memory consumption, not execution time. The failure of Marlin on BLS12-381 for large circuits, as well as a similar problem observed by (Ernstberger et al., 2024) with PLONK and the gnark library, demonstrates that computation time alone is insufficient to assess the feasibility of a protocol under real-world conditions. Available memory is also a determining factor. Finally, the results show that neither the protocol nor the elliptic curve is generally superior to the other: each has its advantages and limitations depending on the criteria considered.

5.4 Recommendations

5.4.1 For the Direct Continuation of this Work

1. Extend the arithmetic analysis to other libraries (gnark, blstrs, halo2 with IPA), following the methodology of (Ernstberger et al., 2024), to verify if the cost proximity of the full pairing observed with arkworks generalizes.
2. Measuring the memory consumption of each protocol will help to precisely identify the causes of the problems encountered with Marlin and PLONK. Future work could, for example, utilize the maximum resident set size measurement method proposed by (Ernstberger et al., 2024).
3. Reproduce this study on representative embedded hardware (ARM Cortex-A/M), as proposed by (Kuznetsov et al., 2026) who carried out their study only on $x86 - 64$ desktop workstations.
4. The formal analysis of the zero-knowledge property established in Chapter 3 of this paper was limited to a validation of soundness. We recommend supplementing this with a definition and explicit empirical verification, similar to the formalization provided by (Jang et al., n.d.) for the four protocols they studied.
5. Reproduce the multi-protocol comparative analysis of this study in an application context distinct from zk-Rollup, for example the decentralized identity studied by (Jang et al., n.d.), to verify if the observed performance ranking (Halo2 < Groth16 < Marlin in proof time) will be maintained.

5.4.2 For the Community and Practice

1. Zk-SNARK library developers should provide, for each elliptic curve, the measured cost of each elementary operation in their implementation. This would allow for the selection of the curve best suited to the needs. This recommendation addresses the shortcomings identified independently by (Liang et al., 2025) and by this work, and confirmed on a large scale by (Ernstberger et al., 2024)
2. In the future, work using zk-SNARK in new fields such as decentralized identity, sensor networks, etc., should specify and justify their choice of elliptic curve. This recommendation is motivated by the results of this study and by the fact that some works do not specify the elliptic curve used, as is the case with (Jang et al., n.d.)

5.5 Self-analysis

We chose to study a single implementation, from arithmetic operations to the complete deployment of a zk-Rollup, rather than comparing an exhaustive number of libraries. This dissertation contributes to the literature an answer to a problem raised by Jang et al. (2025),

complements the work of (Jang et al., [n.d.](#)) by analyzing the impact of the choice of the elliptic curve, and obtains results consistent with those recently obtained by (Ernstberger et al., [2024](#)) and (Kuznetsov et al., [2026](#)). These works were carried out in different contexts and with different tools, but lead to similar conclusions. This strengthens the credibility of our results and demonstrates the value of continuing research along the same lines.

REFERENCES

- African Union. (2020). The digital transformation strategy for africa (2020–2030). <https://au.int/en/documents/20200518/digital-transformation-strategy-africa-2020-2030>
- Augot, D., Bordage, S., El Housni, Y., et al. (2021, September). Zero-knowledge: Trust and privacy on an industrial scale [Outlook #1]. <https://blockchain-polytechnique.com/zero-knowledge-trust-and-privacy-on-an-industrial-scale/>
- Bank, W. (2022). The global finindex database 2021: Financial inclusion, digital payments, and resilience in the age of covid-19. <https://www.worldbank.org/en/publication/globalfinindex>
- Blum, M., Feldman, P., & Micali, S. (1988). Proving security against chosen ciphertext attacks. *Conference on the Theory and Application of Cryptography*, 256–268.
- Blum, M., Feldman, P., & Micali, S. (2019). Non-interactive zero-knowledge and its applications. In *Providing sound foundations for cryptography: On the work of shafi goldwasser and silvio micali* (pp. 329–349).
- Boneh, D., Lynn, B., & Shacham, H. (2001). Short signatures from the weil pairing. *International conference on the theory and application of cryptology and information security*, 514–532.
- Bootle, J., Cerulli, A., Chaidos, P., Groth, J., & Petit, C. (2016). Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, 327–357.
- Bowe, S., Grigg, J., & Hopwood, D. (2019). Recursive proof composition without a trusted setup. *Cryptology ePrint Archive*.
- Bulgakov, A. L., Aleshina, A. V., Smirnov, S. D., Demidov, A. D., Milyutin, M. A., & Xin, Y. (2024). Scalability and security in blockchain networks: Evaluation of sharding algorithms and prospects for decentralized data storage. *Mathematics*, 12(23), 3860.
- Bünz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., & Maxwell, G. (2018). Bulletproofs: Short proofs for confidential transactions and more. *2018 IEEE symposium on security and privacy (SP)*, 315–334.
- Chainalysis. (n.d.). *Blockchain intelligence* [Consulté le 30 mai 2026]. <https://www.chainalysis.com/blockchain-intelligence/>
- Chehade, I. (2022). Blockchain and dlt in the banking system. *Revue d'économie financière*, 145(1), 253–275.
- Chen, B., Ma, L., Xu, H., Ma, J., Hu, D., Liu, X., Wu, J., Wang, J., & Li, K. (2024). A comprehensive survey of blockchain scalability: Shaping inner-chain and inter-chain perspectives. *arXiv preprint arXiv:2409.02968*.

- Chiesa, A., Hu, Y., Maller, M., Mishra, P., Vesely, N., & Ward, N. (2020). Marlin: Preprocessing zkSNARKs with universal and updatable SRS. *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, 738–768.
- Clarisse, R. (2021). *Conception de courbes elliptiques et applications* [Doctoral dissertation, Université de Rennes].
- Danezis, G., Fournet, C., Kohlweiss, M., & Parno, B. (2013). Pinocchio coin: Building zeroCoin from a succinct pairing-based proof system. *Proceedings of the First ACM workshop on Language support for privacy-enhancing technologies*, 27–30.
- Demirgüç-Kunt, A., Klapper, L., Singer, D., Ansar, S., & Hess, J. (2022). *The global findex database 2021: Financial inclusion, digital payments, and resilience in the age of covid-19*. World Bank. <https://www.worldbank.org/en/publication/globalfindex>
- Di Pierro, M. (2017). What is the blockchain? *Computing in Science & Engineering*, 19(5), 92–95.
- Dwork, C., & Naor, M. (1992). Pricing via processing or combatting junk mail. *Annual international cryptology conference*, 139–147.
- Ernstberger, J., Chaliasos, S., Kadianakis, G., Steinhorst, S., Jovanovic, P., Gervais, A., Livshits, B., & Orrù, M. (2024). Zk-bench: A toolset for comparative evaluation and performance benchmarking of SNARKs. *International Conference on Security and Cryptography for Networks*, 46–72.
- Feng, Q., He, D., Zeadally, S., Khan, M. K., & Kumar, N. (2019). A survey on privacy protection in blockchain system. *Journal of network and computer applications*, 126, 45–58.
- Fiat, A., & Shamir, A. (1986). How to prove yourself: Practical solutions to identification and signature problems. *Conference on the theory and application of cryptographic techniques*, 186–194.
- Gabizon, A., Williamson, Z. J., & Ciobotaru, O. (2019). Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive*.
- Garcia, R. D., Ramachandran, G., Dunnett, K., Jurdak, R., Ranieri, C., Krishnamachari, B., & Ueyama, J. (2024). A survey of blockchain-based privacy applications: An analysis of consent management and self-sovereign identity approaches. *arXiv preprint arXiv:2411.16404*.
- Gennaro, R., Gentry, C., Parno, B., & Raykova, M. (2013). Quadratic span programs and succinct NIZKs without PCPs. *Annual international conference on the theory and applications of cryptographic techniques*, 626–645.
- Gerards, C. (2022). Zk-SNARKs: A literature study [Literature Review on Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge].

- Gervais, A., Karame, G. O., Wüst, K., Glykantzis, V., Ritzdorf, H., & Capkun, S. (2016). On the security and performance of proof of work blockchains. *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, 3–16.
- Ghammam, L. (2016). *Utilisation des couplages en cryptographie asymétrique pour la micro-électronique* [Doctoral dissertation, Université de Rennes 1; Université de Monastir (Tunisie)].
- Ghammam, L., & Fouotsa, E. (2016). On the computation of the optimal ate pairing at the 192-bit security level. *Cryptology ePrint Archive*.
- Goldreich, O., Micali, S., & Wigderson, A. (1986). How to prove all np statements in zero-knowledge and a methodology of cryptographic protocol design. *Conference on the Theory and Application of Cryptographic Techniques*, 171–185.
- Goldwasser, S., Kalai, Y. T., & Rothblum, G. N. (2008). Delegating computation: Interactive proofs for muggles. *Proceedings of the fortieth annual ACM symposium on Theory of computing*, 113–122.
- Goldwasser, S., Micali, S., & Rackoff, C. (1989). The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*. <https://doi.org/10.1137/0218012>
- Groth, J. (2016). On the size of pairing-based non-interactive arguments. *Advances in Cryptology – EUROCRYPT 2016*, 9666, 305–326. https://doi.org/10.1007/978-3-662-49896-5_11
- Groth, J., Kohlweiss, M., Maller, M., Meiklejohn, S., & Miers, I. (2018). Updatable and universal common reference strings with applications to zk-snarks. *Annual International Cryptology Conference*, 698–728.
- GSMA Intelligence. (2023). The mobile economy sub-saharan africa 2023. <https://www.gsmainelligence.com/research/the-mobile-economy-sub-saharan-africa-2023>
- Holvast, J. (2007). History of privacy. In *The history of information security* (pp. 737–769). Elsevier.
- Horn, R. A., & Johnson, C. R. (1991). The hadamard product. In *Topics in matrix analysis* (pp. 298–381). Cambridge University Press.
- Jang, J., Jeong, S., Seo, C., & Kim, H. (n.d.). Privacy-preserving credentials in decentralized identity: Integrating zero-knowledge proofs from theory to practice. *Available at SSRN 5402209*.
- Johnson, D., Menezes, A., & Vanstone, S. (2001). The elliptic curve digital signature algorithm (ecdsa). *International journal of information security*, 1(1), 36–63.
- Klapper, L., Singer, D., Starita, L., & Norris, A. (2021). The global findex database 2025.
- Koblitz, N. (1987). Elliptic curve cryptosystems. *Mathematics of computation*, 48(177), 203–209.
- Kuznetsov, O., Kuznetsova, Y., Ziyatbekova, G., Kovalenko, Y., & Palahusynets, R. (2026). Performance evaluation of zk-snark protocols for privacy-preserving sensor data verification: A systematic benchmarking study. *Sensors*, 26(8), 2486.

- Liang, J., Hu, D., Wu, P., Yang, Y., Shen, Q., & Wu, Z. (2025). {Sok}: Understanding {zk-snarks}: The gap between research and practice. *34th USENIX Security Symposium (USENIX Security 25)*, 2085–2104.
- Luo, X., Chen, X., Chen, X., & Cheng, Q. (2024). A survey on the application of blockchain in cryptographic protocols: X. Luo et al. *Cybersecurity*, 7(1), 79.
- Masseport, S. (2021). *Consensus blockchain: Incitation des utilisateurs d'un réseau à la participation et à la loyauté* [Doctoral dissertation, Université Montpellier].
- Meiklejohn, S., Pomarole, M., Jordan, G., Levchenko, K., McCoy, D., Voelker, G. M., & Savage, S. (2013). A fistful of bitcoins: Characterizing payments among men with no names. *Proceedings of the 2013 conference on Internet measurement conference*, 127–140.
- Micali, S. (2000). Computationally sound proofs. *SIAM Journal on Computing*, 30(4), 1253–1298.
- Mssassi, S., & Abou El Kalam, A. (2024). The blockchain trilemma: A formal proof of the inherent trade-offs among decentralization, security, and scalability. *Applied Sciences*, 15(1), 19.
- Nakai, T., Sakurai, A., Hironaka, S., & Shudo, K. (2023). The blockchain trilemma described by a formula. *2023 IEEE International Conference on Blockchain (Blockchain)*, 41–46. <https://doi.org/10.1109/Blockchain60715.2023.00016>
- Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system.
- Parno, B., Howell, J., Gentry, C., & Raykova, M. (2016). Pinocchio: Nearly practical verifiable computation. *Communications of the ACM*, 59(2), 103–112.
- Pfitzmann, A., & Hansen, M. (2010). A terminology for talking about privacy by data minimization: Anonymity, unlinkability, undetectability, unobservability, pseudonymity, and identity management.
- Portuondo, A. C., Robinson, D. G., Guerra, A. A., & Cuba, C. C. (2024). Aplicación de la criptografía de conocimiento cero en la tecnología blockchain: Garantizando la privacidad y la integridad de los datos. *IX Jornadas Nacionales de Investigación En Ciberseguridad*, 638–643.
- Rebello, G. A. F., Camilo, G. F., de Souza, L. A. C., Potop-Butucaru, M., de Amorim, M. D., Campista, M. E. M., & Costa, L. H. M. (2024). A survey on blockchain scalability: From hardware to layer-two protocols. *IEEE Communications Surveys & Tutorials*, 26(4), 2411–2458.
- Sheybani, N., Ahmed, A., Kinsy, M., & Koushanfar, F. (2025). Zero-knowledge proof frameworks: A systematic survey. *arXiv preprint arXiv:2502.07063*.
- Shpilka, A., & Yehudayoff, A. (2010). Arithmetic circuits: A survey of recent results and open questions. *Foundations and Trends® in Theoretical Computer Science*, 5(3-4), 207–388.

- Siddiqui, S., et al. (2026). Zk-rollup for hyperledger fabric: Architecture and performance evaluation. *arXiv preprint arXiv:2602.08870*.
- Silverman, J. H. (2009). *The arithmetic of elliptic curves* (2nd ed., Vol. 106). Springer. <https://doi.org/10.1007/978-0-387-09494-6>
- Thibault, L. T., Sarry, T., & Hafid, A. S. (2022a). Blockchain scaling using rollups: A comprehensive survey. *IEEE Access*, 10, 93039–93054. <https://doi.org/10.1109/ACCESS.2022.3200051>
- Thibault, L. T., Sarry, T., & Hafid, A. S. (2022b). Blockchain scaling using rollups: A comprehensive survey. *IEEE Access*, 10, 93039–93054.
- Unnikrishnan, K., & Victor Paul, P. (2025). Zero-knowledge proof (zkp) techniques within blockchain technology. *Journal of Internet Services and Information Security*, 15(2), 926–941.
- VoD, Y. (2021). Cours complet d'introduction sur la blockchain. <https://youtu.be/uFDoxWGWOBu>
- Wang, X., Zheng, P., & Xing, Q. (2023). Security analysis of pairing-based cryptography. *arXiv preprint arXiv:2309.04693*.
- Wijaya, M., Melvyn, F., Setiawan, R., & Rumagit, R. Y. (2025). Ethereum vs solana: A comparative study of blockchain architecture on performance, security, and ecosystem development [The 10th International Conference on Computer Science and Computational Intelligence 2025]. *Procedia Computer Science*, 269, 200–217. <https://doi.org/https://doi.org/10.1016/j.procs.2025.08.273>
- Wikipedia contributors. (2026). Blockchain privacy [Accessed: 2026-05-30]. https://en.wikipedia.org/wiki/Blockchain_privacy
- Wright, A., & De Filippi, P. (2015). Decentralized blockchain technology and the rise of lex cryptographia. *Available at SSRN 2580664*.
- Xie, T., Zhang, J., Zhang, Y., Papamanthou, C., & Song, D. (2019). Libra: Succinct zero-knowledge proofs with optimal prover computation. *Annual International Cryptology Conference*, 733–764.
- Xue, Y., Wang, J., Shi, K., & Zhang, H. (2024). A privacy-preserving model for blockchain-based data sharing in the industrial internet. *Transactions on Emerging Telecommunications Technologies*, 35(4), e4749.
- Zheng, Z., Xie, S., Dai, H.-N., Chen, X., & Wang, H. (2018). Blockchain challenges and opportunities: A survey. *International journal of web and grid services*, 14(4), 352–375.
- Zhu, P. (2025). Plonkify: R1cs-to-plonk transpiler. *Cryptology ePrint Archive*.

APPENDIX A

ADDITIONAL MATERIALS

A.1 Algebraic Geometry Concepts

Proof. Let $E : y^2 = x^3 + ax + b = f(x)$ be an elliptic curve over $\mathbb{F}_p$. If $f(x) = 0$, there is only one solution $y = 0$. If $f(x) \neq 0$, for half $\frac{p-1}{2}$ the possible nonzero values of $f(x)$ or y , there are two solutions for y . For the other half, there is no solutions. So for the squares and the non-squares, we will have $1(\text{for } y = 0) + (p - 1)$ roots plus the point at infinity, thus $p + 1$ roots. Then the number of solutions should be like: $E(\mathbb{F}_p) = p + 1 - \epsilon$, where ϵ is the trace of Frobenius. Otherwise, the characteristic polynomial of Frobenius is given by:

$$X^2 - \epsilon X + p. \quad (\text{A.1})$$

This equation has two roots, that we denote α and β such that:

$$\alpha + \beta = \epsilon \quad (*) \quad (\text{A.2})$$

$$\alpha\beta = p \quad (**) \quad (\text{A.3})$$

$(**)$ implies that $\alpha^2 = \beta^2 = p$

Meaning that

$$|\alpha| = |\beta| = \sqrt{p}. \quad (\text{A.4})$$

Thus from $(*)$, we obtain that:

$$|\epsilon| = |\alpha + \beta| \leq 2\sqrt{p}. \quad (\text{A.5})$$

By replacing the ϵ in the expression of the cardinality, we obtain:

$$|E(\mathbb{F}_p) - (p + 1)| \leq 2\sqrt{p}. \quad (\text{A.6})$$

.

□

Fundamental Notions: affine variety, affine space, example of affine curve

Let us now define the algebraic and geometric living environment of elliptic curves: affine and projective space.

Definition A.1 (Affine Space). (*Silverman, 2009*)

An affine n -space (over K) is the set of n -tuples

$$\mathbb{A}^n(\bar{K}) = \{(a_1, \dots, a_n) \mid a_i \in \bar{K}, i = 1, \dots, n\}. \quad (\text{A.7})$$

Given a set $S \subset K[x_1, \dots, x_n]$ **any** ideal, we can associate an **affine algebraic set** defined by S :

$$V(S) = \{P \in \mathbb{A}^n(\bar{K}) \mid f(P) = 0, \forall f \in S\}. \quad (\text{A.8})$$

where $S = \{f \in \bar{K}[x_1, \dots, x_n] \mid f(P) = 0, \forall P \in \mathbb{A}^n(\bar{K})\}$.

Definition A.2 (Affine Variety). (Silverman, 2009)

An **affine algebraic** set V is called a (affine) variety if S is a prime ideal in $\bar{K}[x_1, \dots, x_n]$.

Example 4. (Affine Curve): The curve $C : y^2 = x^3 + ax + b$ in $\mathbb{A}^2$ is defined by the polynomial

$$f(x, y) = y^2 - x^3 - ax - b. \quad (\text{A.9})$$

Before ending this part, we should note that, there is a certain lack over these spaces for definition of an elliptic curve. To see that, let us consider the curve $E : y^2 = x^3 - x$ in the field K , with $\text{char } K \neq 2, 3$. Let $P = (0, 0)$, $Q = (1, 0)$, $P, Q \in E$. The line linking P and Q has intersections with the curve at $(0, 0)$, $(1, 0)$, and $(-1, 0)$. The derivative form of E is $2y y' = 3x^2 - 1$. At P : $y = 0 \Rightarrow 0 \cdot y' = -1 \Rightarrow y' = -\frac{1}{0} \rightarrow \infty$. This means the tangent at P is vertical, i.e., $x = 0$. By replacing in $E : y^2 = 0$, meaning we have just 2 points. There is a lack of one point for a cubic algebra E .

Let us give the more complete space that contains this point at infinity.

Projective space, homogeneous polynomials, homogenization, dehomogenization

Definition A.3 (Homogeneous Polynomial). (Silverman, 2009)

A polynomial $f \in K[x_0, \dots, x_n]$ is homogeneous of degree d if:

$$f(\lambda x_0, \dots, \lambda x_n) = \lambda^d f(x_0, \dots, x_n). \quad (\text{A.10})$$

The projective space defines, for a set of homogeneous polynomials S :

$$V(S) = \{[a_0 : \dots : a_n] \in \mathbb{P}^n(K) \mid f(a_0, \dots, a_n) = 0, \forall f \in S\}. \quad (\text{A.11})$$

Definition A.4 (Homogenizing a Polynomial). (Silverman, 2009)

Let $f(x_1, \dots, x_n) \in K[x_1, \dots, x_n]$ of degree d . The homogenized form of f with respect to x_0 is:

$$F(x_0, \dots, x_n) = x_0^d f\left(\frac{x_1}{x_0}, \dots, \frac{x_n}{x_0}\right). \quad (\text{A.12})$$

Inversely, if $F(x_0, \dots, x_n)$ is a homogenized polynomial, its dehomogenized form is:

$$f(\alpha_1, \dots, \alpha_n) = F(1, \alpha_1, \dots, \alpha_n). \quad (\text{A.13})$$

Example 5. Let us consider the curve $y^2 = x^3 + ax + b$. By setting

$$x = \frac{X}{Z}, \quad y = \frac{Y}{Z}. \quad (\text{A.14})$$

we have:

$$\left(\frac{Y}{Z}\right)^2 = \left(\frac{X}{Z}\right)^3 + a\left(\frac{X}{Z}\right) + b. \quad (\text{A.15})$$

Then, by multipliant (2) by Z^3 , the homogenized form will be:

$$Y^2 Z = X^3 + aXZ^2 + bZ^3. \quad (\text{A.16})$$

The point at infinity: $Z = 0 \Rightarrow X^3 = 0$, i.e., $X = 0$, $Y \in K$. Thus $[0 : Y : 0] = [0 : 1 : 0]$. In the plan where $Z = 0$, this point can be seen as in the Figure 3.3.

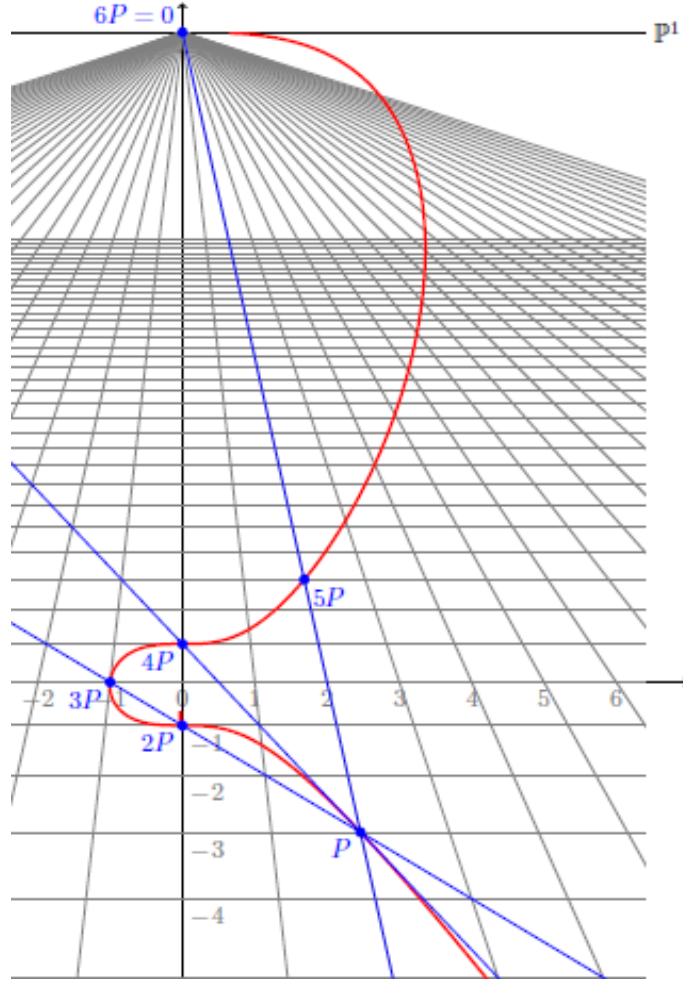


Figure A.1: This is a representation of an elliptic curve with a twist point of order 6, because $6P = O$: the point at infinity.

Definition A.5.

(E, O) is an elliptic curve where E is a nonsingular curve of genus one and $O \in E$.
(A.17)

From **Alain**, it can also be defined as a plane projective curve given by the equation:

$$E : y^2z = x^3 + axz^2 + bz^3, \quad \text{where } a, b \in K, \text{ satisfying the condition (nonsingularity):} \quad (\text{A.18})$$

$$4a^3 + 27b^2 \neq 0. \quad (\text{A.19})$$

Indeed, let

$$F(x, y, z) = y^2z - x^3 - axz^2 - bz^3. \quad (\text{A.20})$$

A singular point satisfies

$$F = \frac{\partial F}{\partial x} = \frac{\partial F}{\partial y} = \frac{\partial F}{\partial z} = 0. \quad (\text{A.21})$$

We compute

$$\frac{\partial F}{\partial x} = -3x^2 - az^2, \quad (\text{A.22})$$

,

$$\frac{\partial F}{\partial y} = 2yz, \quad (\text{A.23})$$

,

$$\frac{\partial F}{\partial z} = y^2 - 2axz - 3bz^2. \quad (\text{A.24})$$

From $\partial F/\partial y = 0$ and $\partial F/\partial x = 0$, if $z = 0$ we get $x = 0$ and $y \in \bar{k}$: this gives the point at infinity, which is nonsingular.

With $y = 0$, the equation $F = 0$ and the partial derivative $\partial F/\partial x = 0$ give

$$x^3 + axz^2 + bz^3 = 0. \quad (\text{A.25})$$

$$3x^2 + az^2 = 0. \quad (\text{A.26})$$

Since we are looking for a nontrivial projective solution, we may take $z \neq 0$ and set $t = \frac{x}{z}$.

Dividing by z^3 and z^2 (7) and (8) respectively, we obtain

$$t^3 + at + b = 0. \quad (\text{A.27})$$

$$3t^2 + a = 0. \quad (\text{A.28})$$

$$a = -3t^2. \quad (\text{A.29})$$

Substituting into (9)

$$t^3 - 3t^3 + b = 0 \implies -2t^3 + b = 0 \implies b = 2t^3. \quad (\text{A.30})$$

Hence

$$4a^3 + 27b^2 = 4(-3t^2)^3 + 27(2t^3)^2 = -108t^6 + 108t^6 = 0. \quad (\text{A.31})$$

Conversely, if $4a^3 + 27b^2 = 0$, one can recover such a t , so a singular point exists. Ensuring that the curve is nonsingular, so it defines an elliptic curve. This condition guarantees that E is smooth. In the affine plane ($z = 1$), we have:

$$E : y^2 = x^3 + ax + b, \quad a, b \in K. \quad (\text{A.32})$$

Remarque 2. (*Uniformizer uniqueness*)

A uniformizer is not unique. If t is a uniformizer at P , for all $u \in \bar{K}[E]_P^*$ a unite,

Ut is also a uniformizer.

Since we have:

$$\text{ord}_P(Ut) = \text{ord}_P(U) + \text{ord}_P(t) = 0 + 1 = 1. \quad (\text{A.33})$$

which is exactly the order of t .

Consider the elliptic curve over $\mathbb{F}_5$:

$$E : y^2 = x^3 - 3x - 2. \quad (\text{A.34})$$

and take the point $P = (-1, 0)$.

(a) **Verification that $P \in E$.**

$$(-1)^3 - 3(-1) - 2 = -1 + 3 - 2 = 0 = 0^2. \quad (\text{A.35})$$

Thus $P = (-1, 0) \in E$.

(b) **Tangent line and uniformizer at P .**

Differentiating $y^2 = x^3 - 3x - 2$ implicitly:

$$2y \frac{dy}{dx} = 3x^2 - 3 \implies \frac{dy}{dx} = \frac{3x^2 - 3}{2y}.$$

At $P = (-1, 0)$, the denominator vanishes ($y = 0$), so the slope is *undefined*: the tangent line is **vertical**. By the tangent-line rule:

Table A.1: Local uniformizers on an elliptical curve

N°	Situation	local Uniformisante t_P
1	Tangent not vertical (m defined, $y_P \neq 0$)	$t_P = x - x_P$
2	vertical Tangent ($y_P = 0$)	$t_P = y - y_P$
3	$P = P_\infty$	$t_P = x/y$

Since the tangent is vertical at $P = (-1, 0)$:

$$t_P = y - 0 = y.$$

We note that for determining the uniformizer of a given curve at a point, we can once of the following methods:

- **Using the tangent line (what we have done).**
- **Local development.**
- **The maximal ideal.**
- **variable change for the point at infinity.**

Tangent line method: to determine t_P as a uniformizer of a given point $P = (x_0, y_0)$, we proceed as follows:

- Write the curve as a function $f(x, y) = y^2 - x^3 - ax - b$.
- Verify $P \neq P_\infty$.
- Compute the slope m .
- If $m = \frac{dy}{dx}|_P \neq 0$, the tangent line is not vertical and the uniformizer will be $t_P = x - x_P$.
- If $m = 0$, the tangent line is vertical and the uniformizer will be $t_P = y - y_P$.
- If $P = P_\infty$: $t_P = x/y$.

Order of a function at a point

Definition A.6 (order of a function at a given point). (Silverman, 2009)

Let C be a curve and $P \in C$ a smooth point. The (normalized) valuation on $\bar{K}[C]_P$ is given by

$$\text{ord}_P : \bar{K}[C]_P \longrightarrow \{0, 1, 2, \dots\} \cup \{\infty\} \quad \text{ord}_P(f) = \sup\{d \in \mathbb{Z} \mid f \in \mathfrak{M}_P^d\}. \quad (\text{A.36})$$

The order at a point P on a curve is the discrete Valuation of this point P .

Property 2. Let $f, g \in \bar{K}(E)^*$ and $P \in E$.

1. $\text{ord}_P(fg) = \text{ord}_P(f) + \text{ord}_P(g)$
2. $\text{ord}_P(f + g) \geq \min(\text{ord}_P(f), \text{ord}_P(g))$
3. $\text{ord}_P(f) = \pm\infty \iff f = 0$

Definition A.7 (Zero and Pole).

Let $f \in \bar{K}(E)$ and $P \in E$ a smooth point.

- If $\text{ord}_P(f) > 0$: f has a zero at P , of order $\text{ord}_P(f)$.
- If $\text{ord}_P(f) < 0$: f has a pole at P , of order $-\text{ord}_P(f)$.
- If $\text{ord}_P(f) \geq 0$: f is regular at P and we can evaluate $f(P) \in \bar{K}$.

If t is a uniformizer at P , any function $f \in \bar{K}(E)^*$ is uniquely written as:

$$f = t^n \cdot u$$

with $u \in \bar{K}(E)_P$ and $n = \text{ord}_P(f) \in \mathbb{Z}$.

Example 6. Now let us define a function f on E by:

$$f(x, y) = x \quad \text{in the polynomial ring of } E.$$

f is well defined since every point of E has a coordinate x .

Since $t_P = y$, we have $y = t_P$. We substitute directly into the curve equation $y^2 = x^3 - 3x - 2$ and write

$$x = -1 + b_1 t_P + b_2 t_P^2 + O(t_P^3).$$

Then:

$$\begin{aligned} x^3 &= (-1 + b_1 t_P + b_2 t_P^2 + O(t_P^3))^3 \\ &= (-1)^3 + 3(-1)^2(b_1 t_P + b_2 t_P^2) + 3(-1)(b_1 t_P)^2 + O(t_P^3) \\ &= -1 + 3b_1 t_P + 3b_2 t_P^2 - 3b_1^2 t_P^2 + O(t_P^3), \\ -3x &= -3(-1 + b_1 t_P + b_2 t_P^2) = 3 - 3b_1 t_P - 3b_2 t_P^2 + O(t_P^3). \end{aligned}$$

Summing:

$$\begin{aligned}
 x^3 - 3x - 2 &= (-1 + 3b_1 t_P + 3b_2 t_P^2 - 3b_1^2 t_P^2) + (3 - 3b_1 t_P - 3b_2 t_P^2) - 2 + O(t_P^3) \\
 &= 0 + 0 \cdot t_P + (3b_2 - 3b_1^2 - 3b_2) t_P^2 + O(t_P^3) \\
 &= -3b_1^2 t_P^2 + O(t_P^3).
 \end{aligned}$$

The curve equation $t_P^2 = x^3 - 3x - 2$ then gives:

$$t_P^2 = -3b_1^2 t_P^2 + O(t_P^3).$$

Identifying at order t_P^2 :

$$-3b_1^2 = 1 \implies b_1^2 = -\frac{1}{3}.$$

Identifying at order t_P^3 (collecting the t_P^3 terms from $x^3 - 3x - 2$):

$$\begin{aligned}
 [t_P^3](x^3 - 3x - 2) &= 3(-1)^2 \cdot 0 + 3(-1) \cdot 2b_1 b_2 t_P^3 + b_1^3 t_P^3 + \dots \\
 &= (-6b_1 b_2 + b_1^3) t_P^3,
 \end{aligned}$$

and $[t_P^3] t_P^2 = 0$, so:

$$0 = -6b_1 b_2 + b_1^3 \implies b_2 = \frac{b_1^2}{6} = \frac{-1/3}{6} = -\frac{1}{18}.$$

So:

$$x = -1 + b_1 t_P - \frac{1}{18} t_P^2 + O(t_P^3), \quad b_1^2 = -\frac{1}{3}.$$

Remarque 3. Orders at $P = (-1, 0)$

- $\text{ord}_P(t_P) = \text{ord}_P(y) = 1$. (by definition)
- $\text{ord}_P(x) = 0$, since $x|_P = -1 \neq 0$.
- $\text{ord}_P(x + 1) = 1$, since $x + 1 = b_1 t_P + O(t_P^2)$ with $b_1 \neq 0$.

In particular, $y = t_P$ directly gives:

$$\boxed{\text{ord}_P(y) = 1,}$$

confirming that $t_P = y$ is indeed the uniformizer at $P = (-1, 0)$.

Summary at $P = (-1, 0)$ on $E : y^2 = x^3 - 3x - 2$:

$$t_P = y, \quad \text{ord}_P(t_P) = 1, \quad \text{ord}_P(x) = 0, \quad \text{ord}_P(x + 1) = 1.$$

Divisors: Generally

Definition A.8. (Divisor Group, General Definition)

The divisor group of a curve C , denoted $\text{Div}(C)$, is the free abelian group generated by the points of C . A divisor $D \in \text{Div}(C)$ is a formal sum:

$$D = \sum_{P \in C} n_P(P),$$

where $n_P \in \mathbb{Z}$ and $n_P = 0$ for all but finitely many $P \in C$. The degree of D is defined by:

$$\deg D = \sum_{P \in C} n_P.$$

The divisors of degree 0 form a subgroup of $\text{Div}(C)$, denoted:

$$\text{Div}^0(C) = \{D \in \text{Div}(C) \mid \deg D = 0\}.$$

Assume that C is smooth and let $f \in K(C)^*$. Then we can associate to f the divisor $\text{div}(f)$ given by:

$$\text{div}(f) = \sum_{P \in C} \text{ord}_P(f) (P).$$

Example 7. Assume $\text{char}(K) \neq 2$. Let $e_1, e_2, e_3 \in \bar{K}$ be distinct and consider the curve $C : y^2 = (x - e_1)(x - e_2)(x - e_3)$. C is smooth and has a single point at infinity P_∞ . For $i = 1, 2, 3$, let $P_i = (e_i, 0) \in C$. Then:

$$\text{div}(x - e_i) = 2(P_i) - 2(P_\infty), \quad \deg(f) = 2,$$

$$\text{div}(y) = (P_1) + (P_2) + (P_3) - 3(P_\infty).$$

Theorem A.1.

Given a rational function f over a smooth projective curve:

$$\deg(\text{div}(f)) = 0.$$

Thus the group of principal divisors of C is $\text{PDiv}(C) \subset \text{Div}^0(C)$.

Definition A.9. (Picard Group)

The Picard group of degree 0 is the quotient group:

$$\text{Pic}^0(C) = \text{Div}^0(C) / \text{PDiv}(C).$$

Theorem A.2. (Pic^0 Structure for Elliptic Curves)

Let E be an elliptic curve over K with the point $\mathcal{O}$. There exists a canonical isomorphism of groups:

$$E(K) \xrightarrow{\sim} \text{Pic}^0(E), \quad P \mapsto [P] - [\mathcal{O}].$$

Application: Divisor Associated to a Line

- If ℓ is the line which cuts E at P, Q, R , then:

$$\text{div}(\ell) = [P] + [Q] + [R] - 3[\mathcal{O}].$$

- If ℓ is the vertical line $x = x_0$ which cuts E at $P = (x_0, y_0)$ and $-P = (x_0, -y_0)$, then:

$$\text{div}(x - x_0) = [P] + [-P] - 2[\mathcal{O}].$$

Definition A.10. (n -Torsion Subgroup)

For $n \geq 1$, the subgroup of n -torsion of E is:

$$E[n] = \{P \in E(\bar{K}) \mid [n]P = \mathcal{O}\}.$$

Divisors over Elliptic Curve

Let E be a curve.

Definition A.11 (Divisor group). (Silverman, 2009) The divisor group of E , denoted $\text{Div}(E)$, is the free abelian group generated by the points of E :

$$\text{Div}(E) = \bigoplus_{P \in E(\bar{K})} \mathbb{Z} \cdot (P)$$

Definition A.12 (Divisor). (Silverman, 2009) A divisor $D \in \text{Div}(E)$ is the formal sum

$$D = \sum_{P \in E(\bar{K})} n_P \cdot (P)$$

where $n_P \in \mathbb{Z}$ and $n_P = 0$ p.p.

Definition A.13 (Degree). The degree of a divisor $D = \sum n_P(P)$ is

$$\deg(D) = \sum_{P \in E(\bar{K})} n_P \in \mathbb{Z}$$

The divisors of degree 0 are denoted by $\text{Div}^0(E)$ and it is a subgroup of $\text{Div}(E)$:

$$\text{Div}^0(E) = \{D \in \text{Div}(E) : \deg(D) = 0\}$$

Definition A.14 (Divisor of a function). (Silverman, 2009) Let $f \in \bar{K}[E]^*$. The divisor of f , denoted $\text{div}(f)$, is:

$$\text{div}(f) = \sum_{P \in E(\bar{K})} \text{ord}_P(f) \cdot (P)$$

Property 3. *Divisor properties* Let $f, g \in \bar{K}(E)^*$.

1. $\text{div}(fg) = \text{div}(f) + \text{div}(g)$
2. $\text{div}(f/g) = \text{div}(f) - \text{div}(g)$

Definition A.15. A divisor D is said principal if there exists $f \in \bar{K}[E]^*$: $D = \text{div}(f)$ with $\deg(D)$ is always null.

The set of principal divisors is denoted by $\text{Prin}(E)$ and it is a subgroup of $\text{Div}^0(E)$.

Then we have this inclusion:

$$\text{Prin}(E) \subset \text{Div}^0(E) \subset \text{Div}(E)$$

Property 4. *Properties of Divisors of Functions* Indeed: $\text{Prin}(E) \subset \text{Div}^0(E)$ because: by taking $P, Q \in E$ such that $Q \neq -P$, we have $(Q) + (P)$ has degree 0, but is not principal: $Q \neq -P \Rightarrow Q + P = R (\neq \mathcal{O})$, so $(Q) + (P) + (R) - 3(\mathcal{O})$ will have 0 as degree.

$$\text{Prin}(E) \subset \text{Div}^0(E)$$

Property 5. Let E be a smooth curve and $f \in \bar{K}(E)^*$ with $f \neq 0$.

1. f has a finite number of zeros and poles over E .
2. $\deg(\operatorname{div}(f)) = 0$: the number of zeros of f is equal to the number of its poles.
3. If f has any pole, then $f \in \bar{K}$.

The property 2 is the most used in practice. It allows to see if a set of points can define the divisor of a function.

Linear equivalence and Picard group

Let D_1 and D_2 be two divisors of E . D_1 and D_2 are said **equivalents**, and we denote $D_1 \sim D_2$ if $D_1 - D_2$ is a principal divisor:

$$D_1 \sim D_2 \implies \exists f \in \bar{K}(E)^* : D_1 - D_2 = \operatorname{div}(f)$$

Definition A.16 (Picard group). (Clarisse, 2021)

The picard group is the group of divisors of degree zero $\operatorname{Div}^0(E)$ modulo the principal divisors over E :

$$\operatorname{Pic}^0(E) = \operatorname{Div}^0(E) / \operatorname{Prin}(E)$$

Theorem A.3. *Abel Jacobi Criteria for elliptic curve*

Let E be an elliptic curve over K , and $D = \sum n_P(P) \in \operatorname{Div}(E)$.

Then $D \in \operatorname{Prin}(E)$ if and only if the following both conditions are satisfied:

$$\sum_P n_P = 0 \quad \text{and} \quad \sum_P [n_P]P = \mathcal{O} \quad \text{over } E$$

Proof. A.2

□

Let consider $E : y^2 = x^3 + ax + b$ and $P, Q \in E$, such that $P + Q \neq \mathcal{O}$.

Denotes ℓ , the line passing by P and Q , and $R = -(P + Q)$, the intersection point between ℓ and E .

Then $\operatorname{div}(\ell) = (P) + (Q) + (R) - 3(\mathcal{O})$.

Verification:

$$\deg = 3 - 3 = 0$$

$$P + Q + R - 3\mathcal{O} = P + Q - P - Q - 3\mathcal{O} = -2\mathcal{O} \neq \mathcal{O}$$

Riemann-Roch Theorem

Link Elliptic Curve and Picard Group:

From the Abel criteria, there exists an isomorphism φ :

$$\varphi : E(K) \xrightarrow{\sim} \operatorname{Pic}^0(E)$$

$$\varphi : P \longmapsto [(P) - (\mathcal{O})]$$

The Riemann-Roch theorem controls the dimension of the space of functions having poles,

and is indispensable as tools to demonstrate that each elliptic curve has a Weierstrass equation, to justify the existence of functions f_T used in pairings.

A divisor $D = \sum n_P(P) \in \text{Div}(E)$ is said positive, denoted $D \geq 0$, if $n_P \geq 0$, $\forall P \in E$. For two divisors D_1 and D_2 , if $D_1 - D_2 \geq 0$. Let $D \in \text{Div}(E)$. We define the K -space of Riemann-Roch

$$L(D) = \{f \in \overline{K}(E)^* : \text{div}(f) \geq -D\}.$$

$f \in L(D)$ meaning that f has pole only at P only if $n_P > 0$, and the order of this pole of f is at most n_P . $L(D) = \{f \in \overline{K}(E)^* : \text{div}(f) \geq -D\} \cup \{0\}$.

The set $L(D)$ is a finite-dimensional $\overline{K}$ -vector space.

We note $\ell(D) = \dim_K L(D)$ its dimension.

The inequality $\text{div}(f) \geq -D$ is read point by point. Meaning that, $\forall P \in E$, we should have:

$$\text{ord}_P(f) \geq -n_P$$

Thus,

- if $n_P > 0$: $\text{ord}_P(f) \geq -n_P$, the pole is of order at most n_P . Exple: $D = 3(P)$; $\text{ord}_P(f) \geq -3$. f can have order: 1, 2, 3.
- if $n_P = 0$: $\text{ord}_P(f) \geq 0$, there is not pole. f is regular.
- if $n_P < 0$: $\text{ord}_P(f) \geq -n_P$, f has zero of order at least $-n_P$.

Property 6. Divisors properties

Let $D \in \text{Div}(C)$.

1. If $\deg D < 0$, then $L(D) = \{0\}$ and $\ell(D) = 0$.
2. $L(D)$ is a K -vectorial space of finite Dimension.
3. If $D \sim D'$ then $L(D) \cong L(D')$ and $\ell(D) = \ell(D')$.
4. If D is defined over k , then $L(D)$ has a basis in $K(C)$.

For $C = \mathbb{P}^1$ and $D = n(\infty)$ with $n \geq 0$.

$$L(n(\infty)) = \{f \in K(t) \mid f \text{ is a polynomial at } t \text{ of degree } \leq n\}$$

Thus $\ell(n(\infty)) = n + 1$, the natural basis is $\{1, t, t^2, \dots, t^n\}$.

Definition A.17. Definition: Canonical divisor

Let ω be a differential over C , and t a uniformizer at P : $\omega = f dt$, $f \in K(C)$.

Assume $\text{ord}_P(\omega) = \text{ord}_P(f)$.

$$\text{div}(\omega) = \sum_{P \in C} \text{ord}_P(\omega) \cdot (P)$$

The linear equivalence class of $\text{div}(\omega)$ is the canonical class $[K_C] \in \text{Pic}(C)$.

All representing $K_C \in [K_C]$ is called canonical divisor.

Theorem A.4 (Riemann-Roch). Let E be a smooth curve and K_E a canonical divisor over E .

There exists an integer $g \geq 0$, called genus of E , such that, for all divisor $D \in \text{Div}(E)$:

$$\ell(D) - \ell(K_E - D) = \deg D - g + 1$$

Proof. A.2 □

Corollary A.1. Let C be a curve of genus g and K_C a canonical divisor.

1. $\ell(K_C) = g$
2. $\deg K_C = 2g - 2$
3. If $\deg D > 2g - 2$, then $\ell(D) = \deg D - g + 1$
4. ($g = 0, C \cong \mathbb{P}^1$): $K_C = -2(\infty)$ and $\deg D \geq -1$:

$$\ell(D) = \deg D + 1$$
5. ($g = 1$, i.e E elliptic curve): $K_E = 0$ and for $\deg D \geq 1$: $\ell(D) = \deg D$.

Which conducts us to the following theorem. But before, note that if the curve is a **supersingular**, then the Frobenius trace will be null and we obtain just $p + 1$ roots!

a- Existence of Weierstrass equation

Let E be a smooth curve of genus 1 with the point at infinity $\mathcal{O}$. Then, E admits a Weierstrass equation:

$$y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6$$

with $a_i \in K$. In particular, $(E, \mathcal{O})$ is an elliptic curve.

Construction via Riemann-Roch using sagemath:

- Since $g = 1$, Corollar (3), for all divisor D :

$$\ell(D) = \deg(D) - g + 1 \quad \text{if } \deg(D) > 2g - 2$$

$g = 1$, thus $\ell(n\mathcal{O}) = n, \forall n \geq 1$.

- $L(\mathcal{O})$:

$D = 0$, thus $\ell(\mathcal{O}) = 1$, it means that $L(\mathcal{O})$ contains one dimension. Thus only constant functions.

- $L(2\mathcal{O})$:

Riemann-Roch gives $\ell(2\mathcal{O}) = 2$, espace of dimension 2.

Contains 1 and non constant function with pole ≤ 2 at $\mathcal{O}$: x .

A basis is $\{1, x\}$ and $\text{ord}_{\mathcal{O}}(x) = -2$.

- $L(3\mathcal{O})$:

Riemann-Roch: $\ell(3\mathcal{O}) = 3$.

Contains $1, x, y$: $\text{ord}_{\mathcal{O}}(y) = -3$.

$$L(3\mathcal{O}) = \text{vect}\{1, x, y\}$$

• $L(6\mathcal{O})$:

Riemann-Roch: $\ell(6\mathcal{O}) = 6$.

$\text{ord}_{\mathcal{O}}(x) = -2, \text{ord}_{\mathcal{O}}(y) = -3$.

Construction via Riemann-Roch using Sagemath:

Thus:

Table A.2: Orders of some rational functions in P_{∞}

N°	Fonction	Order
1	1	0
2	x	-2
3	y	-3
4	x^2	-4
5	xy	$-5 = -2 - 3$
6	x^3	-6
7	y^2	-6

All has a pole ≤ 6 . Thus are in $L(6\mathcal{O})$.

$\dim L(6\mathcal{O}) = 6$, but we have 7 functions. Then, they cannot be independants. Means that there exists $A, B, C, \dots F, G$:

$$Ay^2 + Bxy + Cy + Dx^3 + Ex^2 + Fx + G = 0$$

Construction via Riemann-Roch using Sagemath: By reorganizing, we obtain:

$$y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6 \quad (1)$$

where $a_1 = \frac{B}{-AD}$, $a_3 = \frac{C}{-AD}$, $a_2 = \frac{E}{AD}$, $a_4 = \frac{F}{AD}$ and $a_6 = \frac{G}{AD}$. And (1) is exactly the Weierstrass equation. If E is smooth, of genus 1, $(E, \mathcal{O})$ is an elliptic curve.

b- Existence of functions f_T .

Let E be an elliptic curve of genus 1 and $T \in E[m]$ (m-torsion point). Then the divisor $D = m(T) - m(\mathcal{O})$ is such that $\ell(D) \geq 1$, this guarantees the existence of a function $f_T \in \bar{K}(E)^*$ with $\text{div}(f_T) = m(T) - m(\mathcal{O})$.

Table A.3: Recapitulatif table

N°	Genre	Type of curve	$\deg K_C$	$\ell(D)$ for $\deg D \geq 1$	Key Properties
1	$g = 0$	$\mathbb{P}^1$	-2	$\deg D + 1$	$D \in \text{Div}^0(C) \Rightarrow D \in \text{Prin}(C)$
2	$g = 1$	elliptic curve E	0	$\deg D$	$K_E = 0, E \cong \text{Pic}^0(E)$
3	$g \geq 2$	Hyperelliptic, etc.	$2g - 2$	$\deg D - g + 1$	Presence of holomorphic differential

Order of torsion point and associated function

The construction of Weil pairing is under the function which divisor is imposed.

The theorem of Abel-Jacobi allows to verify the existence and to control the unicity.

Property: Principal Divisor Associated to a Torsion Point

A.2 Proofs

Proof. Let $D = m(T) - m(\mathcal{O})$.

We verify the following properties:

1. The degree of D is zero:

$$\deg(D) = m - m = 0.$$

2. The sum of the points in D is trivial in the group law:

$$\sum n_P P = mT - m\mathcal{O} = [m]T = \mathcal{O},$$

since $T \in E[m]$.

By the theorem on principal divisors on elliptic curves, D is a principal divisor. Hence, there exists a function $f_T \in \bar{K}(E)^*$ such that:

$$\operatorname{div}(f_T) = D.$$

The function is unique up to multiplication by a nonzero constant. □

Property: Provided Divisor Function

Proof. The Abel-Jacobi theorem guarantees the existence of f_T .

Unicity of the function f_T :

Suppose $f, g \in \bar{K}(E)^*$ satisfy $\operatorname{div}(f) = \operatorname{div}(g)$.

Consider the quotient $\frac{f}{g}$.

Then the divisor of the quotient is $\operatorname{div}\left(\frac{f}{g}\right) = \operatorname{div}(f) - \operatorname{div}(g) = 0$.

A function with zero divisor is constant, i.e.,

$$\frac{f}{g} \in \bar{K}^*.$$

Hence $f = c \cdot g$ for some $c \in \bar{K}^*$,

which proves that f_T is unique up to multiplication by a non-zero constant. □

Proof. Assume that $D' = m(T)$, then $\deg(D') = m$.

The Riemann-Roch theorem tell us that:

$$\ell(D) = \deg(D) - g + 1$$

If $\deg(D) > 2g - 2$.

Here, $g = 1$, $2g - 2 = 0$, $\deg(D') = m \geq 1$.

Thus $\ell(m(T)) = m - 1 + 1 = m$, i.e $\dim L(m(T)) = m$.

By definition, $L(m(T)) = \{f \in K(E) \mid \text{div}(f) + m(T) \geq 0\}$.

Means that:

- pole of f : T . $\text{ord}_T(f) \geq -m$.
- order at most m .

Otherwise, $L((m-1)T) = \{f \in K(E) : \text{div}(f) + (m-1)(T) \geq 0\}$:

$$\ell((m-1)(T)) = m-1$$

$$\dim L(m(T)) = m, \quad \dim((m-1)(T)) = m-1$$

$$\Rightarrow L((m-1)(T)) \subsetneq L(m(T))$$

Thus there exists $f \in L(m(T))$:

$$f \notin L((m-1)(T))$$

Order of pole:

$f \notin L((m-1)(T))$ means that $\text{ord}_T(f) < -(m-1)$.

$f \in L(m(T)) \Rightarrow \text{ord}_T(f) \geq -m$.

i.e $\text{ord}_T(f) = -m$.

$\deg(\text{div}(f)) = 0$.

If f has a pole of order m at T , then the zeros will compass.

Thus $\text{div}(f) = m(T) - Z$. Z divisor of degree m .

Over an elliptic curve: $\sum n_P P = \mathcal{O}$ means that $\sum n_P P$ is a principal divisor.

We take $D = m(T) - m(\mathcal{O})$. We will have $mT - m\mathcal{O}$:

$mT = \mathcal{O}$, since $T \in E[m]$.

$mT - m\mathcal{O} = \mathcal{O}$.

There exists a rational function

$$f_T \in K(E)^*$$

such that

$$\text{div}(f_T) = m(T) - m(\mathcal{O}).$$

□

Proof. We know that $\text{div}(f_T) = m(T) - m(\mathcal{O})$.

$$\text{div}(f_T \circ [m]) = [m]^*(\text{div}(f_T)) = [m]^*(m(T) - m(\mathcal{O}))$$

$$= m[m]^*(T) - m[m]^*(\mathcal{O})$$

$$= m\widetilde{[m]}^*(T) - m[m]^*(\mathcal{O}) \quad \text{by linearity}$$

By definition of “tiré en arrière d’un point” by $[m]$:

$$[m]^*(T) = \sum_{R \in E[m]} (T' + R), \text{ where } T' \text{ verifies } [m]T' = T$$

The same $[m]^*(\mathcal{O}) = \sum_{R \in E[m]} (R)$.

Thus:

$$\operatorname{div}(f_T \circ [m]) = m \left(\sum_{R \in E[m]} (T' + R) - (R) \right)$$

Otherwise, g is chosen to satisfy:

$$\operatorname{div}(g) = \sum_{R \in E[m]} (T' + R) - (R)$$

Thus:

$$\operatorname{div}(g^m) = m \operatorname{div}(g) = m \left(\sum_{R \in E[m]} (T' + R) - (R) \right)$$

Comparison:

$$\operatorname{div}(f_T \circ [m]) = \operatorname{div}(g^m)$$

Two functions having same divisor differ by a constant:

$$f_T \circ [m] = c \cdot g^m$$

By normalization, we choose $c = 1$ and obtain the relation: $f_T \circ [m] = g^m$

□

Proof. Riemann-Rock

1. Indeed, assume that $\deg D < 0$.

By contradiction, assume that $f \neq 0$ in $L(D)$.

$f \in L(D) \Rightarrow \operatorname{div}(f) \geq -D$ by definition (Means that $\operatorname{ord}_P(f) \geq -n_P$).

Means that $\deg(\operatorname{div}(f)) \geq \deg(-D)$.

$\forall f \in \bar{K}(E)^*$, $\deg(\operatorname{div}(f)) = 0$, since the number of zeros is egale to the number of poles.

Thus $\deg(-D) \leq 0$.

$\Rightarrow -\deg(D) \leq 0$.

Thus $\deg(D) \leq 0$, contradiction, since $\deg(D) < 0$ by assumption. Thus $L(D) = \{0\}$ and $\ell(D) = 0$.

2. • $f, g \in L(D) \Rightarrow \operatorname{ord}_P(f) \geq -n_P, \operatorname{ord}_P(g) \geq -n_P$.

Otherwise $\operatorname{ord}_P(f + g) \geq \min(\operatorname{ord}_P(f), \operatorname{ord}_P(g)) \geq -n_P$.

i.e $\operatorname{ord}_P(f + g) \geq -n_P$, i.e $f + g \in L(D)$.

- The same $\forall \lambda \in K, \lambda f \in L(D)$.

Thus $L(D)$ is a k -vectorial espace.

3. $D' \sim D \Rightarrow D' = D + \operatorname{div}(g), g \in \bar{K}(C)^*$.

Consider $f \in L(D)$. Then $\operatorname{div}(f) \geq -D$ and also $h = \frac{f}{g}$.

$$\operatorname{div}(h) = \operatorname{div}(f) - \operatorname{div}(g) \geq -D - \operatorname{div}(g) = -D'$$

Thus $\text{div}(h) \geq -D' \Rightarrow h \in L(D')$.

Thus there is an isomorphism π :

$$\pi : L(D) \longrightarrow L(D'), \quad f \longmapsto \frac{f}{g} \quad \text{and } \ell(D) = \ell(D') \text{ since } \pi \text{ is an isomorphism}$$

4. D defined over k means that the coefficients of the divisor are stable under the action of K .

Thus we can choose rational functions with coefficients in k , to form a basis.

□

Maximality of $\mathfrak{m}_P$:

Indeed, considering the morphism:

$$\varphi : \bar{K}[E] \longrightarrow K, \quad f \longmapsto \varphi(f) = f(P)$$

We have:

- $\ker \varphi = \mathfrak{m}_P$
- $\text{Im } \varphi = k$, since for $a \in k$ there exists $f : f(x) = a$, with $x \in E$. Thus $\varphi(f) = a$.

From the isomorphism theorem:

$$\frac{\bar{K}[E]}{\ker \varphi} \cong k$$

Since k is a field, $\bar{K}[E]/\ker \varphi$ is a field too.

Meaning that $\ker \varphi = \mathfrak{m}_P$ is a maximal ideal.

$\mathfrak{m}_P$ is prime because $\forall fg \in \mathfrak{m}_P, f \in \mathfrak{m}_P$ or $g \in \mathfrak{m}_P$.

Thus $\bar{K}[E]/\mathfrak{m}_P$ is an integer domain.

Thus $\mathfrak{m}_P$ is prime.

Proof. Let

$$S(X) = \sum_{i=0}^n L_i(X).$$

The polynomial $S(X)$ has degree at most n . Moreover, for every $j \in \{0, \dots, n\}$,

$$S(a_j) = \sum_{i=0}^n L_i(a_j) = \sum_{i=0}^n \delta_{ij} = 1.$$

Since the constant polynomial 1 also has degree at most n and takes the value 1 at the $n+1$ distinct points $a_0, \dots, a_n$, the uniqueness of Lagrange interpolation implies that

$$S(X) = 1.$$

□

signature proof. We verify that

$$e(v, h) = e(g, \sigma).$$

The formal verification is given in:

$$\begin{aligned}
e(v, h) &= e(g^x, H(M)) \\
&= e(g, H(M))^x \quad (\text{using the 1st given by Equation (3.29)}) \\
&= e(g, H(M)^x) \quad (\text{by (3.29)}) \\
&= e(g, h^x), \quad \text{since } h = H(M) \\
&= e(g, \sigma), \quad \text{by definition of } \sigma = h^x.
\end{aligned}$$

□

Batch verification. By replacing U :

$$\begin{aligned}
U &= \prod_{i=1}^n (H(M)^{x_i})^{c_i} . \\
U &= H(M)^{\sum_{i=1}^n c_i x_i} .
\end{aligned}$$

The same,

$$V = \prod_{i=1}^n (g_2^{x_i})^{c_i} = g_2^{\sum_{i=1}^n c_i x_i} .$$

Then,

$$e(U, g_2) = e(H(M)^{\sum_{i=1}^n c_i x_i}, g_2) = e(H(M), g_2)^{\sum_{i=1}^n c_i x_i} .$$

and,

$$e(H(M), V) = e(H(M), g_2^{\sum_{i=1}^n c_i x_i}) = e(H(M), g_2)^{\sum_{i=1}^n c_i x_i} .$$

Consequently,

$$e(U, g_2) = e(H(M), V) .$$

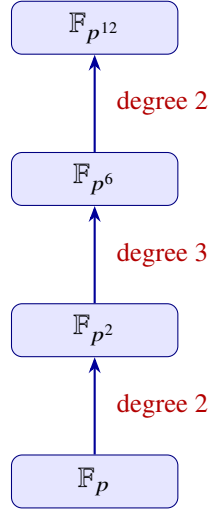
and we hold.

□

A.2.1 Arithmetic in Extension Fields $\mathbb{F}_p^4$ and $\mathbb{F}_p^6$

The analysis in this section is based on the documents (Ghammam, 2016), (Ghammam & Fouotsa, 2016). The calculation of bilinear pairings requires arithmetic in the extension fields of the base field F_p . For the curves $BN254$ and $BLS12-381$, this field is $F_{p^{12}}$, which is constructed by successive extension towers. The efficiency of the pairings depends on

Figure A.2: Extension Tower



the algorithms used to perform the arithmetic within an extension tower.

An extension of degree k of $\mathbb{F}_p$, denoted $\mathbb{F}_{p^k}$, is constructed as a quotient ring:

$$\mathbb{F}_{p^k} \cong \mathbb{F}_p[X]/\langle P(X) \rangle \quad (\text{A.37})$$

where $P(X) \in \mathbb{F}_p[X]$ is an irreducible polynomial of degree k . A representation of the extension tower is given by A.2.

The cost of a Frobenius a^{p^i} is: 15M if i is odd; 8M if i is even.

The most important arithmetic operations are **addition**, **multiplication**, **square**, **Frobenius** and **inverse**. We denote them respectively by A_i , M_i , S_i , F_i , I_i . Several methods are used notably **Karatsuba**, **Schoolbook**, and **Complex** method.

Table A.4: Synthetic Table: Operation Costs in Extension Fields

Extension Type	Operation	Method	Theoretical Cost
Quadratic ($\mathbb{F}_{p^{2i}}$)	Multiplication	Schoolbook	$4M_i + m_{i,\mu} + 2A_i$
		Karatsuba	$3M_i + m_{i,\mu} + 5A_i$
	Squaring	Schoolbook	$M_i + 2S_i + 2A_i + m_{i,\mu}$
		Karatsuba	$3S_i + 4A_i + m_{i,\mu}$
		Complex	$2M_i + 3A_i + A'_i + m_{i,\mu} + m_{i,\mu+1}$
Cubic ($\mathbb{F}_{p^{3i}}$)	Multiplication	Schoolbook	$9M_i + 6A_i + 2m_{i,\xi}$
		Karatsuba	$6M_i + 15A_i + 2m_{i,\xi}$
		Sparse ($y_2 = 0$)	$5M_i + 6A_i + m_{i,\xi}$
	Squaring	Schoolbook	$3M_i + 3S_i + 3A_i + A'_i + 2m_{i,\xi}$
		Karatsuba	$6S_i + 12A_i + 2m_{i,\xi}$
		Chung–Hasan	$2M_i + 3S_i + 8A_i + A'_i + 2m_{i,\xi}$

Legend: M = field multiplication; S = squaring; A = addition; A' = subtraction; $m_{i,\mu}$ = multiplication by the constant μ ; $m_{i,\xi}$ = multiplication by the non-residue ξ . The subscript i denotes operations in the base field $\mathbb{F}_{p^i}$.

Method choice

We observe that the optimal choice between the karatsuba and chung-hasan methods depends on the cost between multiplication and addition. If $M_i \sup 3A_i$, the Karatsuba method become more efficient.

Optimization

For calculating pairings, using sparse multiplications (those with many zeros) in cubic extensions reduces the number of multiplications from 9 to 5 when $y_2 = 0$, for example. This reduces the computational cost of the Miller loop. The inverse operation is the most complex and costly operation in $\mathbb{F}_{p^i}$ and it is optimized by computing the norm N of an element x : We deduce the inverse of any element $x \in \mathbb{F}_{p^{2i}}$ by :

$$x^{-1} = (x_0 + x_1\alpha)^{-1} = \frac{x_0}{x_0^2 - \mu x_1^2} - \frac{x_1}{x_0^2 - \mu x_1^2} \alpha. \quad (\text{A.38})$$

$$N = x_0^2 - \mu x_1^2, \quad (\text{A.39})$$

The choice of the elliptic curve is a critical step for the efficiency of the pairings that will depend on it. In (A.5), we compare the most commonly used elliptic curves in SNARK technologies.

Table A.5: Comparison of the pairing-friendly curves BN254 and BLS12-381

Characteristic	BN254	BLS12-381
Curve family	Barreto-Naehrig (BN)	Barreto-Lynn-Scott (BLS12)
Base field size	254-bit prime field	381-bit prime field
Main application	Pairing-based cryptography	Pairing-based cryptography, zk-SNARKs, Ethereum
Security level	Approximately 100-110 bits after the exTNFS attack	Approximately 128-bit security
Advantages	Fast pairing computation	Higher security and long-term deployment
Limitations	Security weakened by the exTNFS attack	Slightly higher computational cost
Current status	Mostly considered legacy	Recommended for modern applications

Table A.6: Mathematical parameters of BN254 and BLS12-381

Parameter	BN254	BLS12-381
Embedding degree k	12	12
Prime field	$\mathbb{F}_p$	$\mathbb{F}_p$
Curve equation	$y^2 = x^3 + b$	$y^2 = x^3 + b$
Parameter u	Sparse negative integer	Sparse negative integer
Subgroup order	Prime r	Prime r
Trace	t	t
Extension field	$\mathbb{F}_{p^{12}}$	$\mathbb{F}_{p^{12}}$

Long considered the standard, analyses have shown that BN256 curves have a reduced security level compared to BLS12-381 curves. The current standard for applications requires a 128-bit security level. (A.7) provides details concerning the security of some of the most commonly used elliptic curves. The parameters directly impacting the cost of the final exponentiation are p, r, t and u (Wang et al., 2023). curves where: are preferable, because they minimize the size of the basic body, while maintaining sufficient safety.

Table A.7: Comparison of different curves favorable to pairings.

Curve	A	$\log_2(\mathbf{B})$	$\log_2(\mathbf{N_{f1}})$	$\log_2(\mathbf{N_{f2}})$	Security (bits)
BN256	176	57.6	424.80	466.51	100
BN446	993	72.2	538.37	799.88	129
BN512	1541	76.6	572.71	888.52	138
BN638	3666	85.2	633.42	1091.20	153
BLS12-381	639	68.9	725.19	494.18	123
BLS12-440	929	72.8	765.72	559.31	129
BLS12-462	1067	73.6	780.93	582.23	131
BLS24-479	7	103.0	1155.60	1254.60	187
BLS24-559	9	108.6	1248.04	1455.46	187

Aside from the security level of the curve, for a blockchain, one must also consider the cost of pairings, the size of signatures, the size of keys, compatibility with existing libraries, and verification time. Very high-security curves require larger integers. This leads to increased memory requirements, slower calculations, and larger signatures and proofs.

This explains the choice and standardization of the BLS12-381 curve in bold in A.7

A.3 Practices

The accompanying GitHub repository provides the complete LaTeX source code of this dissertation together with the implementation experiments, benchmarking programs, and the corresponding experimental results used to support the analyses presented in this work. It is available at:

<https://github.com/ABDULASISU/My-dissertation>